

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG AIOT GIÁM SÁT GIAO THÔNG THÔNG MINH VÀ PHẠT NGUỘI TỰ ĐỘNG

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **NGUYỄN TIẾN KHÔI**

MSSV: 22119192

NGHIÊM ĐẠI NGÂN

MSSV: 22119201

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG AIOT GIÁM SÁT GIAO THÔNG
THÔNG MINH VÀ PHẠT NGUỘI TỰ ĐỘNG**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **NGUYỄN TIẾN KHÔI**

MSSV: 22119192

NGHIÊM ĐẠI NGÂN

MSSV: 22119201

Hướng dẫn: **ThS. NGUYỄN NGÔ LÂM**

TP. HỒ CHÍ MINH – 06/2026

 TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH Ho Chi Minh City University of Technology and Engineering hcmute.edu.vn	CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM <u>Độc lập – Tự do – Hạnh phúc</u> TP. Thủ Đức, ngày 21 tháng 06 năm 2026
---	---

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Nguyễn Tiến Khôi – Nghiêm Đại Ngân

MSSV: 22119192 – 22119201

Ngành: Công nghệ kỹ thuật máy tính

Lớp: 22119A

Giảng viên hướng dẫn: Th.S Nguyễn Ngô Lâm

Ngày nhận đề tài: 02/03/2026

Ngày nộp đề tài: 26/06/2026

1. Tên đề tài: Hệ thống AIoT giám sát giao thông thông minh và phạt nguội tự động

2. Các số liệu, tài liệu ban đầu

- Kiến thức cơ bản về các môn Mạch điện, Điện tử cơ bản, Hệ thống nhúng, Học máy và trí tuệ nhân tạo, Học máy ứng dụng.
- Lý thuyết về IoT và Điện toán biên.
- Lý thuyết các giao thức truyền dữ liệu và các dịch vụ API RESTful HTTP.
- Tài liệu datasheet, thông số kỹ thuật đi kèm của các linh kiện sử dụng trong đề tài.
- Lập trình ESP32, Raspberry Pi 4 và Antigravity IDE.

3. Nội dung thực hiện đề tài:

- Tìm hiểu về các giao thức truyền dữ liệu.
- Tìm hiểu nguyên lý hoạt động của các vi điều khiển, module sử dụng trong đề tài.
- Xây dựng và lựa chọn mô hình AI nhận diện phương tiện và mô hình AI nhận diện biển số xe Việt Nam.
- Thiết kế giao diện tương tác quản lý thông tin phương tiện vi phạm.
- Tối ưu hóa mô hình AI chạy trên thiết bị nhúng.
- Viết báo cáo.

4. Sản phẩm

- Mô hình phần cứng sa bàn.
- Quyển báo cáo đề tài.

GIẢNG VIÊN HƯỚNG DẪN

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: **HỆ THỐNG AIOT GIÁM SÁT GIAO THÔNG THÔNG MINH
VA PHẠT NGUỘI TỰ ĐỘNG**

Sinh viên: + **NGUYỄN TIẾN KHÔI** MSSV: **22119192**
+ **NGHIÊM ĐẠI NGÂN** MSSV: **22119201**
+ **ThS. NGUYỄN NGÔ LÂM**
Hướng dẫn:

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

phân tích

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

phù hợp

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

hợp lý

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

C' trích dẫn tài liệu tham khảo

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

chưa phát hiện sự trùng lặp

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Nhà tài liệu nghiên cứu chưa hoàn thiện

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

SV có tinh thần học tập, nghiên cứu tích cực

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Đạt yêu cầu

Tp. HCM, ngày 25 tháng 6 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

Nguyễn Ngô Lâm

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong quá trình học tập và thực hiện Đồ án tốt nghiệp tại Trường Đại học Công nghệ Kỹ Thuật Thành phố Hồ Chí Minh, chúng tôi đã nhận được nhiều sự giúp đỡ, hỗ trợ từ các Thầy, Cô, bạn bè và gia đình. Nhân dịp hoàn thành đồ án này, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến tất cả mọi người đã giúp đỡ chúng tôi trong suốt thời gian qua.

Trước hết, chúng tôi xin gửi lời cảm ơn chân thành đến các Thầy, Cô trong khoa Điện – Điện tử. Những kiến thức chuyên môn, kinh nghiệm thực tiễn cùng sự tận tâm trong công tác giảng dạy của quý Thầy, Cô đã giúp chúng tôi có nền tảng vững chắc để hoàn thành đồ án này. Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến Th.S Nguyễn Ngô Lâm, giảng viên hướng dẫn đã luôn tận tình đồng hành trong suốt quá trình thực hiện đồ án. Thầy không chỉ định hướng nội dung nghiên cứu, góp ý chuyên môn và hỗ trợ giải quyết những khó khăn phát sinh, mà còn truyền cho chúng tôi tinh thần làm việc nghiêm túc, tư duy khoa học và sự kiên trì trong nghiên cứu. Những ý kiến đóng góp quý báu và sự hướng dẫn tận tâm của thầy là nguồn động lực to lớn giúp chúng tôi từng bước hoàn thiện đề tài một cách hiệu quả và đúng tiến độ.

Bên cạnh đó, chúng tôi xin gửi lời cảm ơn đến các nhà nghiên cứu, các tác giả trong lĩnh vực thị giác máy tính, nhận dạng phương tiện, hệ thống giám sát giao thông thông minh và xử lý vi phạm giao thông. Những công trình nghiên cứu, tài liệu khoa học và nguồn tham khảo có giá trị đã giúp chúng tôi tiếp cận kiến thức, định hướng ý tưởng và hoàn thiện đề tài một cách hiệu quả hơn.

Mặc dù đã nỗ lực hết sức trong quá trình thực hiện, song do hạn chế về thời gian nghiên cứu cũng như kinh nghiệm thực tiễn, báo cáo không tránh khỏi những thiếu sót nhất định, chúng tôi rất mong nhận được sự góp ý quý báu của tất cả các Thầy, Cô để bài báo cáo của chúng tôi được hoàn thiện hơn.

Nhóm thực hiện

TÓM TẮT

Trong kỷ nguyên đô thị hóa hiện đại, sự bùng nổ phương tiện giao thông cá nhân dẫn tới các thách thức nghiêm trọng về ùn tắc và mất an toàn giao thông tại các giao lộ lớn. Nhằm giải quyết vấn đề này, đề án tốt nghiệp tập trung nghiên cứu và thiết kế Hệ thống AIoT Giám sát Giao thông Thông minh và Phạt nguội Tự động. Đây là một hệ thống phân tán tính toán thực tế hoạt động theo thời gian thực chia thành ba tầng vật lý: Tầng thực thi thiết bị đầu cuối (vi điều khiển ESP32), Tầng xử lý tại biên (Edge Gateway Raspberry Pi 4), và Tầng máy chủ tập trung (Cloud/Laptop Server).

Tại tầng biên, Raspberry Pi 4 ứng dụng mạng học sâu YOLOv11n (đã lượng tử hóa qua NCNN) kết hợp thuật toán bám vết ByteTrack để nhận diện và đếm lưu lượng phương tiện. Dựa trên số liệu này, hệ thống tự động nội suy chu kỳ đèn tín hiệu thích ứng và điều phối ESP32 thông qua giao thức MQTT. Đồng thời, mô-đun phạt nguội giám sát liên tục vạch dừng ảo; khi phát hiện lỗi vượt đèn đỏ, hệ thống sẽ cắt ảnh vi phạm, áp dụng biến đổi không gian Homography để dán biển số giả lập nhằm giải quyết bài toán thiếu hụt dữ liệu, sau đó truyền bất đồng bộ qua HTTP RESTful (FastAPI) lên máy chủ. Tại tầng trung tâm, luồng dữ liệu được xử lý bởi mô hình OCR (kết hợp YOLO và mạng chuỗi CRNN) để trích xuất biển số và đồng bộ hóa lên giao diện quản lý Streamlit Dashboard.

Để đảm bảo hệ thống vận hành mượt mà trên phần cứng nhúng có tài nguyên hạn hẹp, đề án đã triển khai bộ ba kỹ thuật tối ưu hóa: lượng tử hóa mô hình NCNN, kiến trúc đa luồng (Producer-Consumer) tách biệt luồng I/O camera, và kỹ thuật lược bỏ khung hình (Frame Skipping). Kết quả thực nghiệm chứng minh hệ thống giảm tải thành công 50% tài nguyên CPU cốt lõi trong khi vẫn duy trì quỹ đạo bám vết ổn định, đáp ứng toàn diện các tiêu chuẩn của một giải pháp giao thông thông minh thời gian thực với chi phí tối ưu.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC	iii
DANH MỤC TỪ VIẾT TẮT	vii
DANH MỤC BẢNG BIỂU	viii
DANH MỤC HÌNH ẢNH	ix
CHƯƠNG 1: TỔNG QUAN	1
1.1 GIỚI THIỆU ĐỀ TÀI	1
1.2 TÍNH CẤP THIẾT CỦA ĐỀ TÀI	1
1.3 MỤC TIÊU CỦA ĐỀ TÀI	2
1.4 ĐỐI TƯỢNG NGHIÊN CỨU	3
1.5 PHẠM VI NGHIÊN CỨU	4
1.6 PHƯƠNG PHÁP NGHIÊN CỨU	5
1.7 BỐ CỤC ĐỒ ÁN	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	7
2.1 TỔNG QUAN VỀ THỊ GIÁC MÁY TÍNH	7
2.2 MÔ HÌNH NHẬN DIỆN VẬT THỂ YOLO	7
2.2.1 Khái niệm	7
2.2.2 Đặc điểm	7
2.2.3 Đánh giá hiệu suất	8
2.2.4 Kiến trúc YOLOv11	9
2.3 BYTETRACK	12
2.3.1 Khái niệm	12
2.3.2 Đặc điểm	12
2.3.3 Đánh giá hiệu suất	13
2.3.4 Kiến trúc ByteTrack	14
2.3.5 Bộ lọc Kalman và Thuật toán Hungary trong bám vết	15
2.4 PHÉP BIẾN ĐỔI PHỐI CẢNH VÀ HOMOGRAPHY	17
2.4.1 Khái niệm phép biến đổi phối cảnh (Perspective Warping)	17
2.4.2 Cơ sở toán học của phép biến đổi Homography	17

2.4.3	Ứng dụng dán biển số phôi cảnh giả lập trong phạt nguội	18
2.5	MẠNG NƠ-RON TÍCH CHẬP HỒI QUY	18
2.5.1	Khái niệm	18
2.5.2	Cấu tạo từng thành phần của mạng	19
2.6	TỔNG QUAN VỀ ĐIỆN TOÁN BIÊN (EDGE COMPUTING)	23
2.6.1	Khái niệm Điện toán biên	23
2.6.2	Kiến trúc cơ bản của Điện toán biên	24
2.6.3	Các ưu điểm cốt lõi	25
2.6.4	Ứng dụng kiến trúc Edge-Cloud trong hệ thống AIoT	25
2.7	TỐI ƯU HÓA MÔ HÌNH AI TRÊN THIẾT BỊ NHÚNG	25
2.7.1	Lượng tử hóa mô hình (Model Quantization)	25
2.7.2	Kỹ thuật lược bỏ nhíp khung hình (Frame Skipping)	27
2.7.3	Kỹ thuật lập trình đa luồng (Multithreading) và Hàng đợi (Queue)	28
2.7.4	Lý thuyết về hệ điều hành thời gian thực FreeRTOS và cơ chế đa nhiệm	29
2.8	MỘT SỐ LINH KIỆN VÀ MODULE TRONG HỆ THỐNG	30
2.8.1	Raspberry Pi 4	30
2.8.2	ESP32-WROOM-32E	33
2.8.3	Webcam 1080p	38
2.8.4	Nguồn Adapter 12V-2A	39
2.8.5	Nút nhấn 6x6mm	41
2.8.6	Module hiển thị 4 led 7 đoạn TM1637	43
2.9	CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU	45
2.9.1	Chuẩn giao tiếp WiFi	45
2.9.2	Giao thức truyền dữ liệu	45
2.10	TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU FIREBASE	47
2.11	TỔNG QUAN VỀ CÁC CÔNG CỤ XÂY DỰNG VÀ LẬP TRÌNH HỆ THỐNG	48
2.11.1	EasyEDA	48
2.11.2	Antigravity IDE	49
2.11.3	PlatformIO	50
2.11.4	Thư viện OpenCV, Supervision và NCNN-Python	51
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG		53
3.1	YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG	53
3.1.1	Yêu cầu của hệ thống	53
3.1.2	Sơ đồ khối và chức năng của mỗi khối	54
3.1.3	Mô tả hoạt động của hệ thống	56
3.2	THIẾT KẾ PHẦN CỨNG	57
3.2.1	Khối camera 1	57

3.2.2	Khởi nút nhấn	58
3.2.3	Khởi hiển thị	62
3.2.4	Khởi xử lý trung tâm	65
3.2.5	Khởi điều khiển giao thông	65
3.2.6	Khởi nguồn	68
3.2.7	Sơ đồ nguyên lý toàn hệ thống	69
3.3	LỰA CHỌN VÀ HUẤN LUYỆN CÁC MODULE AI	72
3.3.1	Lựa chọn mô hình cho Module Giám sát Giao thông	72
3.3.2	Lựa chọn mô hình cho Module Đọc Biển số	73
3.3.3	Thu thập dữ liệu và Gán nhãn huấn luyện các mô hình AI	74
CHƯƠNG 4: THI CÔNG - LẬP TRÌNH HỆ THỐNG		77
4.1	LƯU ĐỒ GIẢI THUẬT CỦA HỆ THỐNG	77
4.1.1	Lưu đồ giải thuật chương trình chính trên Raspberry Pi 4	77
4.1.2	Lưu đồ giải thuật camera đa luồng	78
4.1.3	Lưu đồ giải thuật xử lý nhận dạng và bám vết đa đối tượng	80
4.1.4	Lưu đồ giải thuật chương trình con phát hiện vi phạm vượt đèn đỏ	81
4.1.5	Lưu đồ giải thuật xử lý ảnh phối cảnh biển số mô phỏng	83
4.1.6	Lưu đồ giải thuật gửi dữ liệu vi phạm bất đồng bộ lên Server	84
4.1.7	Lưu đồ giải thuật truyền thông MQTT quản lý trạng thái	86
4.1.8	Lưu đồ giải thuật chương trình chính trên ESP32	87
4.1.9	Lưu đồ giải thuật tính toán mật độ thích ứng điều chỉnh pha đèn	89
4.1.10	Lưu đồ giải thuật cập nhật màn hình hiển thị TM1637 và giao tiếp Firebase	90
4.1.11	Lưu đồ giải thuật nhận diện biển số xe vi phạm trên FastAPI Server	92
4.1.12	Lưu đồ giải thuật hiển thị danh sách phạt nguội trên Streamlit	93
4.2	QUY TRÌNH HUẤN LUYỆN CÁC MODULE AI	95
4.2.1	Huấn luyện mô hình phát hiện phương tiện giao thông	95
4.2.2	Huấn luyện mô hình định vị vùng chứa biển số xe	96
4.2.3	Huấn luyện mô hình nhận dạng ký tự biển số	96
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ		100
5.1	THIẾT LẬP MÔI TRƯỜNG THỰC NGHIỆM	100
5.1.1	Thiết bị phần cứng hệ thống	100
5.1.2	Môi trường phần mềm và cấu hình tối ưu hóa tại thiết bị biên	101
5.1.3	Kiến trúc mạng truyền thông và luồng trao đổi dữ liệu	102
5.2	KẾT QUẢ HUẤN LUYỆN CÁC MÔ HÌNH AI	103
5.2.1	Mô hình phát hiện phương tiện giao thông	103
5.2.2	Mô hình định vị vùng chứa biển số xe	105
5.2.3	Mô hình nhận dạng ký tự biển số	105
5.3	SO SÁNH HIỆU NĂNG PYTORCH VS NCNN TRÊN EDGE DEVICE ...	107

5.3.1 Phân tích tốc độ suy luận (Inference Speed)	108
5.3.2 Phân tích thời gian nạp mô hình (Load Time)	109
5.3.3 Phân tích sự cân bằng giữa độ phân giải và hiệu năng	110
5.4 ĐÁNH GIÁ ỔN ĐỊNH HỆ THỐNG (STRESS TEST & TELEMETRY)	111
5.4.1 Nhiệt độ và tần số hoạt động của vi xử lý (CPU Telemetry)	111
5.4.2 Tốc độ hiển thị vòng lặp hệ thống (Loop FPS) và cơ chế đa luồng	112
5.4.3 Đánh giá quản lý bộ nhớ RAM và độ trễ runtime	114
5.5 KIỂM CHỨNG CHỨC NĂNG VÀ TÍCH HỢP HỆ THỐNG	115
5.5.1 Theo dõi quỹ đạo phương tiện và điều khiển thích ứng	116
5.5.2 Phát hiện hành vi vi phạm giao thông và cơ chế lọc trùng lặp	116
5.5.3 Nhật ký ghi nhận vi phạm thực tế và kết quả nhận dạng ký tự	117
5.5.4 Hệ thống giám sát trực quan (Web Dashboard)	119
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	121
6.1 KẾT LUẬN	121
6.2 HƯỚNG PHÁT TRIỂN	122
PHỤ LỤC	124
TÀI LIỆU THAM KHẢO	125

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa đầy đủ	Giải thích
AI	Artificial Intelligence	Trí tuệ nhân tạo
AIoT	Artificial Intelligence of Things	Trí tuệ nhân tạo vạn vật
API	Application Programming Interface	Giao diện lập trình ứng dụng
CPU	Central Processing Unit	Bộ xử lý trung tâm
CRNN	Convolutional Recurrent Neural Network	Mạng nơ-ron tích chập hồi quy
CV	Computer Vision	Thị giác máy tính
FPS	Frame per second	Khung hình mỗi giây
GPIO	General Purpose Input Output	Chân vào/ra mục đích chung
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản bảo mật
IoT	Internet of Things	Internet vạn vật
JSON	JavaScript Object Notation	Định dạng đối tượng JavaScript
LSTM	Long Short-Term Memory	Bộ nhớ lưu trữ ngắn dài
OCR	Optical Character Recognition	Nhận diện ký tự quang học
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
YOLO	You Only Look One	Thuật toán phát hiện đối tượng thời gian thực

DANH MỤC BẢNG BIỂU

Bảng 2.1 Thông số kỹ thuật của Raspberry pi 4 model B	31
Bảng 2.2 Thông số kỹ thuật của bo mạch điều khiển ESP32-Wroom-32E	34
Bảng 2.3 Thông số kỹ thuật của Webcam 1080p	38
Bảng 2.4 Thông số kỹ thuật của nguồn Adapter 12V-2A	40
Bảng 2.5 Thông số kỹ thuật của nút nhấn 6x6mm	41
Bảng 2.6 Thông số kỹ thuật của module hiển thị TM1637	44
Bảng 3.1 Đặc điểm của các camera	57
Bảng 3.2 Các lệnh cấu hình TM1637 được sử dụng	63
Bảng 3.3 Một số vi điều khiển phổ biến	66
Bảng 3.4 Thông số điện áp và dòng tiêu thụ của linh kiện trong hệ thống	68
Bảng 3.5 So sánh các mô hình cho Module Giám sát	73
Bảng 5.1 So sánh hiệu năng giữa PyTorch và NCNN trên thiết bị nhúng	108

DANH MỤC HÌNH ẢNH

Hình 2.1 Tính toán IoU	9
Hình 2.2 Kiến trúc mạng YOLOv11	10
Hình 2.3 Kiến trúc của mạng CRNN	19
Hình 2.4 The receptive field	20
Hình 2.5 (a) Cấu trúc của một đơn vị LSTM cơ bản. (b) Cấu trúc của LSTM hai chiều sâu (deep bidirectional) trong mạng CRNN	21
Hình 2.6 Kiến trúc tổng quan mô hình phân tán Edge-Cloud.	24
Hình 2.7 Raspberry pi 4 model B	31
Hình 2.8 Kí hiệu nguyên lý Raspberry pi 4 model B	31
Hình 2.9 Bo mạch điều khiển ESP32-WROOM-32E	33
Hình 2.10 Kí hiệu nguyên lý vi điều khiển ESP32-WROOM-32E	33
Hình 2.11 Sơ đồ chân của bo mạch điều khiển ESP32-DevkitC-32D	37
Hình 2.12 Sơ đồ nguyên lý của ESP32	37
Hình 2.13 Webcam 1080p	38
Hình 2.14 Ký hiệu webcam 1080p	38
Hình 2.15 Màn hình nguồn Adapter 12V-2A	40
Hình 2.16 Kí hiệu nguyên lý nguồn Adapter 12V-2A	40
Hình 2.17 Nút nhấn 6x6mm	41
Hình 2.18 Kí hiệu nguyên lý nút nhấn 6x6mm	41
Hình 2.19 Module TM 1637	43
Hình 2.20 Kí hiệu nguyên lý của TM1637	43
Hình 2.21 Sơ đồ nguyên lý của module hiển thị 4 led 7 đoạn TM1637	43
Hình 2.22 Cơ sở dữ liệu Firebase	47
Hình 2.23 Phần mềm EasyEda	48

Hình 2.24 Phần mềm Antigravity IDE	49
Hình 2.25 Phần mềm PlatformIO	50
Hình 3.1 Sơ đồ khối của hệ thống.....	54
Hình 3.2 Mô tả kết nối của khối nút nhấn với khối xử lý trung tâm	59
Hình 3.3 Mô tả kết nối của khối hiển thị	63
Hình 3.4 Sơ đồ chân của khối xử lý trung tâm	67
Hình 3.5 Mô tả kết nối của khối nguồn	69
Hình 3.6 Sơ đồ nguyên lý toàn hệ thống	71
Hình 3.7 Sa bàn giao lộ kích thước 40cm x 60cm	75
Hình 4.1 Lưu đồ giải thuật chương trình chính trên Raspberry Pi 4	77
Hình 4.2 Lưu đồ giải thuật Camera đa luồng	79
Hình 4.3 Lưu đồ giải thuật xử lý nhận dạng và bám vết đa đối tượng	80
Hình 4.4 Lưu đồ giải thuật phát hiện vi phạm vượt đèn đỏ	82
Hình 4.5 Lưu đồ giải thuật xử lý phối cảnh biến số xe mô phỏng	83
Hình 4.6 Lưu đồ giải thuật gửi dữ liệu bất đồng bộ lên Server	85
Hình 4.7 Lưu đồ giải thuật truyền nhận dữ liệu giữa Raspberry Pi 4 và ESP32	86
Hình 4.8. Lưu đồ giải thuật chương trình chính trên ESP32	88
Hình 4.9 Lưu đồ giải thuật tính toán thời gian đèn giao thông tương ứng	89
Hình 4.10 Lưu đồ giải thuật hiển thị trên TM1637 và gửi dữ liệu qua Firebase	91
Hình 4.11 Lưu đồ giải thuật nhận diện biển số xe vi phạm	92
Hình 4.12 Lưu đồ giải thuật hiển thị danh sách phạt nguội trên Streamlit Dashboard	94
Hình 5.1 Mô hình cấu hình phần cứng thực nghiệm thực tế của hệ thống.	101
Hình 5.2 Các đường cong huấn luyện và đánh giá mô hình YOLOv11n	104
Hình 5.3 Kết quả suy luận thực tế của mô hình phát hiện phương tiện.	105

Hình 5.4 Kết quả suy luận của mô hình định vị biên số xe.	105
Hình 5.5 Đồ thị suy giảm CTC Loss của mô hình nhận dạng ký tự biên số.	107
Hình 5.6 Đồ thị so sánh tốc độ xử lý (FPS) giữa PyTorch và NCNN.	111
Hình 5.7 So sánh độ trễ suy luận trung bình và P95.	111
Hình 5.8 Đồ thị telemetry nhiệt độ vi xử lý.	112
Hình 5.9 Đồ thị biến thiên Loop FPS của hệ thống biên.	113
Hình 5.10 Đồ thị viến đo bộ nhớ RAM.	114
Hình 5.11 Đồ thị biến động độ trễ chu kỳ xử lý.	115
Hình 5.12 Giao diện phát hiện vi phạm vượt đèn đỏ và dán biển số Homography giả lập trên sa bàn thực nghiệm.	117
Hình 5.13 Nhật ký kết nối và xử lý truyền thông thời gian thực trên FastAPI Server và ESP32.	119
Hình 5.14 Giao diện xử lý hình ảnh thời gian thực tại Edge (NCNN-optimized).	119

CHƯƠNG 1: TỔNG QUAN

1.1 GIỚI THIỆU ĐỀ TÀI

Sự bùng nổ của phương tiện giao thông cá nhân tại các đô thị đang vượt quá khả năng đáp ứng của cơ sở hạ tầng, dẫn đến các hệ lụy nghiêm trọng về ùn tắc, ô nhiễm và tai nạn. Để giải quyết bài toán này, các hệ thống giám sát hiện hành vẫn bộc lộ nhiều hạn chế: camera truyền thống chỉ ghi hình thụ động gây tốn nhân lực, trong khi các giải pháp thông minh lại phụ thuộc hoàn toàn vào máy chủ trung tâm dẫn đến chi phí cao, dễ nghẽn băng thông và có độ trễ lớn.

Trong bối cảnh đó, sự kết hợp giữa Trí tuệ nhân tạo, Internet vạn vật (AIoT) và Điện toán biên (Edge Computing) mở ra hướng tiếp cận đột phá: đưa năng lực phân tích dữ liệu về ngay tại rìa mạng để tối ưu độ trễ và tiết kiệm băng thông đường truyền. Xuất phát từ thực tiễn này, nhóm quyết định nghiên cứu đề tài: "Hệ thống AIoT Giám sát Giao thông Thông minh và Phạt nguội Tự động".

Hệ thống được thiết kế theo mô hình phân tán tính toán 3 tầng vật lý (vi điều khiển ESP32, máy tính biên Raspberry Pi 4 và Laptop Server trung tâm), vận hành liên tục qua hai luồng nhiệm vụ cốt lõi:

Điều phối giao thông thích ứng: Tự động đếm và phân loại phương tiện tại các làn đường, từ đó nội suy chu kỳ đèn xanh tối ưu và điều khiển thiết bị tại chỗ theo thời gian thực qua giao thức MQTT.

Phạt nguội tự động: Nhận diện phương tiện dừng vạch dừng khi đèn đỏ, dán biển số giả lập bằng phép biến đổi Homography, sau đó chỉ gửi dữ liệu ảnh đã lọc qua FastAPI lên máy chủ trung tâm để nhận dạng chuỗi ký tự (OCR) bằng mạng CRNN.

1.2 TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Trong xu thế phát triển đô thị thông minh, quản lý giao thông tại Việt Nam đang đối mặt với nhiều thách thức lớn do mật độ phương tiện cao và hạ tầng phức tạp. Tình trạng ùn tắc kéo dài cùng các hành vi vi phạm như vượt đèn đỏ,

đề vạch diễn ra phổ biến, đặt ra nhu cầu cấp thiết về một giải pháp giám sát tự động, thông minh, có khả năng triển khai rộng rãi với chi phí tối ưu.

Để giải quyết bài toán này, đề tài đề xuất hệ thống giám sát toàn diện dựa trên kiến trúc phân tán Edge-Cloud, kết hợp máy tính nhúng Raspberry Pi 4, vi điều khiển ESP32 và các thuật toán học sâu (YOLOv11n, ByteTrack, CRNN OCR). Hệ thống có khả năng tự động đánh giá mật độ xe thời gian thực để điều phối linh hoạt chu kỳ đèn tín hiệu qua MQTT, đồng thời phát hiện lỗi vượt vạch dừng ảo khi đèn đỏ. Khi xảy ra vi phạm, thiết bị biên tự động cắt ảnh và truyền bất đồng bộ qua FastAPI lên máy chủ trung tâm phục vụ nhận diện biển số xe, giúp nâng cao ý thức người dân và giảm tải đáng kể cho lực lượng chức năng.

Hệ thống cung cấp khả năng theo dõi từ xa trực quan thông qua giao diện Streamlit Dashboard kết nối cơ sở dữ liệu Firebase. Nhờ áp dụng triệt để các kỹ thuật tối ưu hóa nhúng cốt lõi như lượng tử hóa mô hình NCNN, lập trình đa luồng đa nhiệm và lược bỏ khung hình, thiết bị biên luôn duy trì sự ổn định lâu dài. Việc tích hợp các tính năng nâng cao như định nghĩa vùng ROI đếm xe, vẽ vạch cấm ảo và ứng dụng phép biến đổi phối cảnh Homography không chỉ giúp hệ thống thích ứng linh hoạt với thực tế Việt Nam, mà còn thúc đẩy tiềm năng nghiên cứu, ứng dụng sâu rộng trong lĩnh vực giao thông thông minh và tự động hóa.

1.3 MỤC TIÊU CỦA ĐỀ TÀI

Đề tài hướng đến việc nghiên cứu, thiết kế và xây dựng thành công một mô hình hệ thống AIoT giám sát giao thông thông minh và xử lý vi phạm tự động dựa trên kiến trúc phân tán Edge-Cloud hoạt động theo thời gian thực.

Cụ thể, các mục tiêu trọng tâm bao gồm:

- Thiết kế kiến trúc hệ thống phân tán 3 tầng: Quy hoạch rõ ràng chức năng của tầng vi điều khiển thực thi (ESP32), tầng điều phối tại biên (Raspberry Pi 4) và tầng máy chủ trung tâm (FastAPI Server + Streamlit).
- Xây dựng luồng Logic 1 - Giám sát mật độ và điều phối đèn: Triển khai mô hình YOLOv11n và thuật toán ByteTrack trên Raspberry Pi 4 để phát hiện, phân loại (xe máy, ô tô con, xe buýt, xe tải) và đếm số lượng xe

trong các vùng quan tâm (ROI). Lập trình thuật toán tính toán thời gian đèn thích ứng trên ESP32 thông qua giao tiếp MQTT.

- Xây dựng luồng Logic 2 - Phát hiện vi phạm và phạt nguội: Thiết lập thuật toán phát hiện xe vượt vạch cấm dừng ảo khi có tín hiệu đèn đỏ. Triển khai phép biến đổi Homography để dán biển số mô phỏng lên đuôi xe vi phạm ngay tại biên.

- Xây dựng bộ nhận diện biển số OCR trung tâm: Phát triển server FastAPI nhận ảnh vi phạm, sử dụng mô hình YOLO để khoanh vùng biển số và mô hình CRNN (tích hợp ResNet34 + GRU + CTC Loss) để đọc ký tự biển số xe Việt Nam chính xác.

- Xây dựng giao diện quản lý trực quan: Thiết kế web Streamlit Dashboard hiển thị lịch sử xe vi phạm (biển số, ảnh chụp xe vi phạm, thời gian, độ tin cậy) và đồng bộ trạng thái giao lộ lên Firebase Realtime Database.

- Tối ưu hóa hiệu năng thiết bị nhúng: Áp dụng thành công các kỹ thuật lập trình đa luồng (Multithreading), lược bỏ nhịp khung hình (Frame Skipping) và lượng tử hóa mô hình sang định dạng tối ưu NCNN để hệ thống chạy ổn định 24/7 trên CPU của Raspberry Pi 4.

1.4 ĐỐI TƯỢNG NGHIÊN CỨU

Về phương diện phần cứng và phần mềm, đề tài tập trung nghiên cứu và ứng dụng các đối tượng công nghệ cốt lõi sau:

- Thiết bị xử lý và điều khiển: Máy tính nhúng Raspberry Pi 4 Model B (khai thác kiến trúc ARM Cortex-A72, tập lệnh tối ưu AI và vận hành MQTT Broker) cùng vi điều khiển ESP32-WROOM-32E (quản lý GPIO điều khiển ngoại vi và giao tiếp mạng WiFi).

- Mô hình Học sâu và Thị giác máy tính: Kiến trúc mạng YOLOv11 kết hợp thuật toán bám vết ByteTrack (ứng dụng bộ lọc Kalman và thuật toán Hungary) phục vụ giám sát và đếm lưu lượng.

- Xử lý ảnh và Nhận dạng ký tự (OCR): Nghiên cứu phép biến đổi phối cảnh phẳng (Homography) qua ma trận 3×3 để tạo dữ liệu biển số giả lập, kết hợp mạng nơ-ron CRNN (sử dụng cơ chế giải mã CTC Loss) để trích xuất văn bản biển số.
- Hạ tầng truyền thông và Đám mây: Triển khai giao thức MQTT (điều phối thiết bị thời gian thực), HTTP RESTful qua FastAPI (truyền ảnh bất đồng bộ), Firebase Realtime Database (đồng bộ trạng thái) và framework Streamlit để xây dựng Dashboard quản lý trung tâm.
- Thiết kế mạch và Ngoại vi: Giao tiếp module hiển thị LED 7 đoạn TM1637, thiết kế mạch hạ áp và chống nhiễu cho hệ thống vi điều khiển sử dụng IC ổn áp LM7805 kết hợp bộ tụ lọc.

1.5. PHẠM VI NGHIÊN CỨU

Để đảm bảo tính khả thi và hoàn thành đề tài đúng thời hạn, phạm vi nghiên cứu được giới hạn như sau:

- Giới hạn về môi trường thực nghiệm: Hệ thống hoạt động tốt nhất trong điều kiện ánh sáng đầy đủ ban ngày và góc camera thuận lợi từ trên cao nhìn xuống ngã tư. Các yếu tố thời tiết khắc nghiệt như mưa bão lớn, sương mù hoặc ánh sáng ban đêm quá yếu có thể làm giảm độ chính xác của mô hình nhận diện xe và đọc biển số.
- Giới hạn về dữ liệu phạt nguội: Do không có điều kiện lắp đặt camera kép thực tế ngoài đường phố để thu thập dữ liệu vi phạm đồng bộ, luồng dữ liệu phạt nguội được mô phỏng thông qua việc sử dụng video mẫu của bộ dữ liệu công khai 'UIT-ADrone'^[14] phối hợp cùng phép biến đổi Homography để dán biển số xe giả lập vào đuôi xe vi phạm ngay tại thiết bị biên.
- Giới hạn hiệu năng nhúng: Raspberry Pi 4 là máy tính nhúng chạy CPU, không có bộ tăng tốc GPU chuyên dụng. Do đó, hệ thống được giới hạn chạy suy luận AI ở độ phân giải 320x320 pixel (đối với mô hình NCNN)

và áp dụng Frame Skipping (chỉ xử lý 10-15 FPS thay vì 30 FPS) để bảo vệ thiết bị không bị quá nhiệt và treo luồng xử lý chính.

1.6 PHƯƠNG PHÁP NGHIÊN CỨU

Để triển khai đồ án, nghiên cứu đã tổng hợp linh hoạt nhiều phương pháp khoa học. Phương pháp phân tích và so sánh được áp dụng nhằm đánh giá, lựa chọn nền tảng phần cứng (Raspberry Pi 4, ESP32) cùng kiến trúc AI tối ưu nhất, giúp cân bằng giữa năng lực xử lý thời gian thực tại biên và chi phí đầu tư. Quá trình thi công khai thác tối đa các công cụ chuyên dụng như Visual Studio Code (PlatformIO), Google Colab, EasyEDA kết hợp cùng hệ sinh thái thư viện OpenCV, Supervision và NCNN. Cuối cùng, phương pháp thực nghiệm trên sa bàn giao lộ mô phỏng được tiến hành để đo lường, kiểm định khắt khe độ chính xác nhận diện phương tiện, khả năng giải mã OCR, tốc độ khung hình và độ trễ mạng, qua đó khẳng định tính ổn định của toàn hệ thống trước khi ứng dụng thực tiễn.

1.7 BỐ CỤC ĐỒ ÁN

Chương 1: Tổng Quan: Giới thiệu bối cảnh, lý do chọn đề tài, tính cấp thiết, mục tiêu, đối tượng, giới hạn và phương pháp nghiên cứu.

Chương 2: Cơ sở lý thuyết: Trình bày nền tảng lý thuyết về thị giác máy tính, mô hình YOLO, thuật toán ByteTrack, mạng CRNN OCR, phép biến đổi phối cảnh Homography, các linh kiện phần cứng nhúng, giao thức MQTT, FastAPI và các kỹ thuật tối ưu hóa thiết bị nhúng.

Chương 3: Thiết Kế và Xây Dựng Hệ Thống : Mô tả chi tiết yêu cầu hệ thống, sơ đồ khối kiến trúc, thiết kế sơ đồ đấu nối dây phần cứng và quy trình huấn luyện các mô hình AI.

Chương 4: Thi Công và Lập Trình Hệ Thống: Mô tả quá trình lắp ráp phần cứng, kiến trúc phần mềm phân tán và trình bày các lưu đồ giải thuật chi tiết của hệ thống (Raspberry PI, ESP32, Server, Dashboard).

Chương 5: Kết Quả Thực Nghiệm và Đánh giá: Trình bày hình ảnh chạy demo thực tế và đưa ra các bảng số liệu so sánh định lượng về độ chính xác AI, hiệu năng xử lý (FPS, CPU/RAM) trên Raspberry Pi 4.

Chương 6: Kết luận và Hướng phát triển: Tổng kết các kết quả đạt được, chỉ rõ những hạn chế còn tồn đọng và đề xuất các giải pháp nâng cấp hệ thống trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ THỊ GIÁC MÁY TÍNH

Thị giác máy tính là một công nghệ mà máy sử dụng để tự động nhận biết và mô tả hình ảnh một cách chính xác và hiệu quả. Ngày nay, các hệ thống máy tính có quyền truy cập vào khối lượng lớn hình ảnh và dữ liệu video bắt nguồn từ hoặc được tạo bằng điện thoại thông minh, camera giao thông, hệ thống bảo mật và các thiết bị khác. Ứng dụng thị giác máy tính sử dụng trí tuệ nhân tạo và máy học để xử lý dữ liệu này một cách chuẩn xác cho việc xác định đối tượng và nhận diện khuôn mặt, cũng như phân loại, đề xuất, giám sát và phát hiện.

2.2 MÔ HÌNH NHẬN DIỆN VẬT THỂ YOLO

2.2.1 Khái niệm^[1]

YOLO (You Only Look Once) là một thuật toán nhận dạng đối tượng được phát triển bởi Joseph Redmon, Santosh Divvala, Ross Girshick và Ali Farhadi vào năm 2015. YOLO là một thuật toán nhận dạng đối tượng một giai đoạn (one-stage detector), trong đó toàn bộ quá trình phát hiện và phân loại đối tượng được thực hiện trong một lần lan truyền tiến duy nhất của mạng nơ-ron.

Khác với các phương pháp hai giai đoạn (two-stage detector), YOLO xem bài toán phát hiện đối tượng như một bài toán hồi quy trực tiếp từ ảnh đầu vào sang các tham số mô tả bounding box và xác suất lớp. Cách tiếp cận này giúp YOLO đạt được tốc độ xử lý cao, phù hợp với các ứng dụng yêu cầu thời gian thực.

2.2.2 Đặc điểm^[1]

Độ chính xác cao và tốc độ nhanh: YOLO có khả năng đưa ra các dự đoán đồng thời về các đối tượng trong hình ảnh chỉ trong một lần chạy mô hình. Điều này giúp YOLO vượt trội về tốc độ so với các phương pháp truyền thống.

Sử dụng mạng nơ-ron tích chập: YOLO sử dụng mạng nơ-ron tích chập để học và trích xuất đặc trưng từ ảnh, giúp cho việc phát hiện đối tượng trở nên hiệu quả và chính xác hơn.

Phân cấp dự đoán: YOLO chia hình ảnh thành một lưới các ô, và mỗi ô sẽ dự đoán các *bounding box* và xác suất của các đối tượng trong ô đó. Quá trình này giúp giảm thiểu số lượng dự đoán sai lệch.

Học từ dữ liệu lớn: Mô hình YOLO được huấn luyện trên các bộ dữ liệu lớn, từ đó học được nhiều đặc điểm của các đối tượng và có khả năng tổng quát hóa tốt trên các tình huống thực tế.

Hỗ trợ đa lớp: YOLO có thể phát hiện nhiều lớp đối tượng khác nhau trong cùng một hình ảnh, từ người đi bộ đến các phương tiện giao thông và vật dụng.

2.2.3 Đánh giá hiệu suất

2.2.3.1 Union (IoU)^[2]

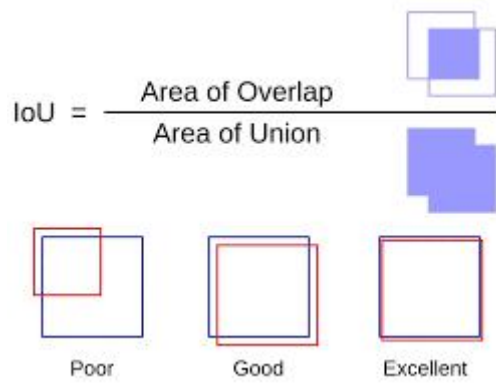
IoU (Intersection over Union) là một chỉ số đánh giá cơ bản trong lĩnh vực thị giác máy tính, đặc biệt đối với các tác vụ phát hiện đối tượng.

IoU đo lường sự chồng chéo giữa hộp giới hạn được dự đoán (do mô hình tạo ra) và hộp giới hạn ground-truth (được gắn nhãn thủ công).

Điểm số IoU là một giá trị từ 0 đến 1, định lượng mức độ chính xác của mô hình trong việc xác định vị trí của đối tượng (ví dụ: đối với mô hình Ultralytics YOLO11).

Về cốt lõi, IoU được tính bằng cách lấy tỷ lệ giao (diện tích chồng lên nhau) trên hợp (tổng diện tích mà cả hai hộp bao phủ) của bounding box được dự đoán và ground-truth.

Khái niệm đơn giản nhưng mạnh mẽ này là nền tảng của deep learning hiện đại để phát hiện đối tượng.



Hình 2.1 Tính toán IoU

2.2.3.2 Độ chính xác trung bình (mAP)^[3]

Độ chính xác trung bình (Mean Average Precision - mAP) là một chỉ số đánh giá quan trọng và được sử dụng rộng rãi trong thị giác máy tính, đặc biệt cho các tác vụ phát hiện đối tượng.

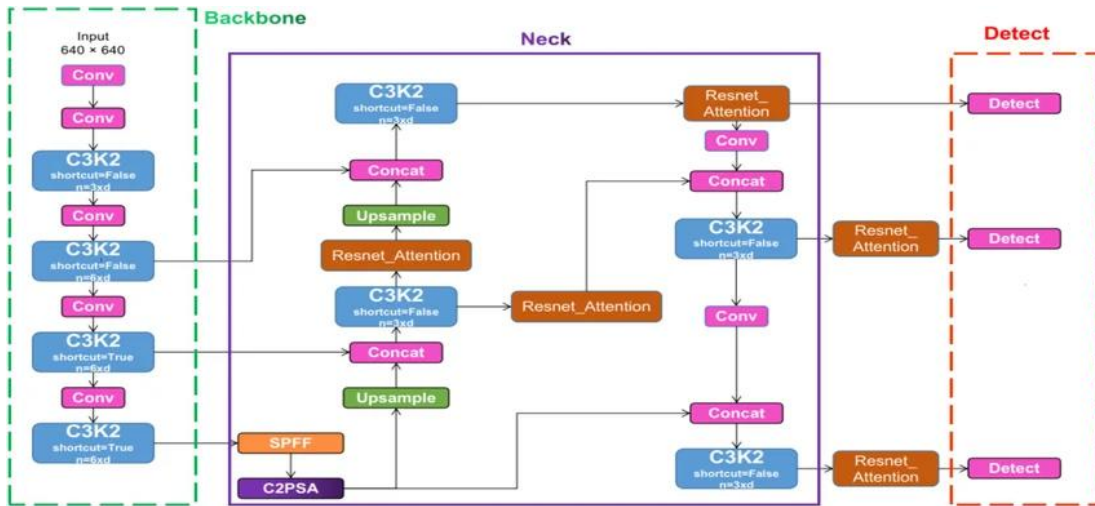
mAP cung cấp một điểm số duy nhất, toàn diện để tóm tắt hiệu suất của một mô hình, đo lường độ chính xác của các dự đoán trên tất cả các danh mục đối tượng.

Chỉ số này tính đến cả tính chính xác của phân loại (đối tượng có đúng không) và chất lượng của bản địa hóa (mức độ phù hợp của khung giới hạn).

Vì mAP cung cấp một đánh giá cân bằng, nó đã trở thành thước đo tiêu chuẩn để so sánh hiệu suất của các mô hình phát hiện đối tượng khác nhau, chẳng hạn như Ultralytics YOLO. mAP rất quan trọng trong các ứng dụng AI như xe tự lái và chăm sóc sức khỏe. Hiểu rõ các chỉ số này giúp phân tích hiệu suất của các mô hình như YOLO và đưa ra những quyết định thông minh về việc áp dụng chúng trong các bối cảnh thực tế. YOLO đã đổi mới trong lĩnh vực phát hiện đối tượng bằng cách thực hiện cả hai nhiệm vụ phát hiện và phân loại trong một lần duy nhất, mang lại tốc độ và hiệu quả ấn tượng so với các phương pháp truyền thống.

2.2.4 Kiến trúc YOLOv11

YOLOv11 (phiên bản mới nhất thuộc dòng Ultralytics YOLO) đánh dấu một bước tiến lớn trong việc thiết kế mạng nơ-ron tối ưu hóa tham số. Thay vì chỉ tăng kích thước mô hình để đạt độ chính xác cao, YOLOv11 tập trung cấu trúc lại luồng dữ liệu bên trong để cân bằng hoàn hảo giữa tốc độ suy luận (Inference Speed) và độ chính xác (mAP). Kiến trúc của YOLOv11 được chia thành ba mô-đun cốt lõi: Backbone, Neck và Head.



Hình 2.2 Kiến trúc mạng YOLOv11

2.2.4.1 Mạng xương sống (Backbone)

Nhiệm vụ của Backbone là quét qua hình ảnh đầu vào để rút trích các đặc trưng từ cơ bản (cạnh, góc, màu sắc) đến phức tạp (hình dáng vật thể). Backbone của YOLOv11 sở hữu những cải tiến đột phá:

- Khối C3k2 (Cross Stage Partial Networks with 3 Convolutions): Đây là bản nâng cấp cấu trúc từ khối C2f của các thế hệ trước. C3k2 tối ưu hóa đường dẫn gradient (gradient path), giúp luồng thông tin truyền qua các lớp chập sâu mà không bị suy hao. Nhờ đó, mô hình học được các biểu diễn đặc trưng phong phú hơn nhưng lại sử dụng ít tham số hơn.
- Cơ chế tập trung không gian C2PSA (Contextual Spatial Attention): Điểm sáng của YOLOv11 là việc tích hợp khối C2PSA ở cuối mạng

xương sống. Khối này hoạt động như một "bộ lọc sự chú ý", giúp mô hình tự động làm nổi bật các khu vực chứa đối tượng (ví dụ: phương tiện giao thông) và triệt tiêu các nhiễu nền không cần thiết.

- Lớp SPPF (Spatial Pyramid Pooling - Fast): Sử dụng các cửa sổ gộp (pooling) với kích thước khác nhau để trích xuất thông tin đa quy mô (Multi-scale). SPPF giúp mô hình duy trì độ nhạy bén cực cao đối với sự biến thiên về kích thước đối tượng, đặc biệt là các phương tiện ở xa camera.

2.2.4.2 Mạng cổ (Neck)

Các đặc trưng trích xuất từ Backbone thường bị phân mảnh: lớp nông chứa thông tin tọa độ rất tốt nhưng thiếu ngữ nghĩa, trong khi lớp sâu hiểu rõ vật thể là gì nhưng lại mất đi tọa độ chính xác. Neck làm nhiệm vụ trộn lẫn chúng lại thông qua:

- Cấu trúc FPN + PAN (Feature Pyramid Network & Path Aggregation Network): Kiến trúc tháp đặc trưng kép này tạo ra hai luồng truyền tải thông tin. FPN truyền đặc trưng ngữ nghĩa mạnh từ trên xuống dưới (Top-down), trong khi PAN truyền thông tin định vị mạnh từ dưới lên trên (Bottom-up). Sự kết hợp này đảm bảo mọi kích thước vật thể, từ chiếc xe tải lớn đến biển số xe rất nhỏ, đều được giữ lại đầy đủ thông tin không gian cục bộ trước khi đưa vào dự đoán

2.2.4.3 Mạng đầu ra (Head)

Nơi xuất ra kết quả cuối cùng của hệ thống. YOLOv11 sử dụng thiết kế tinh gọn để đáp ứng yêu cầu xử lý thời gian thực:

- Thiết kế tách biệt (Decoupled Head): Thay vì gộp chung, YOLOv11 tách độc lập hai nhánh: nhánh phân loại lớp (Classification - xác định vật thể là ô tô hay xe máy) và nhánh hồi quy tọa độ (Regression - xác định hộp giới hạn Bounding Box). Việc tách biệt này giúp giải quyết xung đột toán học:

phân loại cần tính bất biến không gian (vật thể ở đâu cũng là vật thể đó), trong khi hồi quy lại rất nhạy cảm với sự thay đổi vị trí.

- Cơ chế Anchor-Free (Không neo trước): Loại bỏ hoàn toàn các hộp neo (anchor boxes) được định nghĩa sẵn. Thay vì dự đoán sự dịch chuyển so với hộp neo, mô hình dự đoán trực tiếp tọa độ tâm và kích thước của đối tượng. Điều này không chỉ giúp giảm triệt để số lượng tham số tính toán và sự phức tạp của bước hậu xử lý (Non-Maximum Suppression), mà còn giúp mô hình hội tụ nhanh hơn, cực kỳ lý tưởng để triển khai trên các thiết bị nhúng có tài nguyên hạn hẹp như Raspberry Pi.

2.3 BYTETRACK

2.3.1 Khái niệm

ByteTrack là một thuật toán theo dõi đa đối tượng (Multi-Object Tracking - MOT) được đề xuất bởi Yikai Wang và các cộng sự vào năm 2021. Thuật toán này thuộc mô hình theo dõi dựa trên phát hiện (tracking-by-detection), tập trung vào việc liên kết gần như mọi hộp giới hạn (bounding box) được phát hiện, kể cả những hộp có điểm số thấp (low-score detections), để khắc phục tình trạng mất dấu đối tượng do che khuất hoặc phát hiện sai.

Khác với các phương pháp truyền thống chỉ sử dụng hộp giới hạn có điểm số cao (thường > 0.5), ByteTrack coi bài toán theo dõi như quá trình liên kết tất cả các phát hiện với các tracklet hiện có, sử dụng độ tương đồng (IoU hoặc Re-ID) để khôi phục đối tượng thực và loại bỏ nền. Cách tiếp cận này giúp giảm số lượng lỗi theo dõi, cải thiện tính liên tục của quỹ đạo (trajectories) và đạt hiệu suất cao trong các ứng dụng thời gian thực như giám sát giao thông hoặc theo dõi đám đông.

2.3.2 Đặc điểm

ByteTrack có các đặc điểm chính như sau:

- Liên kết mọi hộp phát hiện: Thuật toán xử lý cả hộp giới hạn có điểm số thấp để khôi phục đối tượng bị che khuất, giảm số lượng đối tượng bị bỏ lỡ và số lần chuyển ID (ID switches).
- Liên kết theo giai đoạn: Thực hiện hai bước liên kết – đầu tiên với hộp điểm số cao sử dụng độ tương đồng IoU hoặc Re-ID, sau đó với hộp điểm số thấp chỉ sử dụng IoU để tránh lỗi từ đặc trưng không đáng tin cậy.
- Tích hợp dự đoán chuyển động: Sử dụng bộ lọc Kalman để dự đoán vị trí tracklet, giúp tăng độ chính xác liên kết mà không cần dữ liệu huấn luyện phức tạp.
- Tính tổng quát hóa cao: Có thể kết hợp với nhiều detector (như YOLOX) và phương pháp liên kết khác nhau (IoU, Re-ID, attention-based), cải thiện hiệu suất của các tracker hiện có lên đến 10 điểm IDF1.
- Tốc độ xử lý nhanh: Đạt 30 FPS trên GPU V100, phù hợp cho ứng dụng thời gian thực, và không yêu cầu lượng dữ liệu lớn (chỉ khoảng 29.000 ảnh cho hiệu suất hàng đầu trên MOT17).
- Nội suy tracklet: Sử dụng nội suy tuyến tính để lấp đầy các khoảng trống do che khuất hoàn toàn, cải thiện MOTA và IDF1.

2.3.3 Đánh giá hiệu suất

2.3.3.1 Multiple Object Tracking Accuracy (MOTA)

MOTA là chỉ số đánh giá tổng hợp hiệu suất theo dõi đa đối tượng, tính toán dựa trên tỷ lệ phát hiện đúng, phát hiện sai và số lần chuyển ID.

Công thức:

$$MOTA = 1 - \frac{FN + FP + IDs}{GT}$$

Trong đó FN là false negatives (bỏ lỡ), FP là false positives (phát hiện sai), IDs là ID switches, GT là ground-truth. MOTA nằm trong khoảng từ $-\infty$ đến 1, giá trị cao hơn cho thấy tracker chính xác hơn.

2.3.3.2 ID F1 Score (IDF1)^[3]

IDF1 là chỉ số đo lường tính nhất quán của ID theo dõi, kết hợp ID precision (IDP) và ID recall (IDR) theo công thức F1:

$$IDF1 = \frac{2 \times IDTP}{2 \times IDTP + IDFP + IDFN}$$

Trong đó IDTP là true positives về ID, IDFP và IDFN là false positives/negatives về ID. IDF1 đánh giá khả năng duy trì ID ổn định qua các frame.

2.3.3.3 Higher Order Tracking Accuracy (HOTA)

HOTA là chỉ số mới kết hợp độ chính xác phát hiện và liên kết, tính trung bình giữa Detection Accuracy (DetA) và Association Accuracy (AssA). HOTA cân bằng hơn so với MOTA, đặc biệt trong các tình huống đông đúc.

Trên các benchmark:

- MOT17: ByteTrack đạt 80.3 MOTA, 77.3 IDF1, 63.1 HOTA, vượt trội so với các tracker khác (ví dụ: cải thiện CenterTrack từ 66.1 lên 67.4 MOTA).
- MOT20: 77.8 MOTA, 75.2 IDF1, 61.3 HOTA, xử lý tốt đám đông với ít ID switches (1223).
- BDD100K: 40.1 mMOTA, 55.8 mIDF1, dẫn đầu trong theo dõi phương tiện.
- Các benchmark khác như HiEve, DanceTrack, KITTI, TAO cũng cho thấy hiệu suất cao, chứng minh tính mạnh mẽ.

2.3.4 Kiến trúc ByteTrack

Kiến trúc ByteTrack bao gồm hai thành phần chính: Detector (YOLOX) và Associator (BYTE).

- Detector: Sử dụng YOLOX để phát hiện hộp giới hạn và điểm số từ ảnh đầu vào. Hình ảnh được chia thành lưới, trích xuất đặc trưng qua CNN backbone, sau đó dự đoán hộp và lớp.

- Associator (BYTE): Xử lý theo hai giai đoạn:

1. Phân loại phát hiện thành cao/thấp điểm số (ngưỡng $\tau = 0.6$).
2. Dự đoán vị trí tracklet bằng Kalman filter.
3. Liên kết giai đoạn 1: Sử dụng thuật toán Hungarian để khớp hộp cao điểm với tracklet dựa trên IoU hoặc Re-ID.
4. Liên kết giai đoạn 2: Khớp hộp thấp điểm với tracklet chưa khớp chỉ bằng IoU.
5. Quản lý track: Khởi tạo track mới từ hộp cao điểm chưa khớp, xóa track lâu không khớp (giữ tối đa 30 frame).

Thuật toán được huấn luyện trên các bộ dữ liệu như MOT17, CrowdHuman với kỹ thuật tăng cường dữ liệu (Mosaic, Mixup). Kết quả đầu ra là các quỹ đạo liên tục, phù hợp cho ứng dụng theo dõi video thời gian thực.

2.3.5 Bộ lọc Kalman và Thuật toán Hungary trong bám vết

Bộ lọc Kalman và Thuật toán Hungary là hai mảnh ghép toán học cốt lõi trong thuật toán bám vết đa đối tượng ByteTrack, giúp duy trì định danh (Tracking ID) của phương tiện giao thông liên tục qua các khung hình:

- Bộ lọc Kalman (Kalman Filter): Là một bộ lọc đệ quy tối ưu, được sử dụng để ước lượng trạng thái động học của phương tiện trong không gian 2D. Trạng thái của một phương tiện tại mỗi thời điểm được biểu diễn bằng vector trạng thái 8 chiều:

$$x = [x_c, y_c, a, h, v_x, v_y, v_a, v_h]^T$$

Trong đó, x_c, y_c là tọa độ tâm của hộp giới hạn (Bounding Box), a là tỷ lệ

khung hình (width/height), h là chiều cao hộp giới hạn, và các tham số còn lại đại diện cho vận tốc biến thiên tương ứng.

Quá trình lọc Kalman gồm hai bước lặp đệ quy liên tục:

- *Bước dự báo (Prediction)*: Dự báo trạng thái của xe ở khung hình tiếp theo dựa trên ma trận chuyển trạng thái F (mô hình chuyển động thẳng đều):

$$\begin{aligned}\hat{x}_{k|k-1} &= F P \hat{x}_{k-1|k-1} \\ P_{k|k-1} &= F P_{k-1|k-1} F^T + Q\end{aligned}$$

Với P là ma trận hiệp phương sai sai số ước lượng và Q là ma trận hiệp phương sai nhiễu hệ thống.

- *Bước cập nhật (Update)*: Hiệu chỉnh trạng thái dự báo bằng phép đo thực tế z_k (khung hộp giới hạn được phát hiện từ YOLOv11 ở khung hình hiện tại) thông qua hệ số tăng Kalman K_k :

$$\begin{aligned}K_k &= P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1} \\ \hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k (z_k - H \hat{x}_{k|k-1}) \\ P_{k|k} &= (I - K_k H) P_{k|k-1}\end{aligned}$$

Với H là ma trận đo lường và R là ma trận hiệp phương sai nhiễu đo lường.

- Thuật toán Hungary (Hungarian Algorithm): Khi đã có vị trí dự báo từ bộ lọc Kalman và vị trí phát hiện thực tế từ YOLOv11, hệ thống cần giải quyết bài toán liên kết dữ liệu (data association) để xác định hộp giới hạn nào thuộc về xe nào. Thuật toán Hungary được sử dụng để tìm phương án khớp tối ưu giữa tập hợp các vết bám (Tracks) và tập hợp các hộp giới hạn phát hiện mới (Detections) nhằm cực tiểu hóa chi phí khoảng cách IoU. Thuật toán hoạt động trên ma trận chi phí C kích thước $M \times N$ (với chi phí $C_{ij} = 1 - \text{IoU}(\text{Track}_i, \text{Detection}_j)$) và giải quyết bài toán phân công tối ưu với độ phức tạp $O(N^3)$, giúp loại bỏ các trường hợp hoán đổi định danh (ID Switch) khi các xe di chuyển đan xen nhau.

2.4 PHÉP BIẾN ĐỔI PHỐI CẢNH VÀ HOMOGRAPHY

2.4.1 Khái niệm phép biến đổi phối cảnh (Perspective Warping)

Trong thị giác máy tính, khi quan sát một mặt phẳng trong không gian ba chiều từ hai góc nhìn (camera) khác nhau, các điểm ảnh trên hai góc nhìn đó có một mối liên hệ hình học nhất định. Mối liên hệ này được biểu diễn thông qua phép biến đổi đồng hình (Homography) hay còn gọi là biến đổi phối cảnh (Perspective Warping). Kỹ thuật này cho phép ánh xạ tọa độ của các điểm từ mặt phẳng ảnh gốc sang một mặt phẳng ảnh đích, giúp tạo ra hiệu ứng thay đổi góc nhìn (perspective) của vật thể.

2.4.2 Cơ sở toán học của phép biến đổi Homography

Phép biến đổi Homography là một phép ánh xạ tuyến tính phẳng giữa hai mặt phẳng ảnh. Để biểu diễn sự thay đổi góc nhìn (phối cảnh), toán học giải tích sử dụng hệ tọa độ đồng nhất (Homogeneous Coordinates), bổ sung chiều thứ ba cho hệ tọa độ Descartes (x, y) thành $(x, y, 1)$.

Giả sử cần biến đổi điểm $P(x, y, 1)$ trên ảnh gốc sang điểm $P'(x', y', w')$ trên mặt phẳng đích. Phép ánh xạ này được xác định bởi ma trận vuông 3×3 , ký hiệu là H :

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Tọa độ thực tế x_{dst} , y_{dst} trên mặt phẳng đích được tính bằng cách chuẩn hóa các thành phần tọa độ cho phần tử trọng số w' :

$$x_{dst} = \frac{x'}{w'} = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$
$$y_{dst} = \frac{y'}{w'} = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

Do tính chất bất biến theo tỷ lệ, ma trận H có thể được chuẩn hóa bằng cách chia cho h_{33} ($h_{33} = 1$), khiến số lượng ẩn số giảm xuống còn 8 (tương đương 8 bậc tự do).

Để máy tính giải được hệ số của ma trận H , cần giải hệ phương trình tuyến tính dựa trên tối thiểu 4 cặp điểm tương đồng (không có 3 điểm nào thẳng hàng). Trong phạm vi đề án, 4 cặp điểm này chính là 4 góc của biển số xe. Sau khi tìm được ma trận H , toàn bộ điểm ảnh của bức ảnh biển số gốc sẽ được nhân với ma trận này để tự động bẻ góc, thay đổi kích thước và khớp hoàn toàn vào đuôi xe vi phạm.

2.4.3 Ứng dụng dán biển số phối cảnh giả lập trong phạt nguội

Trong thực tế triển khai hệ thống phạt nguội thông minh, một thách thức lớn là sự thiếu hụt bộ dữ liệu đồng bộ góc nhìn tại Việt Nam (một camera góc cao giám sát ngã tư và một camera góc thấp sát đường để trích xuất biển số). Kỹ thuật Homography được ứng dụng như một giải pháp cốt lõi để tạo ra luồng dữ liệu mô phỏng.

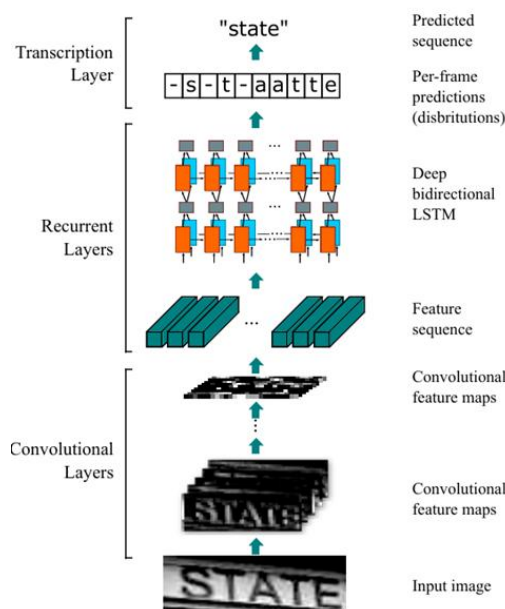
Khi hệ thống tại biên phát hiện phương tiện vượt đèn đỏ, ảnh chiếc xe sẽ được cắt ra (crop). Thay vì sử dụng một ảnh biển số nhìn thẳng không khớp với góc nghiêng của đuôi xe camera trên cao, hệ thống xác định 4 điểm tọa độ tại khu vực gắn biển số trên xe. Một ảnh biển số thực tế sẽ được đưa qua phép biến đổi Perspective Warping bằng ma trận H để làm nghiêng, thay đổi tỷ lệ sao cho khớp hoàn toàn với góc nhìn thực tế của camera. Bức ảnh hoàn chỉnh sau khi "dán" biển số ảo sẽ được đẩy lên máy chủ trung tâm để phục vụ công đoạn nhận diện ký tự quang học (OCR).

2.5 MẠNG NƠ-RON TÍCH CHẬP HỒI QUY

2.5.1 Khái niệm^[4]

Mạng nơ-ron tích chập hồi quy được cấu thành từ ba thành phần chính, bao gồm: các lớp tích chập (Convolutional layers), các lớp tuần tự (Recurrent layers) và lớp phiên âm (Transcription layer), được sắp xếp theo thứ tự từ dưới lên trên. Ở tầng dưới cùng, các lớp tích chập có nhiệm vụ tự động trích xuất chuỗi đặc trưng từ hình ảnh đầu vào. Tiếp theo, trên nền tảng các đặc trưng đã được trích xuất, các lớp tuần tự được sử dụng để mô hình hóa sự phụ thuộc theo chuỗi và đưa ra dự đoán cho từng khung trong chuỗi đặc trưng. Cuối cùng, lớp phiên âm ở

tầng trên cùng có chức năng chuyển đổi các dự đoán theo từng khung thành một chuỗi nhãn hoàn chỉnh. Mặc dù CRNN là sự kết hợp của nhiều kiến trúc mạng khác nhau (chẳng hạn như CNN và RNN), toàn bộ mô hình có thể được huấn luyện một cách đồng thời thông qua việc tối ưu một hàm mất mát duy nhất.



Hình 2.3 Kiến trúc của mạng CRNN

2.5.2 Cấu tạo từng thành phần của mạng

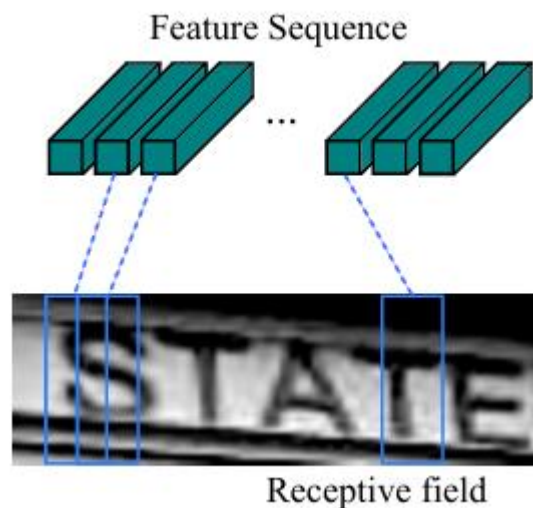
2.5.2.1 Trích xuất chuỗi đặc trưng (Feature Sequence Extraction) ^[4]

Trong mô hình CRNN, thành phần các lớp tích chập được xây dựng dựa trên việc kế thừa các lớp tích chập và lớp gộp cực đại từ một mạng CNN tiêu chuẩn, trong khi các lớp kết nối đầy đủ (fully connected) được loại bỏ. Thành phần này đóng vai trò trích xuất biểu diễn đặc trưng mang tính tuần tự từ hình ảnh đầu vào. Trước khi đưa vào mạng, tất cả các hình ảnh được chuẩn hóa về cùng một chiều cao nhằm đảm bảo tính nhất quán trong quá trình xử lý.

Từ các bản đồ đặc trưng thu được sau quá trình tích chập, một chuỗi các vector đặc trưng sẽ được trích xuất và sử dụng làm đầu vào cho các lớp tuần tự. Cụ thể, mỗi vector đặc trưng trong chuỗi được hình thành bằng cách quét các bản đồ đặc trưng theo chiều từ trái sang phải, theo từng cột. Điều này có nghĩa là

vector đặc trưng thứ i được tạo thành từ sự kết hợp của các giá trị tại cột thứ i trên tất cả các bản đồ đặc trưng. Trong thiết lập này, chiều rộng của mỗi cột được cố định là một pixel.

Do các phép tích chập, gộp cực đại và các hàm kích hoạt theo phần tử chỉ tác động trên các vùng cục bộ, chúng có tính bất biến đối với các phép tịnh tiến. Vì vậy, mỗi cột trên bản đồ đặc trưng tương ứng với một vùng hình chữ nhật xác định trên ảnh gốc, được gọi là trường tiếp nhận (receptive field). Các vùng này được sắp xếp theo cùng thứ tự với các cột tương ứng trên bản đồ đặc trưng, từ trái sang phải. Như minh họa trong hình 2.4, mỗi vector trong chuỗi đặc trưng tương ứng với một trường tiếp nhận cụ thể, và có thể được xem như một biểu diễn mô tả cho vùng ảnh tương ứng đó.



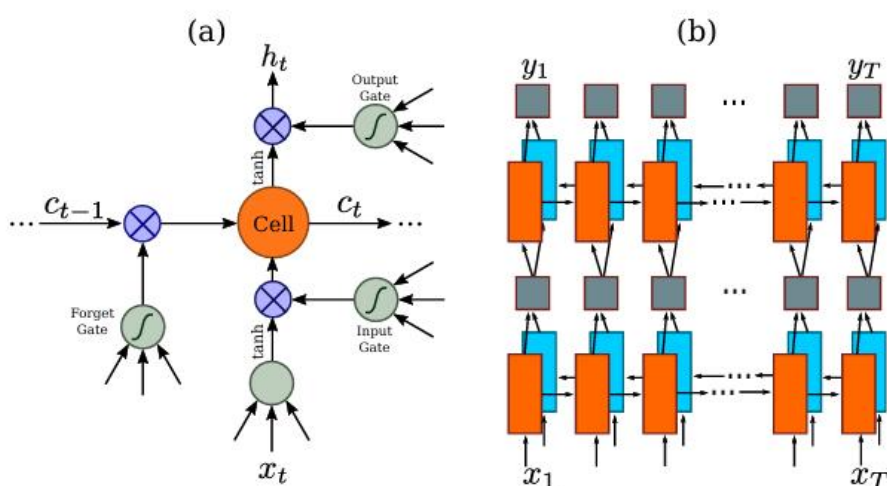
Hình 2.4 The receptive field

2.5.2.2 Gán nhãn chuỗi (Sequence Labeling) ^[4]

Trong mô hình CRNN, một mạng nơ-ron hồi tiếp hai chiều (Bidirectional Recurrent Neural Network – BiRNN) được xây dựng phía trên các lớp tích chập và đóng vai trò là các lớp tuần tự. Các lớp này có nhiệm vụ dự đoán phân phối nhãn y_t cho từng khung đặc trưng x_t trong chuỗi đặc trưng $x=(x_1,...,x_T)$.

Việc sử dụng mạng hồi tiếp mang lại một số ưu điểm quan trọng. Thứ nhất, RNN có khả năng khai thác hiệu quả thông tin ngữ cảnh trong chuỗi dữ liệu. Điều này đặc biệt hữu ích trong các bài toán nhận dạng chuỗi dựa trên hình ảnh, khi việc tận dụng ngữ cảnh giúp tăng độ ổn định và độ chính xác so với việc xử lý từng ký hiệu một cách độc lập. Chẳng hạn, trong bài toán nhận dạng văn bản trong ảnh tự nhiên, một số ký tự có độ rộng lớn có thể cần nhiều khung liên tiếp để được biểu diễn đầy đủ. Bên cạnh đó, các ký tự dễ gây nhầm lẫn có thể được phân biệt rõ ràng hơn khi xét đến ngữ cảnh xung quanh, ví dụ như việc nhận diện chuỗi “il” thông qua sự so sánh chiều cao giữa các ký tự.

Thứ hai, RNN cho phép lan truyền ngược sai số đến các lớp phía dưới, cụ thể là các lớp tích chập, từ đó hỗ trợ việc huấn luyện đồng thời toàn bộ mô hình theo cách thống nhất. Thứ ba, RNN có khả năng xử lý các chuỗi có độ dài biến đổi, cho phép mô hình linh hoạt trong việc xử lý các đầu vào có kích thước khác nhau.



Hình 2.5. (a) Cấu trúc của một đơn vị LSTM cơ bản. (b) Cấu trúc của LSTM hai chiều sâu (deep bidirectional) trong mạng CRNN

Một đơn vị RNN truyền thống bao gồm một lớp ẩn có kết nối hồi tiếp, nằm giữa lớp đầu vào và lớp đầu ra. Tại mỗi thời điểm, khi nhận được một khung đặc trưng x_t trong chuỗi, trạng thái ẩn h_t được cập nhật thông qua một hàm phi tuyến phụ thuộc vào cả đầu vào hiện tại và trạng thái trước đó, theo công thức $h_t = g(x_t, h_{t-1})$

Sau đó, dự đoán y_t được suy ra dựa trên trạng thái h_t . Cơ chế này cho phép mô hình khai thác thông tin ngữ cảnh từ các bước thời gian trước $\{x_t\}_{t' < t}$ phục vụ cho việc dự đoán tại thời điểm hiện tại. Tuy nhiên, RNN truyền thống thường gặp phải vấn đề tiêu biến gradient, làm hạn chế khả năng ghi nhớ các phụ thuộc dài hạn và gây khó khăn trong quá trình huấn luyện.

Để khắc phục hạn chế này, mô hình Long Short-Term Memory (LSTM) đã được đề xuất như một biến thể đặc biệt của RNN. Một đơn vị LSTM bao gồm một ô nhớ (memory cell) cùng với ba cổng điều khiển, bao gồm cổng vào (input gate), cổng ra (output gate) và cổng quên (forget gate). Về mặt chức năng, ô nhớ có nhiệm vụ lưu trữ thông tin ngữ cảnh trong thời gian dài, trong khi các cổng điều khiển quyết định việc cập nhật, duy trì hoặc loại bỏ thông tin trong ô nhớ. Nhờ cấu trúc này, LSTM có khả năng nắm bắt hiệu quả các phụ thuộc dài hạn trong chuỗi dữ liệu, điều thường xuyên xuất hiện trong các bài toán nhận dạng dựa trên hình ảnh.

Tuy nhiên, LSTM một chiều chỉ khai thác thông tin theo hướng quá khứ. Trong khi đó, đối với các chuỗi đặc trưng từ hình ảnh, ngữ cảnh từ cả hai phía (trước và sau) đều có vai trò quan trọng và mang tính bổ sung cho nhau. Do đó, hai mạng LSTM theo hai hướng tiến và lùi thường được kết hợp để tạo thành mô hình LSTM hai chiều (Bidirectional LSTM – BiLSTM). Ngoài ra, nhiều lớp BiLSTM có thể được xếp chồng lên nhau, hình thành kiến trúc BiLSTM sâu, giúp mô hình học được các biểu diễn trừu tượng ở mức cao hơn và cải thiện đáng kể hiệu năng.

Trong quá trình huấn luyện, sai số được lan truyền ngược qua các lớp hồi tiếp theo cơ chế lan truyền ngược theo thời gian (Backpropagation Through Time – BPTT). Ở tầng thấp nhất của các lớp hồi tiếp, các sai phân lỗi được chuyển đổi ngược lại thành dạng bản đồ đặc trưng, tương ứng với quá trình chuyển đổi từ bản đồ sang chuỗi đặc trưng ban đầu và tiếp tục được truyền về các lớp tích chập. Trong thực tế, một lớp trung gian thường được xây dựng để kết nối giữa phần tích chập và phần hồi tiếp, thường được gọi là lớp “Map-to-Sequence”, đóng vai

trò chuyển đổi các bản đồ đặc trưng thành chuỗi đặc trưng phù hợp cho đầu vào của các lớp hồi tiếp.

2.5.2.3 Phiên âm (Transcription) ^[4]

Phiên âm là quá trình chuyển đổi các dự đoán theo từng khung thời gian do mạng RNN tạo ra thành một chuỗi nhãn hoàn chỉnh. Về mặt toán học, bài toán phiên âm được biểu diễn như việc tìm kiếm chuỗi nhãn có xác suất lớn nhất dựa trên các phân phối xác suất theo từng khung thời gian.

Trong thực tế, quá trình phiên âm có thể được thực hiện theo hai chế độ chính, bao gồm: phiên âm không dựa trên từ điển và phiên âm dựa trên từ điển. Từ điển trong ngữ cảnh này được hiểu là một tập hợp các chuỗi nhãn hợp lệ mà kết quả dự đoán phải tuân theo, ví dụ như một bộ từ vựng trong bài toán kiểm tra chính tả.

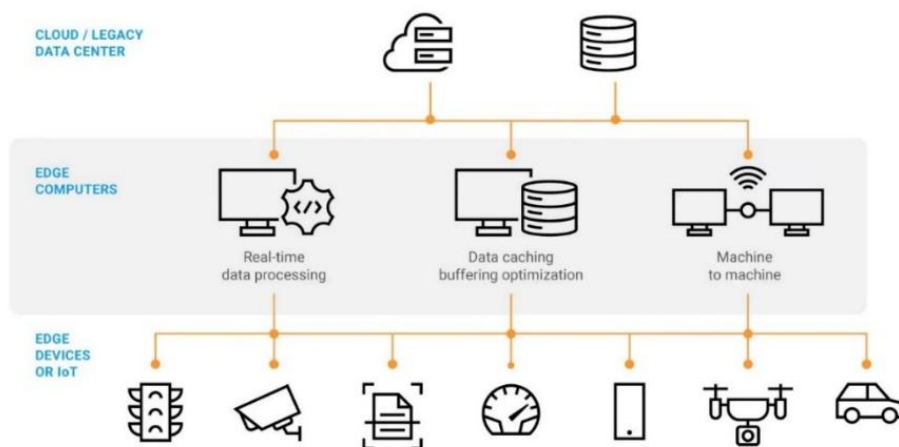
Ở chế độ không dựa trên từ điển, mô hình thực hiện dự đoán hoàn toàn dựa trên xác suất đầu ra mà không áp dụng bất kỳ ràng buộc nào từ tập từ vựng. Ngược lại, trong chế độ dựa trên từ điển, quá trình phiên âm được thực hiện bằng cách lựa chọn chuỗi nhãn có xác suất cao nhất trong phạm vi các chuỗi hợp lệ được xác định trước bởi từ điển. Điều này giúp cải thiện độ chính xác của kết quả dự đoán trong các bài toán có tập nhãn giới hạn.

2.6 TỔNG QUAN VỀ ĐIỆN TOÁN BIÊN (EDGE COMPUTING)

2.6.1 Khái niệm Điện toán biên ^[11]

Trong các hệ thống Internet vạn vật truyền thống, dữ liệu thu thập từ các cảm biến hoặc camera thường được gửi toàn bộ lên máy chủ đám mây (Cloud) để xử lý. Tuy nhiên, khi số lượng thiết bị tăng lên, việc truyền tải khối lượng dữ liệu khổng lồ liên tục sẽ gây ra hiện tượng nghẽn mạng và tăng độ trễ (latency). Điện toán biên (Edge Computing) ra đời như một giải pháp khắc phục triệt để hạn chế này. Thay vì đẩy dữ liệu đi xa, Điện toán biên mang khả năng tính toán, lưu trữ và phân tích dữ liệu về gần với "rìa" của mạng lưới (gần với nguồn phát sinh dữ liệu nhất). Các thiết bị tại biên (Edge Devices) được trang bị vi xử lý để tự đưa ra

quyết định tại chỗ theo thời gian thực mà không cần phụ thuộc hoàn toàn vào máy chủ trung tâm.



Hình 2.6 Kiến trúc tổng quan mô hình phân tán Edge-Cloud.

2.6.2 Kiến trúc cơ bản của Điện toán biên

Hệ thống điện toán biên thường được tổ chức theo mô hình phân tầng từ ngoài vào trong, giúp tối ưu hóa luồng dữ liệu và giảm tải cho mạng lõi. Một kiến trúc phân tán tiêu chuẩn thường bao gồm 3 tầng (Tiers) vật lý chính:

- Tầng thiết bị ngoại vi (Edge Devices/Sensors): Đây là các thiết bị thu thập dữ liệu trực tiếp từ môi trường vật lý như camera giám sát, cảm biến hoặc vi điều khiển thực thi. Chúng có khả năng tính toán rất cơ bản, chủ yếu làm nhiệm vụ ghi nhận tín hiệu và phản hồi lệnh vật lý tức thời.
- Tầng nút biên (Edge Nodes/Gateways): Đóng vai trò là trung tâm điều phối tại hiện trường. Các Edge Node được trang bị vi xử lý đủ mạnh để chạy các mô hình học sâu nhẹ (Edge AI), sàng lọc dữ liệu, đưa ra quyết định tự trị theo thời gian thực và đóng gói dữ liệu quan trọng trước khi truyền đi, đóng vai trò như một cầu nối (Gateway) giao tiếp.
- Tầng đám mây hoặc Máy chủ trung tâm (Cloud/Central Server): Nơi sở hữu năng lực điện toán và lưu trữ lớn. Tầng này không xử lý toàn bộ dữ liệu thô liên tục mà chỉ tiếp nhận các gói dữ liệu đã được tinh lọc từ tầng biên để thực hiện

các tác vụ suy luận AI phức tạp, quản lý cơ sở dữ liệu tập trung và cung cấp giao diện giám sát tổng thể.

2.6.3 Các ưu điểm cốt lõi

Mô hình Điện toán biên mang lại những lợi ích vượt trội, đặc biệt trong các bài toán yêu cầu xử lý thời gian thực như giám sát giao thông:

- Giảm thiểu độ trễ (Low Latency): Dữ liệu được phân tích ngay tại thiết bị cục bộ, giúp hệ thống đưa ra phản hồi gần như ngay lập tức.
- Tiết kiệm băng thông mạng (Bandwidth Optimization): Thay vì truyền luồng video độ phân giải cao liên tục lên máy chủ, thiết bị biên chỉ cần gửi đi những kết quả đã được trích xuất (ví dụ: chuỗi văn bản cảnh báo, số lượng xe, hoặc hình ảnh xe vi phạm đã được cắt nhỏ).
- Giảm tải cho máy chủ đám mây: Phân tán khối lượng tính toán giúp máy chủ trung tâm không bị quá tải khi phải phục vụ hàng ngàn thiết bị cùng lúc.

2.6.4 Ứng dụng kiến trúc Edge-Cloud trong hệ thống AIoT

Trong lĩnh vực giao thông thông minh, việc kết hợp giữa Điện toán biên (Edge) và Điện toán đám mây (Cloud) tạo ra một hệ thống mạng vi dịch vụ hoàn chỉnh. Áp dụng vào phạm vi của đề tài, hệ thống được thiết kế dựa trên kiến trúc phân tán này. Cụ thể, thiết bị máy tính nhúng (Raspberry Pi) đóng vai trò là thiết bị biên (Edge AI), chịu trách nhiệm trực tiếp chạy các mô hình học sâu nhẹ (YOLO) để phân tích luồng video camera, đếm lưu lượng và điều khiển đèn tín hiệu tại chỗ. Tầng điện toán đám mây (Cloud/Server) được đảm nhiệm bởi máy tính trung tâm có cấu hình mạnh, chỉ nhận các luồng dữ liệu nhỏ (gói tin vi phạm) từ biên gửi lên để chạy các mô hình AI nhận diện ký tự phức tạp (CRNN). Sự kết hợp này giúp hệ thống vừa đạt được tốc độ phản hồi tức thời, vừa giải quyết được bài toán giới hạn phân cứng.

2.7 TỐI ƯU HÓA MÔ HÌNH AI TRÊN THIẾT BỊ NHÚNG

2.7.1 Lượng tử hóa mô hình (Model Quantization)

Trong quá trình huấn luyện bằng các framework học sâu, trọng số (weights) và độ kích hoạt (activations) của mạng nơ-ron thường được biểu diễn và lưu trữ dưới dạng dấu phẩy động độ chính xác đơn 32-bit (FP32). Lượng tử hóa (Quantization) là kỹ thuật tối ưu hóa sau huấn luyện (Post-Training Quantization), thực hiện ánh xạ các dải giá trị FP32 này sang một không gian dữ liệu rời rạc có độ chính xác thấp hơn, phổ biến nhất là dấu phẩy động 16-bit (FP16) hoặc số nguyên 8-bit (INT8).

Bản chất toán học của quá trình lượng tử hóa tuyến tính thường được biểu diễn qua phương trình ánh xạ:

$$x_{float} = S \times (q_{int} - Z)$$

Trong đó:

- x_{float} là giá trị thực tế ở định dạng FP32.
- q_{int} là giá trị nguyên sau khi lượng tử hóa (ví dụ từ -128 đến 127 đối với INT8).
- S (Scale) là hệ số tỷ lệ dương, xác định bước nhảy lượng tử.
- Z (Zero-point) là giá trị nguyên tương ứng với mốc 0 của dữ liệu thực, giúp đảm bảo phép toán với số 0 không sinh ra sai số.

Đối với hệ thống triển khai trên thiết bị biên như Raspberry Pi, vi xử lý ARM thường gặp "nghẽn cổ chai" về băng thông bộ nhớ và không có các lõi Tensor chuyên dụng để tính toán ma trận FP32 tốc độ cao. Việc ép cân mô hình từ định dạng gốc .pt sang các định dạng suy luận tối ưu cho IoT (như NCNN, ONNX hoặc TFLite) mang lại ba lợi ích:

1. Giảm dung lượng: Đưa trọng số về INT8 giúp giảm dung lượng lưu trữ của mô hình đi 4 lần.
2. Tiết kiệm RAM: Giảm tải áp lực truyền dữ liệu giữa CPU và bộ nhớ đệm (Cache).
3. Tăng tốc độ suy luận (Inference Speed): Phép toán số nguyên (Integer Math) được thực thi nhanh hơn rất nhiều so với dấu phẩy động, đặc biệt khi tận dụng được các tập lệnh tối ưu phần cứng trên chip ARM.

Dù quá trình lượng tử hóa phải đánh đổi bằng một sự sụt giảm rất nhỏ về độ chính xác trung bình (mAP), sự cải thiện đột phá về tốc độ khung hình (FPS) khiến kỹ thuật này trở thành bước thiết yếu để triển khai các mô hình phát hiện đối tượng thời gian thực trên môi trường Edge AI.

2.7.2 Kỹ thuật lược bỏ nhịp khung hình (Frame Skipping)

Trong xử lý thị giác máy tính theo thời gian thực, luồng video đầu vào từ camera thường đạt tốc độ tiêu chuẩn là 30 khung hình/giây (FPS). Điều này đồng nghĩa với việc vi xử lý chỉ có khoảng 33 mili-giây để hoàn thành toàn bộ chu trình suy luận (Inference) cho một khung hình. Đối với phần cứng nhúng có giới hạn về xung nhịp và bộ nhớ đệm như Raspberry Pi, việc ép CPU liên tục xử lý mô hình học sâu ở tần suất này sẽ lập tức dẫn đến hiện tượng quá tải nhiệt (Thermal Throttling), gây nghẽn cổ chai và làm trễ cục bộ toàn hệ thống.

Kỹ thuật lược bỏ nhịp khung hình (Frame Skipping) được áp dụng như một cơ chế điều tiết luồng dữ liệu (Data Throttling) chủ động nhằm giảm tải phần cứng. Thay vì đẩy toàn bộ luồng ảnh qua mạng nơ-ron, hệ thống sẽ bỏ qua k khung hình liên tiếp và chỉ trích xuất khung hình thứ $k+1$ để đưa vào xử lý.

Giả sử tốc độ đầu vào của camera là FPS_{in} , tần suất suy luận thực tế qua mô hình AI FPS_{out} được xác định bằng công thức:

$$FPS_{out} = \frac{FPS_{in}}{k + 1}$$

Trong mô hình thực nghiệm của đề án, tham số được tinh chỉnh ở mức $k = 1$ hoặc $k = 2$, giúp hạ tần suất xử lý AI lõi xuống ngưỡng an toàn từ 10 FPS đến 15 FPS. Quãng thời gian tính toán dư ra ở các khung hình bị bỏ qua được CPU phân bổ để xử lý các luồng I/O bất đồng bộ khác (như gửi lệnh điều khiển MQTT hoặc truyền ảnh vi phạm qua FastAPI).

Tuy nhiên, việc giảm tần số quét thông thường sẽ làm đứt gãy quá trình bám sát phương tiện. Để giải quyết triệt để bài toán này, hệ thống tích hợp cơ chế Frame Skipping với thuật toán theo dõi đa đối tượng ByteTrack. Trong khi YOLOv11n chỉ cập nhật lại hộp giới hạn (Bounding Box) ở các khung hình được chọn, bộ lọc Kalman (Kalman Filter) bên trong ByteTrack sẽ đảm nhiệm việc

tính toán vận tốc và nội suy quỹ đạo tuyến tính của xe trong các khung hình bị bỏ qua.

Sự kết hợp này tạo ra một luồng xử lý khép kín: mô hình phát hiện (Detector) được giảm tải tới 50-66% khối lượng tính toán, trong khi mô hình theo dõi (Tracker) vẫn liên tục cập nhật tọa độ, giúp hệ thống duy trì định danh (Track ID) ổn định mà không làm suy giảm tính mượt mà của tổng thể luồng giám sát giao thông.

2.7.3 Kỹ thuật lập trình đa luồng (Multithreading) và Hàng đợi (Queue)

Trong kiến trúc phần mềm tuần tự truyền thống, hệ thống phải hoàn thành trọn vẹn một chu trình theo thứ tự: đọc ảnh từ camera, chạy suy luận AI, và chờ gửi/nhận dữ liệu qua mạng trước khi bắt đầu khung hình tiếp theo. Quá trình này sinh ra hiện tượng nghẽn I/O (Input/Output Blocking), trong đó các tác vụ tính toán lõi bị đình trệ chỉ để chờ tín hiệu phản hồi từ phần cứng hoặc đường truyền mạng, gây ra độ trễ (latency) nghiêm trọng cho toàn hệ thống.

Để khắc phục điểm nghẽn này, hệ thống áp dụng kỹ thuật lập trình đa luồng (Multithreading) kết hợp với cấu trúc dữ liệu Hàng đợi (Queue), được xây dựng dựa trên mẫu thiết kế Sản xuất - Tiêu thụ (Producer-Consumer Pattern). Khối lượng công việc nguyên khối được phân rã thành các luồng (Thread) độc lập, chạy song song và giao tiếp với nhau thông qua bộ đệm bộ nhớ chung một cách an toàn.

Cụ thể, kiến trúc phần mềm tại thiết bị biên (Raspberry Pi) được cấu trúc thành 3 luồng xử lý:

1. Luồng thu thập (Producer Thread): Chuyên trách giao tiếp với camera, liên tục đọc khung hình thô và đẩy (push) vào Hàng đợi. Luồng này hoạt động ở tốc độ quét vật lý của camera mà không bị cản trở bởi thời gian xử lý AI.
2. Luồng suy luận AI (Consumer Thread): Đóng vai trò tiêu thụ, liên tục lấy (pop) các khung hình mới nhất từ Hàng đợi để đưa qua mạng YOLOv11n và thuật toán ByteTrack. Việc tách rời luồng này đảm bảo vi

xử lý luôn được cấp dữ liệu để tính toán liên tục, tối đa hóa thông lượng (Throughput).

3. Luồng truyền thông (Network I/O Thread): Nhận kết quả từ luồng AI để thực thi các tác vụ truyền tải mạng bất đồng bộ (Async) như gửi lệnh MQTT đến vi điều khiển ESP32 hay đẩy gói dữ liệu API lên máy chủ trung tâm. Nếu xảy ra sự cố độ trễ mạng, luồng này tự động chờ mà không làm "đóng băng" (freeze) quá trình ghi hình hay giám sát.

Trong kiến trúc này, Hàng đợi (Queue) đóng vai trò là vùng đệm an toàn (Thread-safe Buffer). Nó giúp hấp thụ sự chênh lệch về tốc độ xử lý giữa luồng I/O và luồng AI, ngăn chặn tình trạng tràn bộ nhớ và loại bỏ rủi ro xung đột tài nguyên (Race Condition). Sự kết hợp đa luồng này cho phép tận dụng triệt để kiến trúc đa nhân của vi xử lý ARM, đảm bảo hệ thống giám sát phân tán vận hành mượt mà theo đúng thời gian thực.

2.7.4 Lý thuyết về hệ điều hành thời gian thực FreeRTOS và cơ chế đa nhiệm

Trong các ứng dụng điều khiển thời gian thực trên vi điều khiển có tài nguyên hạn chế như ESP32, việc sử dụng vòng lặp tuần tự truyền thống sẽ gặp hạn chế lớn khi phải thực hiện nhiều tác vụ đồng thời (như vừa quét nút nhấn, vừa cập nhật màn hình hiển thị TM1637, vừa duy trì kết nối mạng MQTT và gửi dữ liệu lên Firebase). Hệ điều hành thời gian thực FreeRTOS (Free Real-Time Operating System) được tích hợp để giải quyết bài toán này:

- Lập lịch ưu tiên (Preemptive Scheduling): Nhân FreeRTOS quản lý các tác vụ (Tasks) chạy độc lập và phân bổ thời gian sử dụng CPU dựa trên độ ưu tiên (Task Priorities). Tác vụ có độ ưu tiên cao hơn sẽ lập tức chiếm quyền xử lý của tác vụ có độ ưu tiên thấp hơn (preemption), đảm bảo các tác vụ quan trọng như điều khiển đèn tín hiệu và quét nút bấm khẩn cấp luôn được thực thi đúng hạn (real-time constraints).
- Đa nhiệm song song trên ESP32 (Multi-core Execution): Vi xử lý ESP32 sở hữu 2 nhân Xtensa 32-bit LX6 hoạt động độc lập. FreeRTOS hỗ trợ gán các tác vụ cụ thể vào các nhân xử lý riêng biệt thông qua hàm `xTaskCreatePinnedToCore()`. Đồ án tận dụng kiến trúc này để gán tác vụ

điều khiển đèn giao thông vật lý chạy ổn định trên Core 1, và tác vụ giao tiếp Firebase chạy trên Core 0, giúp triệt tiêu hoàn toàn hiện tượng nghẽn I/O mạng gây trễ chu kỳ đèn.

- Hàng đợi an toàn luồng (FreeRTOS Queue): Để trao đổi dữ liệu an toàn giữa các tác vụ chạy trên hai nhân khác nhau mà không xảy ra tranh chấp tài nguyên (race condition), FreeRTOS cung cấp cơ chế hàng đợi Queue. Hàng đợi hoạt động theo nguyên lý FIFO (First-In, First-Out) và được bảo vệ bằng các cờ đồng bộ nội bộ, cho phép luồng chính ghi dữ liệu ngã tư vào hàng đợi một cách bất đồng bộ, trong khi luồng phụ đọc dữ liệu từ hàng đợi để đẩy lên cloud một cách độc lập.

2.8 MỘT SỐ LINH KIỆN VÀ MODULE TRONG HỆ THỐNG

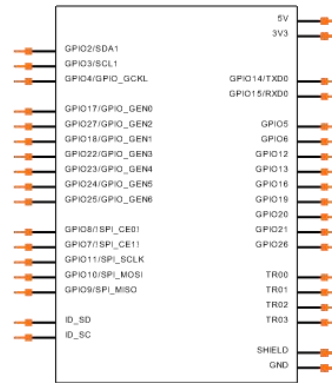
2.8.1 Raspberry Pi 4^[4]

Raspberry Pi là một máy tính nhỏ gọn, giá cả phải chăng và tiêu thụ ít năng lượng. Mỗi chiếc Raspberry Pi có thể hoạt động như một máy tính đầy đủ, có thể kết nối với màn hình, bàn phím, chuột và các thiết bị khác qua cổng USB và HDMI. Được xây dựng trên nền tảng ARM và chạy hệ điều hành Linux, Raspberry Pi có nhiều phiên bản khác nhau.

Phiên bản Raspberry Pi 4 có nhiều cải tiến về hiệu suất và tính năng so với các phiên bản trước đó. Với kiến trúc lõi tứ và các cổng kết nối đa dạng, Raspberry Pi 4 đã trở thành một công cụ đa năng có thể được sử dụng trong nhiều lĩnh vực như xây dựng máy tính nhúng, IoT, phát triển web, robotics, hệ thống giám sát và quản lý, và nhiều ứng dụng khác.



Hình 2.7 Raspberry pi 4 model B



Hình 2.8 Kí hiệu nguyên lý Raspberry pi 4 model B

Thông số kỹ thuật của Raspberry pi 4 model B được mô tả chi tiết ở bảng 2.1:

Bảng 2.1 Thông số kỹ thuật của Raspberry pi 4 model B

STT	Tên thông số	Giá trị / Đặc tính kỹ thuật	Ghi chú
1	Model	Raspberry Pi 4 Model B	Phiên bản 8GB RAM
2	Bộ vi xử lý (CPU)	Broadcom BCM2711, Quad-core Cortex-A72 (ARM v8, 64-bit), 1.5 GHz	Kiến trúc ARMv8-A, 64-bit
3	Bộ nhớ RAM	8 GB LPDDR4-3200 SDRAM	Có các bản 2 GB, 4 GB và 8 GB
4	GPU	Broadcom VideoCore VI	Hỗ trợ OpenGL ES 3.0, 4K video decode H.265
5	Cổng USB	2 × USB 3.0, 2 × USB 2.0	Tổng cộng 4 cổng USB
6	Cổng mạng (Ethernet)	Gigabit Ethernet (1000 Mbps)	Cải thiện đáng kể so với Pi 3

7	Wi-Fi	IEEE 802.11 b/g/n/ac (2.4 GHz & 5.0 GHz)	Dual-band Wi-Fi 5
8	Bluetooth	Bluetooth 5.0, BLE	Kết nối thiết bị không dây
9	Cổng hiển thị	2 × Micro-HDMI	Hỗ trợ độ phân giải tối đa 2× 4K@30 Hz hoặc 1× 4K@60 Hz
10	Cổng camera	1 × CSI (Camera Serial Interface)	Dành cho Camera Module
11	Cổng màn hình (DSI)	1 × DSI (Display Serial Interface)	Kết nối màn hình cảm ứng
12	Cổng GPIO	40 chân GPIO (2 × 20)	Tương thích với các HAT chuẩn
13	Bộ nhớ lưu trữ	MicroSD card slot	Dùng để cài OS và lưu dữ liệu
14	Nguồn điện	USB-C (5 V = 3 A DC)	Có thể cấp qua GPIO (5 V)
15	Cổng âm thanh / video	Jack 3.5 mm 4-pole (stereo + composite video)	Xuất âm thanh và AV
16	Kích thước	85 mm × 56 mm × 17 mm	Chuẩn HAT compatible
17	Trọng lượng	Khoảng 46 g	Không bao gồm vỏ và tản nhiệt
18	Hệ điều hành hỗ trợ	Raspberry Pi OS, Ubuntu, Debian, Kali Linux, v.v.	Cài qua Raspberry Pi Imager

19	Hiệu năng video	H.265 (4K@60 fps) decode, H.264 1080p@60 fps encode/decode	Xuất song song qua HDMI
20	Tản nhiệt / Nhiệt độ	Có thể cần quạt hoặc heatsink khi tải cao	Để tránh throttling khi CPU > 80 °C

2.8.2 ESP32-WROOM-32E^[5]

Bo mạch ESP32-DevKitC (38-pin) là board phát triển chính thức của Espressif, tích hợp module ESP32-WROOM-32E – SoC mạnh mẽ hỗ trợ Wi-Fi 2.4 GHz (IEEE 802.11 b/g/n) và Bluetooth v4.2 BR/EDR & BLE. Board hướng tới nhu cầu nghiên cứu, học tập và phát triển nhanh các ứng dụng IoT, robot, nhà thông minh, điều khiển tự động.

Thiết kế nhỏ gọn, nạp chương trình qua cổng Micro-USB, hoạt động ổn định, tiêu thụ điện năng thấp và xử lý song song nhờ 2 lõi CPU (dual-core) tốc độ cao.



Hình 2.9 Bo mạch điều khiển ESP32-WROOM-32E



Hình 2.10 Ký hiệu nguyên lý vi điều khiển ESP32-WROOM-32E

Bo mạch sử dụng vi xử lý 32-bit Xtensa® LX6 chạy tối đa 240 MHz, 520 KB SRAM, flash 4 MB (tùy phiên bản có thể lớn hơn). Tích hợp kết nối không

dây Wi-Fi 2.4 GHz và Bluetooth v4.2. Cung cấp >30 chân GPIO đa năng có thể cấu hình làm ADC, DAC, PWM, SPI, I²C, UART, ...

Nguồn hoạt động 3.3 V (cấp qua USB 5 V hoặc chân VIN/5V), trên board có mạch ổn áp hạ xuống 3.3 V (thường dùng AMS1117/ME6211). Trên board tích hợp mạch nạp USB-UART (CP2102/CP2102N), hỗ trợ nạp và debug trực tiếp từ máy tính.

Thông số kỹ thuật của bo mạch điều khiển ESP32-WROOM-32E được thể hiện chi tiết trong bảng 2.2

Bảng 2.2 Thông số kỹ thuật của bo mạch điều khiển ESP32-Wroom-32E

STT	Tên thông số	Giá trị / Đặc tính kỹ thuật	Ghi chú
1	Model	ESP32-WROOM-32E	Phiên bản 38 chân phổ biến
2	Bộ vi xử lý (CPU)	Xtensa® dual-core 32-bit LX6, 240 MHz	Hiệu năng cao, hỗ trợ chế độ tiết kiệm năng lượng
3	Bộ nhớ Flash	4 MB hoặc 8 MB (tích hợp sẵn)	Tùy từng module
4	SRAM	520 KB	RAM nội cho ứng dụng và dữ liệu
5	ROM	448 KB	Chứa firmware khởi động và chức năng hệ thống
6	PSRAM (tùy chọn)	Tối đa 8 MB	Chỉ có trên một số phiên bản

7	Wi-Fi	IEEE 802.11 b/g/n (2.4 GHz)	Tốc độ dữ liệu đến 150 Mbps
8	Bluetooth	Bluetooth v4.2 BR/EDR và BLE	Tương thích thiết bị BLE và Classic
9	Số chân GPIO	38 chân (30 GPIO khả dụng)	Hỗ trợ PWM, I2C, SPI, UART, ADC, DAC, Touch
10	ADC	12-bit SAR ADC, tối đa 18 kênh	Độ phân giải cao cho cảm biến analog
11	DAC	2 × 8-bit DAC	Phát tín hiệu analog
12	SPI	4 SPI interface (HSPI, VSPI...)	Hỗ trợ giao tiếp ngoại vi tốc độ cao
13	I2C	2 interface I2C	Cấu hình linh hoạt qua phần mềm
14	UART	3 UART interface	UART0, UART1, UART2
15	PWM	16 kênh PWM (LEDC)	Điều khiển LED, động cơ
16	Touch Sensor	10 kênh cảm ứng điện dung	Ứng dụng cảm ứng chạm
17	Nguồn hoạt động	3.0 V – 3.6 V (điện hình 3.3 V)	Khuyến nghị 3.3 V ± 5%

18	Mức logic I/O	3.3 V TTL	Không tương thích trực tiếp 5 V
19	Công suất tiêu thụ	Active: ~240 mA, Deep sleep: ~10 μ A	Phụ thuộc chế độ hoạt động
20	Nhiệt độ hoạt động	-40 °C đến +85 °C	Phù hợp môi trường công nghiệp
21	Kích thước module	25.5 mm $\times$ 18 mm $\times$ 3.1 mm	Bao gồm ăng-ten PCB
22	Ăng-ten	PCB trace antenna	Tích hợp sẵn, có thể dùng anten ngoài qua U.FL
23	Chuẩn chứng nhận	CE, FCC, SRRC, IC, ELEC	Tùy nhà sản xuất
24	Hệ điều hành hỗ trợ	ESP-IDF, Arduino, MicroPython, PlatformIO	Tùy theo mục đích phát triển

Bo mạch ESP32-DevkitC-32D được thiết kế với 2 hàng chân mở rộng (Header 2 $\times$ 19), cung cấp tổng cộng 38 chân (pin) để kết nối với các thiết bị ngoại vi. Các chân này được bố trí đối xứng hai bên bo mạch, giúp người dùng dễ dàng kết nối vào breadboard hoặc hàn mạch thử nghiệm.

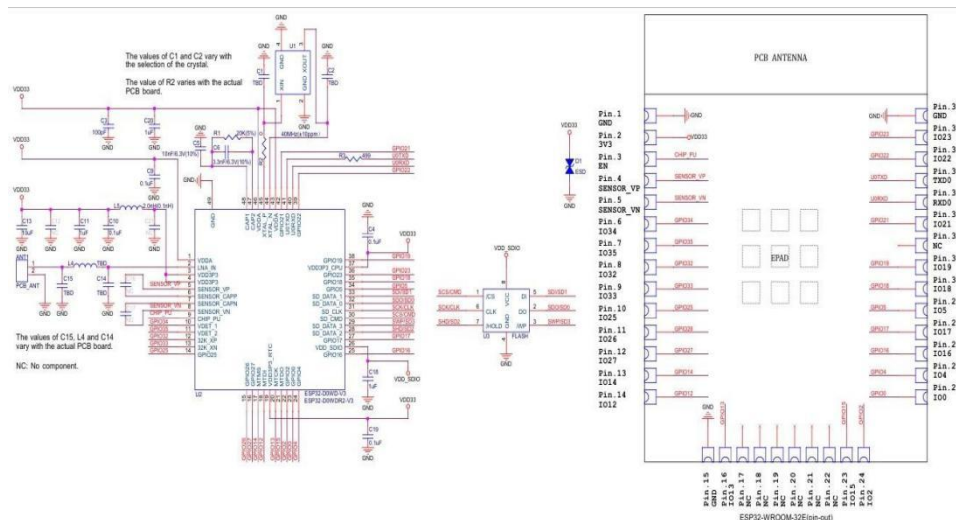


Hình 2.11 Sơ đồ chân của bo mạch điều khiển ESP32-DevkitC-32D

Thành phần trung tâm là module ESP32-WROOM-32E, tích hợp MCU Xtensa LX6 dual-core 240 MHz, 520 KB SRAM, 4 MB Flash, ăng-ten PCB cho Wi-Fi, khối khuếch đại công suất RF và Bluetooth BLE, cho phép truyền nhận dữ liệu không dây ổn định.

Nguồn nuôi có thể cấp qua Micro-USB 5 V hoặc VIN/5V, sau đó được ổn áp xuống 3.3V DC để an toàn cho toàn bộ mạch. Khi cần, có thể cấp trực tiếp 3V3 vào chân 3V3 của board.

Sơ đồ mạch nguyên lý bên trong ESP32-WROOM-32E



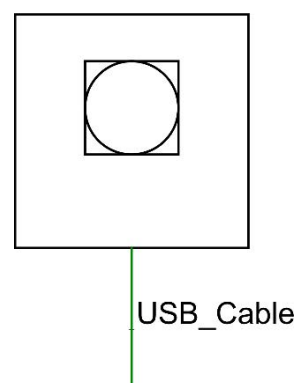
Hình 2.12 Sơ đồ nguyên lý của ESP32

2.8.3 Webcam 1080p^[6]

Webcam 1080p là thiết bị camera USB cung cấp hình ảnh độ phân giải Full HD và khả năng quay video mượt mà ở tốc độ 30 fps. Thiết bị phù hợp cho nhiều ứng dụng như hội nghị trực tuyến, giám sát, robot và các dự án AIoT, thị giác máy tính. Webcam kết nối qua giao tiếp USB phổ biến, dễ dàng tích hợp với máy tính, Raspberry Pi hoặc các hệ thống nhúng khác mà không cần cấu hình phức tạp. Với thiết kế nhỏ gọn, trọng lượng nhẹ và khả năng lắp đặt linh hoạt, webcam đáp ứng tốt yêu cầu triển khai trong nhiều môi trường khác nhau.



Hình 2.13 Webcam 1080p



Hình 2.14 Ký hiệu webcam 1080p

Thông số kỹ thuật của Webcam 1080p được thể hiện chi tiết trong bảng 2.3

Bảng 2.3 Thông số kỹ thuật của Webcam 1080p

STT	Tên thông số	Giá trị/ Đặc tính kỹ thuật	Ghi chú
1	Model	Webcam USB 1080p 30fps	Tương thích PC, laptop, Raspberry Pi
2	Cảm biến / Độ phân giải	2 megapixel	Cảm biến CMOS
3	Độ phân giải	Lên đến 1920×1080	Phụ thuộc phần mềm

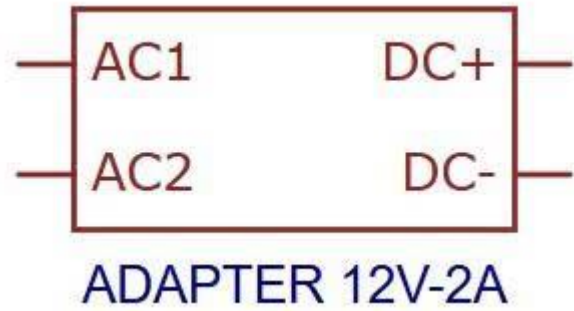
	ảnh tĩnh		và driver
4	Video 1080p	$1920 \times 1080 @ 30 \text{ fps}$	Full HD
5	Video 720p	$1280 \times 720 @ 30 \text{ fps}$	HD
6	Video VGA	$640 \times 480 @ 30 \text{ fps}$	Chuẩn VGA
7	Giao tiếp	USB 2.0	Chuẩn UVC, cắm là nhận
8	Trọng lượng mô - đun	$\approx 80 - 120 \text{ g}$	Tùy mẫu webcam
9	Trọng lượng cáp	$\approx 1.2 - 1.5 \text{ m}$	Cáp USB liền
10	Phụ kiện kèm theo	Webcam + cáp USB	Không cần phụ kiện bổ sung

2.8.4 Nguồn Adapter 12V-2A^[7]

Nguồn adapter 12V-2A là bộ chuyển đổi điện xoay chiều (AC) sang điện một chiều (DC) với điện áp ra 12V và dòng tối đa 2A (tương đương công suất khoảng 24W). Adapter được sử dụng phổ biến cho các thiết bị như camera giám sát, đèn LED, router, thiết bị IoT hoặc mạch điều khiển. Thiết bị hỗ trợ dải điện áp đầu vào rộng (100-240V AC, 50/60Hz), bảo vệ ngắn mạch, quá dòng và quá áp, đảm bảo hoạt động an toàn và ổn định.



Hình 2.15 Màn hình nguồn Adapter 12V-2A



Hình 2.16 Kí hiệu nguyên lý nguồn Adapter 12V-2A

Thông số kỹ thuật của màn hình nguồn Adapter 12V-2A được thể hiện chi tiết trong bảng 2.4

Bảng 2.4 Thông số kỹ thuật của nguồn Adapter 12V-2A

STT	Tên thông số	Giá trị / Đặc điểm kỹ thuật	Ghi chú
1	Điện áp ngõ vào	100–240VAC, 50/60Hz	Tương thích nhiều quốc gia
2	Điện áp ngõ ra	12VDC	Ổn định, phù hợp cho thiết bị IoT
3	Dòng điện ngõ ra	2A	Công suất tối đa 24W
4	Đầu cắm DC	5.5 × 2.5mm	Phổ biến cho camera, router, ESP32, v.v.
5	Chiều dài dây	1 mét	Cáp mềm, chống gãy tốt
6	Hiệu suất	≥ 85%	Giảm hao tổn năng lượng
7	Bảo vệ	Quá dòng, quá áp, ngắn mạch	An toàn cho thiết bị
8	Đèn báo nguồn	Không	Không có LED hiển thị

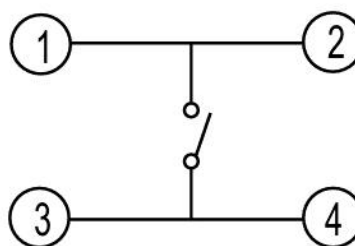
9	Kiểu đầu AC	2 chân dẹt (chuẩn Mỹ)	Phổ biến, dễ cắm
10	Ứng dụng	Đa năng	Camera, router, mạch Arduino, ESP32, v.v.

2.8.5 Nút nhấn 6x6mm^[9]

Nút nhấn 6x6mm cao 6mm 4 chân xuyên lỗ là loại công tắc nhấn tạm thời (tact switch) được sử dụng phổ biến trong các mạch điện tử, Arduino, Raspberry Pi, và các mạch nhúng. Khi nhấn, hai cặp chân của nút sẽ được nối lại với nhau để tạo tín hiệu ON, khi thả ra nút trở lại trạng thái OFF nhờ vào lò xo đàn hồi bên trong. Sản phẩm có kích thước nhỏ gọn, dễ dàng hàn lên PCB hoặc cắm thử trên breadboard.



Hình 2.17 Nút nhấn 6x6mm



Hình 2.18 Kí hiệu nguyên lý nút nhấn 6x6mm

Thông số kỹ thuật của nút nhấn 6x6mm được thể hiện chi tiết trong bảng 2.5

Bảng 2.5 Thông số kỹ thuật của nút nhấn 6x6mm

STT	Tên thông số	Giá trị / Đặc điểm kỹ thuật	Ghi chú
1	Kích thước thân	6mm x 6mm	Hình vuông tiêu chuẩn
2	Chiều cao tổng thể	6mm	Tính từ đáy chân tới đỉnh nút

3	Số chân	4 chân	Chân xuyên lỗ (DIP type)
4	Khoảng cách chân	4.5mm	Phù hợp gắn trên PCB tiêu chuẩn
5	Điện áp hoạt động	DC 12V (tối đa)	Điện áp chịu tải
6	Dòng điện định mức	50mA	Dòng tối đa qua tiếp điểm
7	Tuổi thọ hoạt động	≥ 100.000 lần nhấn	Độ bền cơ học cao
8	Kiểu hoạt động	Momentary (nhấn giữ – nhả ngắt)	Tác động tức thời
9	Chất liệu vỏ	Nhựa chịu nhiệt (Nylon/ABS)	Chịu nhiệt, bền bỉ
10	Ứng dụng	Arduino, mạch reset, thiết bị IoT, module điều khiển	Thông dụng trong dự án học tập và DIY

Đoạn code kiểm tra hoạt động của nút nhấn

```
#define BUTTON_PIN 21

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);
```

```

Serial.println(buttonState);
}

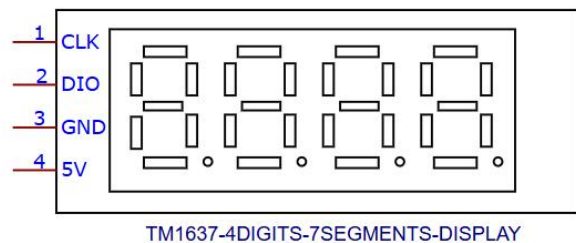
```

2.8.6 Module hiển thị 4 led 7 đoạn TM1637^[10]

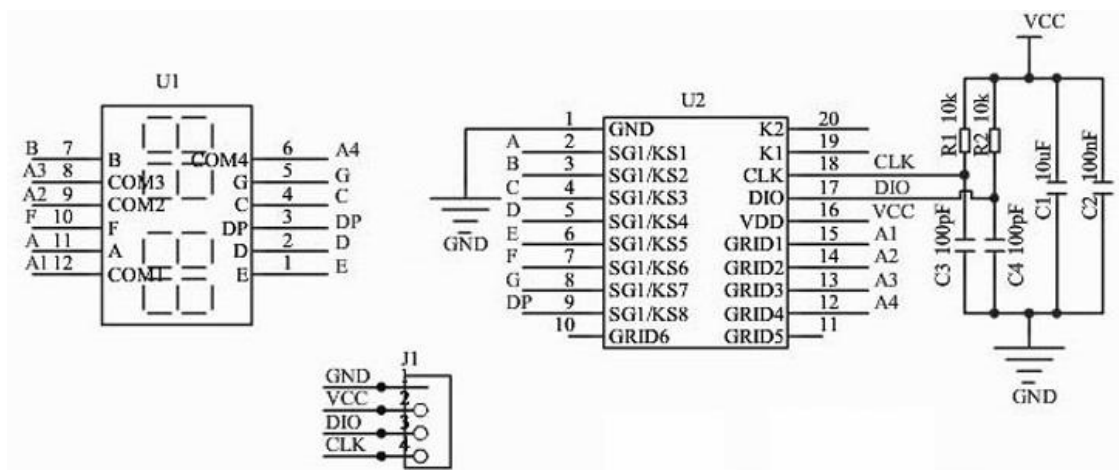
Module hiển thị 4 led 7 đoạn TM1637 là mạch hiển thị số có 4 led 7 đoạn và 2 dấu hai chấm (thường để hiển thị thời gian), sử dụng IC TM1637 cho phép điều khiển chỉ bằng 2 chân tín hiệu (CLK và DIO) của vi điều khiển. Điện áp hoạt động của module từ 3.3V đến 5V, rất phổ biến trong các dự án vì nhỏ gọn và tiện dụng.



Hình 2.19 Module TM 1637



Hình 2.20 Kí hiệu nguyên lý của TM1637



Hình 2.21 Sơ đồ nguyên lý của module hiển thị 4 led 7 đoạn TM1637

Thông số kỹ thuật của module hiển thị 4 led 7 đoạn được thể hiện chi tiết trong bảng 2.6

Bảng 2.6 Thông số kỹ thuật của module hiển thị TM1637

STT	Tên thông số	Giá trị / Đặc điểm kỹ thuật	Ghi chú
1	Model	TM1637 4-Digit Display 0.36 Inch Module	Màn hình hiển thị 4 led 7 đoạn
2	Điện áp tiêu thụ	3.3 – 5V DC	Tương thích với ESP32, Arduino
3	Dòng điện tiêu thụ	30 – 80 mA	Tùy độ sáng (brightness level)
4	IC Driver	TM1637	Điều khiển LED 7 đoạn và quét từng digit
5	Chân giao tiếp	CLK, DIO	Giao tiếp kiểu 2-wire (giống I2C nhưng không phải I2C)

Đoạn code kiểm tra hoạt động của module TM1637

```
#include <TM1637Display.h>

#define CLK  22 // The ESP32 pin GPIO22 connected to CLK
#define DIO  23 // The ESP32 pin GPIO23 connected to DIO

// Create a TM1637Display object
TM1637Display display = TM1637Display(CLK, DIO);

// Display numbers
display.showNumberDec(-12);           // displayed _-12
display.showNumberDec(-999);          // displayed -999
display.showNumberDec(42);             // displayed __42
display.showNumberDec(42, false);      // displayed __42
display.showNumberDec(42, false, 2, 0); // displayed 42__ => display 2 digit
```

```
at position 0
display.showNumberDec(42, true);          // displayed 0042 => zero padding
display.showNumberDec(14, false, 2, 1);   // displayed _14_
display.showNumberDec(-5, false, 3, 0);   // displayed _-5_
display.showNumberDec(1234);              // displayed 1234
```

2.9 CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU

2.9.1 Chuẩn giao tiếp WiFi

Wi-Fi (IEEE 802.11) là chuẩn giao tiếp không dây được sử dụng phổ biến để kết nối ESP32 và Raspberry Pi trong mạng cục bộ. Chuẩn này cho phép truyền dữ liệu tốc độ cao, dễ triển khai và phù hợp cho hệ thống giám sát giao thông.

Ưu điểm:

- Băng thông cao, hỗ trợ truyền hình ảnh và dữ liệu thời gian thực.
- Dễ dàng kết nối với mạng Wi-Fi sẵn có.
- Hỗ trợ nhiều giao thức tầng cao như HTTP, MQTT, WebSocket.

Nhược điểm:

- Tiêu thụ năng lượng cao hơn BLE hoặc LoRa.
- Phụ thuộc vào vùng phủ sóng của router.

Trong hệ thống giám sát giao thông, ESP32 đóng vai trò thu thập dữ liệu cảm biến hoặc điều khiển đèn giao thông, trong khi Raspberry Pi hoạt động như gateway xử lý ảnh và truyền thông tin lên server. Hai thiết bị này kết nối với nhau thông qua Wi-Fi 2.4 GHz.

2.9.2 Giao thức truyền dữ liệu

Để truyền dữ liệu giữa các thiết bị IoT, có nhiều giao thức được sử dụng tùy theo nhu cầu về tốc độ, độ tin cậy và băng thông. Dưới đây là một số giao thức phổ biến được áp dụng trong hệ thống giám sát giao thông.

2.9.2.1 MQTT (Message Queuing Telemetry Transport)

Là giao thức nhẹ, sử dụng mô hình publish/subscribe, phù hợp cho các thiết bị có tài nguyên hạn chế như ESP32. Dữ liệu được gửi tới một broker trung gian (ví dụ Mosquitto) rồi phân phối đến các client đăng ký.

Ưu điểm:

- Nhẹ, tiêu thụ băng thông thấp.
- Hỗ trợ QoS đảm bảo độ tin cậy.

Nhược điểm: Không phù hợp để truyền tệp lớn như ảnh hoặc video.

2.9.2.2 HTTP / HTTPS

Là giao thức phổ biến trên nền TCP/IP, hoạt động theo mô hình request-response. HTTP được sử dụng để tải lên dữ liệu, ảnh hoặc yêu cầu điều khiển từ ESP32 đến Raspberry Pi hoặc server.

Ưu điểm:

- Dễ triển khai, tương thích cao.
- Hỗ trợ bảo mật với HTTPS.

Nhược điểm: Tốc độ chậm hơn MQTT cho dữ liệu nhỏ.

2.9.2.3 Giao thức HTTP RESTful và Framework FastAPI^[12]

HTTP RESTful (Representational State Transfer) là tiêu chuẩn kiến trúc mạng sử dụng các phương thức HTTP (GET, POST, PUT, DELETE) để các hệ thống phân tán giao tiếp độc lập qua internet. Đặc trưng cốt lõi của RESTful là tính phi trạng thái (stateless) – mỗi yêu cầu (request) đều chứa đủ thông tin để máy chủ xử lý độc lập, giúp giảm tải bộ nhớ và dễ dàng mở rộng hệ thống.

Dựa trên chuẩn giao tiếp này, FastAPI là một web framework hiện đại bằng Python chuyên dùng để xây dựng các API hiệu năng cao. Hoạt động trên nền tảng ASGI (Asynchronous Server Gateway Interface), FastAPI nổi bật với khả năng xử lý bất đồng bộ và tự động xác thực gói tin thông qua kiểu dữ liệu (Type Hints). Những ưu điểm này giúp tối ưu hóa quá trình truyền tải luồng dữ liệu nặng (như hình ảnh) và kết nối mạng vi dịch vụ giữa thiết bị biên với máy chủ một cách an toàn, mượt mà và không gây nghẽn hệ thống.

Ưu điểm:

- Hiệu năng thực thi cực cao, hỗ trợ xử lý bất đồng bộ (async/await) mượt mà.
- Tự động khởi tạo tài liệu API (Swagger UI) trực quan, tích hợp sẵn kiểm tra kiểu dữ liệu.

Nhược điểm: Tiêu tốn nhiều tài nguyên băng thông hơn MQTT đối với các luồng dữ liệu nhỏ, liên tục.

2.10 TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU FIREBASE

Firebase là một nền tảng phát triển ứng dụng di động và web do Google cung cấp, hỗ trợ các nhà phát triển xây dựng, tối ưu hóa và mở rộng ứng dụng một cách dễ dàng. Nền tảng này cung cấp nhiều dịch vụ và công cụ backend-as-a-service (BaaS), bao gồm cơ sở dữ liệu thời gian thực, xác thực người dùng, lưu trữ đám mây, phân tích và nhiều tính năng khác, giúp giảm bớt gánh nặng quản lý cơ sở hạ tầng cho các nhà phát triển.



Hình 2.22 Cơ sở dữ liệu Firebase

Firebase không chỉ giúp tiết kiệm thời gian triển khai ứng dụng mà còn hỗ trợ mở rộng quy mô một cách dễ dàng. Nền tảng này cung cấp dịch vụ cơ sở dữ liệu đám mây, được vận hành trên hệ thống máy chủ mạnh mẽ của Google, giúp việc lập trình trở nên đơn giản hơn nhờ tự động hóa nhiều thao tác liên quan đến cơ sở dữ liệu.

Firebase còn cung cấp các giao diện lập trình ứng dụng (API) thân thiện, giúp thu hút nhiều người dùng hơn và tăng khả năng sinh lời cho các nhà phát triển. Đặc biệt, Firebase đảm bảo tính đa năng và bảo mật cao, hỗ trợ cả hai hệ điều hành phổ biến là Android và iOS.

Chính vì những lý do này, Firebase trở thành lựa chọn hàng đầu cho nhiều lập trình viên trên toàn thế giới trong việc phát triển các ứng dụng dành cho hàng triệu người dùng.

2.11 TỔNG QUAN VỀ CÁC CÔNG CỤ XÂY DỰNG VÀ LẬP TRÌNH HỆ THỐNG

2.11.1 EasyEDA

EasyEDA là một nền tảng thiết kế mạch điện tử trực tuyến hoạt động dựa trên công nghệ điện toán đám mây. Công cụ này cho phép người dùng thao tác trực tiếp thông qua trình duyệt web mà không yêu cầu cài đặt phần mềm cục bộ. Nền tảng này được phát triển nhằm cung cấp một môi trường làm việc linh hoạt và tiện lợi, phục vụ hiệu quả cho quá trình học tập, nghiên cứu của sinh viên cũng như hỗ trợ các kỹ sư chuyên nghiệp trong quy trình phát triển phần cứng.

Trong giai đoạn thiết kế, EasyEDA hỗ trợ xây dựng sơ đồ nguyên lý một cách trực quan và chính xác nhờ vào hệ thống thư viện linh kiện đồ sộ và liên tục được cập nhật. Bên cạnh đó, phần mềm còn tích hợp công cụ mô phỏng trước khi thi công thực tế. Chức năng này cho phép người thiết kế kiểm thử và phân tích chi tiết hoạt động của các mạch tín hiệu tương tự, tín hiệu số cũng như tín hiệu hỗn hợp thông qua các mô hình chuẩn được tích hợp sẵn.



Hình 2.23 Phần mềm EasyEda

Đối với công đoạn thiết kế bo mạch in, nền tảng này cung cấp các giải pháp định tuyến và phân bổ linh kiện tối ưu, đáp ứng tốt các yêu cầu kỹ thuật khắt khe của những dự án mạch in đa lớp phức tạp. Với giao diện làm việc trực quan kết hợp cùng khả năng lưu trữ trực tuyến đồng bộ, EasyEDA không chỉ là một giải pháp hữu hiệu giúp rút ngắn chu trình thiết kế mà còn góp phần nâng cao hiệu suất làm việc toàn diện cho người sử dụng.

2.11.2 Antigravity IDE

Antigravity IDE là một nền tảng phát triển phần mềm dựa trên tác nhân (agentic development platform) thế hệ mới do Google phát triển. Mặc dù được xây dựng dựa trên kiến trúc cốt lõi của Visual Studio Code, Antigravity đánh dấu sự chuyển dịch mạnh mẽ sang mô hình "ưu tiên tác nhân" (agent-first). Nhờ khả năng hoạt động như một kỹ sư phần mềm tự chủ có tư duy đa bước, nền tảng này hiện là một trong những công cụ hỗ trợ lập trình tiên tiến nhất, định hình lại quy trình phát triển phần mềm tự động hóa.



Hình 2.24 Phần mềm Antigravity IDE

Nền tảng này cung cấp một hệ sinh thái toàn diện được vận hành bởi mô hình AI đa phương thức Gemini 3 nhằm tối ưu hóa triệt để tiến độ hoàn thiện dự án. Nổi bật nhất là hệ thống Trình quản lý tác nhân (Agent Manager), cho phép lập

trình viên giao phó các luồng công việc phức tạp để AI tự động lên kế hoạch, viết mã và kiểm thử đồng thời trên trình chỉnh sửa, thiết bị đầu cuối (terminal) và trình duyệt web. Thay vì chỉ gợi ý cú pháp mã nguồn đơn thuần, hệ thống tự động sinh ra các "Tạo tác" (Artifacts) trực quan như tài liệu kế hoạch hoặc ảnh chụp màn hình thực thi, giúp dễ dàng đánh giá và kiểm soát tác vụ. Đồng thời, Antigravity vẫn hỗ trợ quản lý hệ thống phân phiên bản thông qua Git và cung cấp bộ công cụ gỡ lỗi (Debugging) toàn diện, tương thích sâu với các ngôn ngữ cốt lõi trong lĩnh vực trí tuệ nhân tạo như Python.

2.11.3 PlatformIO

PlatformIO là một hệ sinh thái mã nguồn mở thế hệ mới phục vụ chuyên sâu cho việc phát triển các ứng dụng nhúng và Internet vạn vật (IoT). Khác với các môi trường lập trình độc lập truyền thống, PlatformIO thường được tích hợp dưới dạng tiện ích mở rộng (extension) trên các trình biên tập mã nguồn hiện đại như Visual Studio Code, mang đến một không gian làm việc chuyên nghiệp, linh hoạt và đáp ứng tốt các dự án có độ phức tạp cao.

Nền tảng này hỗ trợ đa hệ điều hành, có thể cài đặt và sử dụng mượt mà trên Windows, macOS và Linux. Được trang bị một hệ thống quản lý xây dựng (build system) độc lập cùng các công cụ phân tích mã tĩnh (static code analyzer), PlatformIO cung cấp khả năng tự động hoàn thiện mã nguồn, gợi ý cú pháp và kiểm tra lỗi thông minh, qua đó giúp quá trình lập trình trở nên chặt chẽ và tối ưu hóa hiệu suất quản lý dự án.



Hình 2.25 Phần mềm PlatformIO

Môi trường làm việc của PlatformIO nổi bật với cơ chế quản lý tự động. Người dùng chủ yếu viết chương trình bằng ngôn ngữ C hoặc C++ (dựa trên các framework như Arduino cốt lõi hoặc ESP-IDF). Sau đó, hệ thống sẽ tự động tải về các chuỗi công cụ tương ứng, biên dịch mã nguồn thành tệp thực thi (như .bin hoặc .hex) và nạp trực tiếp lên bo mạch thông qua kết nối USB hoặc cập nhật không dây (OTA). Quá trình này diễn ra hoàn toàn tự động mà không cần người dùng phải cấu hình thủ công phức tạp.

2.11.4 Thư viện OpenCV, Supervision và NCNN-Python^[13]

Hệ thống sử dụng ba thư viện Python cốt lõi để tối ưu hóa luồng xử lý thị giác máy tính tại thiết bị biên:

- OpenCV: Đảm nhiệm xử lý ảnh mức thấp, bao gồm đọc luồng video, cắt vùng ảnh vi phạm và tính toán ma trận biến đổi phối cảnh (Homography) để tạo dữ liệu giả lập.
- Supervision: Cung cấp bộ công cụ bậc cao để thiết lập vùng giám sát (PolygonZone, LineZone), quản lý vết đối tượng cho ByteTrack và vẽ đồ họa (bounding box) với hiệu năng cao.
- NCNN-Python: Framework suy luận nơ-ron tối ưu cho thiết bị IoT. Thư viện này hỗ trợ nạp mô hình lượng tử hóa trực tiếp lên CPU vi kiến trúc ARM của Raspberry Pi, giúp tăng tốc độ xử lý vượt trội so với PyTorch gốc.

2.12 CƠ SỞ LÝ THUYẾT VỀ CƠ SỞ DỮ LIỆU SQLITE VÀ CHẾ ĐỘ GHI TRƯỚC NHẬT KÝ (WAL)

Cơ sở dữ liệu đóng vai trò lưu trữ toàn bộ dữ liệu vi phạm phạt nguội tại máy chủ phục vụ tra cứu dài hạn. Hệ thống lựa chọn SQLite làm hệ quản trị cơ sở dữ liệu nhờ tính tinh gọn và hiệu năng cao trong môi trường Edge/Server phân tán nhỏ:

- Cơ sở dữ liệu SQLite: Là hệ quản trị cơ sở dữ liệu quan hệ nhúng (embedded database), hoạt động không cần tiến trình server độc lập (serverless). Toàn bộ dữ liệu cấu trúc bảng, chỉ mục (index) và bản ghi được lưu trữ trong một tệp

tin duy nhất trên đĩa cứng. SQLite tuân thủ đầy đủ các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo an toàn giao dịch dữ liệu tuyệt đối.

- Chế độ ghi trước nhật ký WAL (Write-Ahead Logging): Mặc định, SQLite sử dụng chế độ rollback journal để quản lý giao dịch. Tuy nhiên, chế độ này khóa toàn bộ cơ sở dữ liệu khi có tiến trình thực hiện ghi, gây lỗi blocking đối với các ứng dụng có tính đồng thời. Chế độ WAL được kích hoạt để khắc phục điểm yếu này:

- *Cơ chế hoạt động*: Khi có dữ liệu vi phạm mới cần chèn (FastAPI Server ghi), thay vì ghi trực tiếp vào tệp cơ sở dữ liệu chính (.db), các thay đổi được ghi tuần tự vào tệp nhật ký WAL phụ (.db-wal).

- *Đọc và ghi song song*: Các tiến trình đọc dữ liệu (Streamlit Dashboard truy vấn dữ liệu) vẫn có thể đọc trực tiếp từ tệp cơ sở dữ liệu chính và tệp WAL mà không bị khóa bởi tiến trình ghi. Nhờ đó, SQLite WAL cho phép một tiến trình ghi và nhiều tiến trình đọc hoạt động đồng thời thực sự.

- *Tối ưu hóa I/O*: Do dữ liệu được ghi tuần tự vào tệp WAL, SQLite giảm thiểu tối đa các thao tác di chuyển đầu đọc ngẫu nhiên trên đĩa, tăng tốc độ ghi phát nguội lên gấp nhiều lần. Định kỳ, SQLite sẽ tự động thực hiện quá trình đồng bộ (Checkpoint) để chuyển các trang dữ liệu đã ghi từ tệp WAL về tệp cơ sở dữ liệu chính và giải phóng tệp nhật ký.

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 YÊU CẦU VÀ SƠ ĐỒ KHỎI HỆ THỐNG

3.1.1 Yêu cầu của hệ thống

Hệ thống giám sát lưu lượng phương tiện và phát hiện đối tượng vi phạm giao thông được thiết kế nhằm mục đích giám sát, điều phối mật độ giao thông tại ngã tư và tự động phát hiện hành vi vượt đèn đỏ để phục vụ phạt nguội. Các yêu cầu chức năng chính bao gồm:

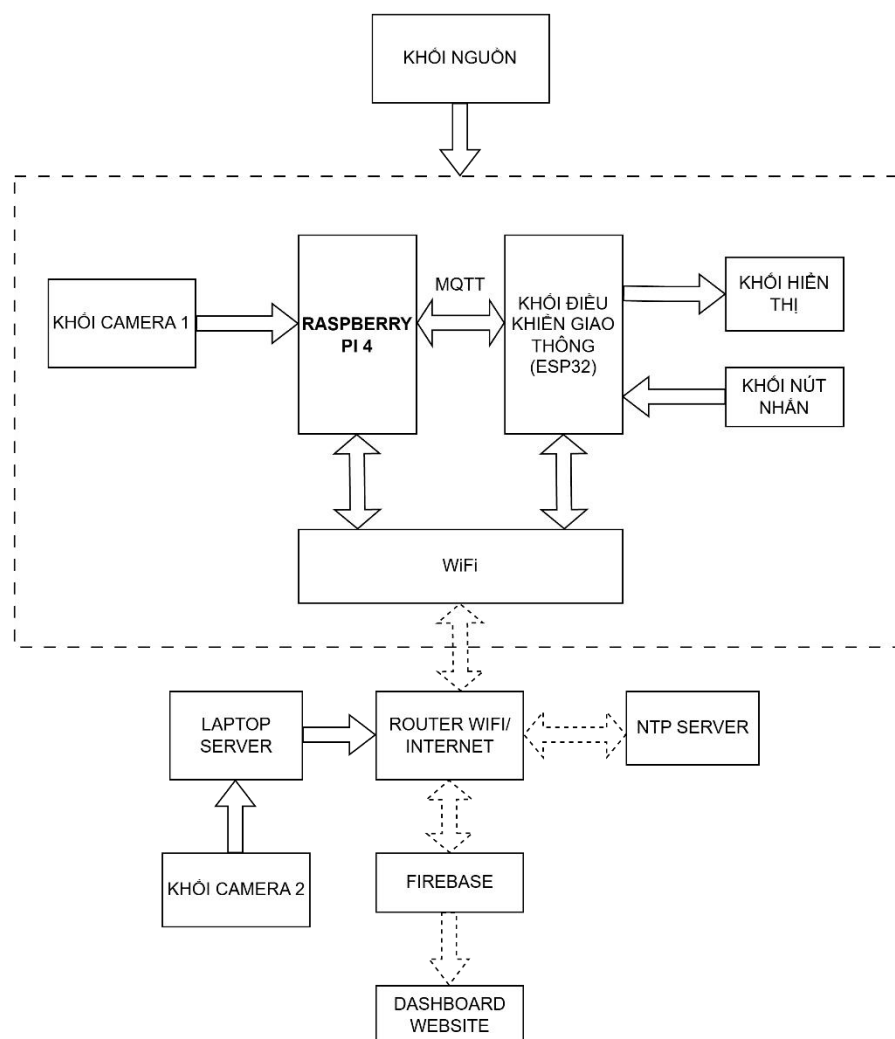
- **Giám sát lưu lượng phương tiện:** Hệ thống liên tục thu thập luồng hình ảnh giao lộ từ trên cao thông qua camera đơn (trong điều kiện thực nghiệm sử dụng luồng video mô phỏng độ nét cao gồm video chuẩn giao lộ `vehicle_counting.mp4` và video sa bàn toy thực tế `vehicle_toy.mp4`). Thiết bị xử lý tại biên thực hiện nhận dạng, phân loại phương tiện (xe máy, ô tô con, xe tải, xe buýt) và đếm lưu lượng tích lũy theo thời gian thực tại các làn đường của giao lộ.
- **Điều phối giao thông thích ứng:** Dữ liệu mật độ xe từ thiết bị biên được truyền tải tức thời qua giao thức truyền thông MQTT tới bộ điều khiển đèn giao thông tại giao lộ để tự động phân bổ pha đèn xanh tối ưu theo tỉ lệ mật độ xe thực tế.
- **Phát hiện hành vi vi phạm (Vượt đèn đỏ):** Hệ thống thiết lập vạch dừng ảo tương ứng trên làn đường. Khi pha đèn tín hiệu chuyển sang màu đỏ, bất kỳ phương tiện nào có hành vi cắt ngang qua vạch dừng ảo này sẽ bị hệ thống tự động ghi nhận sự kiện vi phạm, cắt ảnh bằng chứng xe vi phạm, áp dụng phép biến đổi phối cảnh dán biển số giả lập và truyền bất đồng bộ về máy chủ trung tâm.
- **Nhận dạng biển số và lưu trữ dữ liệu:** Máy chủ trung tâm sau khi tiếp nhận hình ảnh vi phạm sẽ kích hoạt mô hình OCR nhận diện biển số xe (định vị vùng chứa biển số và đọc ký tự), sau đó tự động lưu trữ siêu dữ liệu (thời gian, loại xe,

chuỗi ký tự biên số, ảnh bằng chứng) vào cơ sở dữ liệu SQLite WAL và cập nhật lên Web Dashboard Streamlit.

- **Đồng bộ hóa đám mây:** Bộ mạch điều khiển giao lộ liên tục đẩy các thông tin trạng thái hoạt động trực tiếp (chế độ tự động/thủ công/ngủ, thời gian đếm ngược của hai hướng chính) lên Firebase Realtime Database phục vụ giám sát hai chiều từ xa.

3.1.2 Sơ đồ khối và chức năng của mỗi khối

Sơ đồ khối của hệ thống được mô tả như hình 3.1



Hình 3.1 Sơ đồ khối của hệ thống

Chức năng cơ bản của các khối trong hệ thống:

- Khối thu nhận hình ảnh (Webcam / Video Simulation):

Thực hiện thu nhận dữ liệu hình ảnh tại khu vực giao lộ. Trong mô hình thực nghiệm, nguồn dữ liệu được cung cấp từ các tệp video mô phỏng giao thông nhằm tạo luồng dữ liệu đầu vào cho hệ thống xử lý.

- Khối xử lý biên (Edge Agent – Raspberry Pi 4):

Tiếp nhận dữ liệu hình ảnh từ khối thu nhận hình ảnh, thực hiện phát hiện và theo dõi phương tiện, phân tích mật độ giao thông, xử lý các chức năng điều khiển đèn tín hiệu và phát hiện vi phạm. Ngoài ra, khối này thực hiện phép biến đổi phối cảnh phục vụ quá trình nhận dạng biển số xe và truyền dữ liệu đến các thành phần khác của hệ thống.

- Khối điều khiển đèn giao thông (Traffic Light Controller – ESP32):

Nhận lệnh điều khiển từ Raspberry Pi 4 thông qua giao thức MQTT và điều khiển trạng thái hoạt động của các cụm đèn tín hiệu giao thông. Đồng thời, khối này cập nhật dữ liệu trạng thái và thời gian đếm ngược lên Firebase Realtime Database.

- Khối nút nhấn:

Cho phép thay đổi chế độ vận hành của hệ thống, bao gồm chế độ tự động, chế độ thủ công, chuyển pha đèn và kích hoạt chế độ Sleep Mode.

- Khối hiển thị (TM1637):

Hiển thị thời gian đếm ngược tương ứng với trạng thái hoạt động của hệ thống đèn giao thông tại các hướng của giao lộ.

- Khối nguồn (LM7805 và Adapter):

Cung cấp nguồn điện cho các thiết bị trong hệ thống. Module LM7805 thực hiện ổn áp từ nguồn 12V DC xuống 5V DC để cấp nguồn cho ESP32 và các linh kiện ngoại vi, trong khi Raspberry Pi 4 sử dụng bộ nguồn độc lập 5.1V–3A.

- Khối WiFi và MQTT Broker (Mosquitto):

Đảm nhiệm chức năng truyền thông giữa Raspberry Pi 4 và ESP32 thông qua mạng WiFi nội bộ. MQTT Broker Mosquitto được cài đặt trên Raspberry Pi 4 để quản lý việc trao đổi bản tin giữa các thiết bị.

- Firebase Database:

Lưu trữ và đồng bộ dữ liệu trạng thái của hệ thống theo thời gian thực, phục vụ cho việc giám sát và trao đổi dữ liệu giữa các thành phần trong hệ thống.

- Khối máy chủ trung tâm (FastAPI Server):

Tiếp nhận dữ liệu hình ảnh vi phạm từ Raspberry Pi 4, thực hiện nhận dạng biển số xe bằng mô hình YOLOv11n (License Plates) kết hợp CRNN, sau đó lưu trữ kết quả nhận dạng vào cơ sở dữ liệu SQLite.

- Khối Dashboard giám sát (Streamlit Web App):

Khai thác dữ liệu từ cơ sở dữ liệu SQLite để hiển thị thông tin phương tiện vi phạm, bao gồm hình ảnh vi phạm, biển số xe và các thông tin liên quan. Đồng thời, khối này nhận dữ liệu trạng thái giao lộ từ Firebase và hỗ trợ gửi lệnh điều khiển đến hệ thống thông qua giao thức MQTT.

3.1.3 Mô tả hoạt động của hệ thống

Hệ thống hoạt động theo luồng xử lý như sau:

- Thu nhận và phân tích hình ảnh biên: Luồng video được đưa vào Raspberry Pi 4. Mô hình YOLOv11n (NCNN lượng tử hóa) định vị các phương tiện, trả tọa độ cho thuật toán ByteTrack nội suy quỹ đạo và gán mã định danh duy nhất (Tracking ID). Dữ liệu số lượng xe nằm trong 4 vùng đa giác (ROI) đại diện cho 4 hướng rẽ tại ngã tư được tổng hợp liên tục.

- Tự động điều phối thích ứng: Định kỳ, Raspberry Pi 4 gửi dữ liệu mật độ xe của 4 vùng qua MQTT xuống ESP32. ESP32 thực hiện hàm tính toán phân bổ chu kỳ đèn thích ứng dựa trên tỷ lệ xe của hai hướng chính: hướng nào đông xe hơn sẽ tự động tăng thời gian đèn xanh (trong khoảng giới hạn từ 5s đến 30s) và giảm thời gian đèn xanh của hướng trống xe. Đồng thời, ESP32 đồng bộ trạng thái đếm ngược lên Firebase Realtime Database.

- Phát hiện và xử lý vi phạm: ESP32 xuất bản trạng thái màu đèn lên MQTT. Nếu hướng làn đang có tín hiệu đèn Đỏ, Raspberry Pi 4 sẽ kích hoạt giám sát lỗi đèn vạch ảo. Khi một xe có Tracking ID di chuyển cắt qua ranh giới vạch cấm, hệ thống tự động cắt vùng ảnh xe, thực hiện biến đổi Homography dán biển số giả

lập góc nghiêng đuôi xe rồi đẩy bắt đồng bộ lên FastAPI Server để tránh treo luồng phân tích.

- OCR biển số và lưu trữ: Máy chủ FastAPI tiếp nhận ảnh vi phạm, chạy mô hình YOLOv11 định vị chính xác khung biển số và chuyển sang mô hình CRNN-CTC giải mã chuỗi ký tự. Siêu dữ liệu vi phạm được ghi trực tiếp vào cơ sở dữ liệu SQLite WAL (traffic.db). Streamlit Web Dashboard tự động làm mới để hiển thị bảng chứng phạt nguội phục vụ công tác giám sát của cơ quan chức năng.

3.2 THIẾT KẾ PHẦN CỨNG

3.2.1 Khối camera 1

Khối camera trong hệ thống có chức năng ghi lại hình ảnh tại khu vực giao lộ theo thời gian thực và gửi dữ liệu này đến khối xử lý trung tâm xử lý. Thông qua các khung hình thu được, Raspberry Pi 4 sẽ phân tích và dự đoán mật độ phương tiện giao thông trên từng hướng.

Để ghi nhận hình ảnh phương tiện giao thông, một số camera có thể sử dụng, bao gồm: Raspberry Pi Camera Module 3, Raspberry Pi High Quality (HQ) Camera và Camera Pi 5 MP. Đặc điểm của các camera trên được mô tả như bảng 3.

Bảng 3.1 Đặc điểm của các camera

STT	TÊN GỌI	ĐẶC ĐIỂM
1	Raspberry Pi Camera Module 3	- Sử dụng cảm biến Sony IMX708, độ phân giải 12 MP - Hỗ trợ lấy nét tự động (autofocus), có phiên bản NoIR ghi hình ban đêm, chất lượng ảnh cao, độ trễ thấp - Ảnh rõ nét, hoạt động ổn định, thích hợp cho ứng dụng giám sát
2	Webcam 1080p	- Sử dụng cảm biến hình ảnh độ phân giải Full HD 1080p (1920 × 1080) - Góc nhìn cố định, phù hợp cho hội họp trực

		tuyến, giám sát và các ứng dụng AI cơ bản - Ghi hình rõ nét trong phạm vi gần và trung bình, đáp ứng nhu cầu nhận diện khuôn mặt hoặc theo dõi đối tượng trong nhà
3	Camera Pi 5 MP	- Sử dụng cảm biến OV5647, độ phân giải 5 MP - Mô-đun phổ biến, kích thước nhỏ gọn, giá thành thấp, dễ lắp đặt - Hình ảnh đủ tốt cho ứng dụng xử lý ảnh hoặc thị giác máy tính

Webcam 1080p là lựa chọn phổ biến cho các dự án sử dụng Raspberry Pi. Thiết bị hỗ trợ giao tiếp qua cổng CSI chuyên dụng, giúp truyền dữ liệu nhanh và ổn định, có mức độ tích hợp cao. Bên cạnh đó, việc tiêu thụ điện năng thấp và khả năng tương thích cao nên Camera Pi 5MP rất phù hợp cho việc giám sát mật độ giao thông bất kể ngày và đêm.

Module camera được kết nối với Raspberry Pi 4 qua cổng USB 2.0.

3.2.2 Khôi nút nhấn

Khôi nút nhấn được sử dụng trong hệ thống là nút nhấn 2 chân. Khi được nhấn, nó tạo ra một kết nối cho phép dòng điện chạy qua mạch. Khi thả ra, kết nối bị ngắt không cho dòng điện chạy qua mạch.

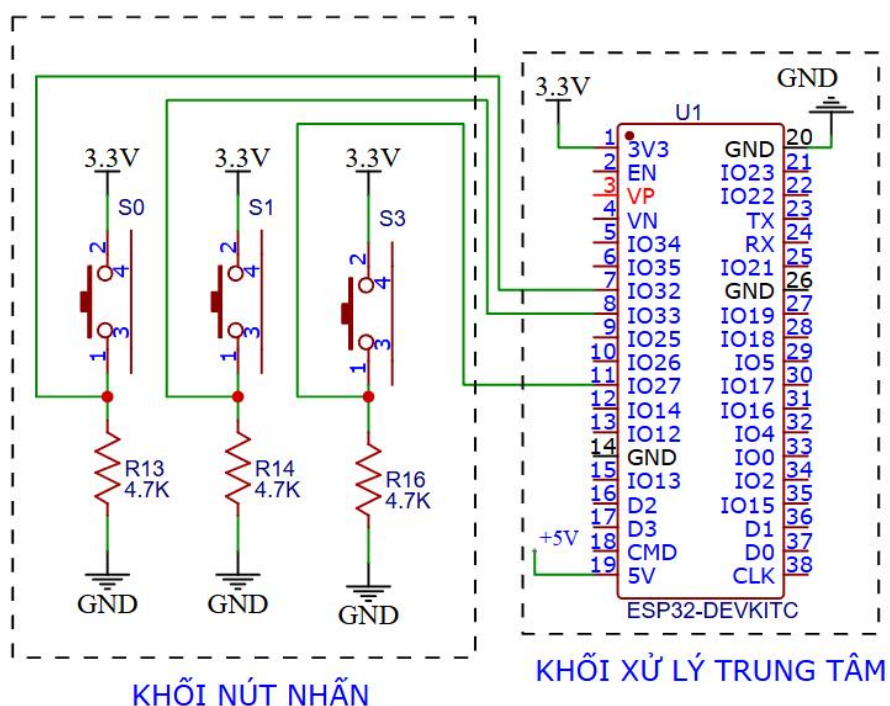
Hiện nay, có nhiều loại nút nhấn được sử dụng trong các mạch điện tử, trong đó ba loại phổ biến là nút nhấn 3x4mm cao 2.5mm 2 chân SMD, nút nhấn 6x6mm cao 6mm 4 chân xuyên lỗ và nút nhấn 3x6mm cao 2.5mm 2 chân SMD.

Phương án 1: Nút nhấn 3x4mm SMD có kích thước rất nhỏ gọn, được thiết kế dạng gắn dán trực tiếp trên bề mặt mạch in, giúp tiết kiệm diện tích và phù hợp với những thiết bị điện tử mini như remote, module cảm biến hoặc tay cầm điều khiển.

Phương án 2: Nút nhấn 6x6mm xuyên lỗ có kích thước lớn hơn, cấu tạo chắc chắn với 4 chân cắm qua lỗ mạch, mang lại độ bền cơ học cao và cảm giác bấm tốt, thường được sử dụng trong các mạch thử nghiệm, Arduino hoặc các thiết bị điện tử dân dụng.

Phương án 3: Nút nhấn 3x6mm SMD có dạng gắn dán như loại 3x4mm nhưng kích thước dài hơn, tạo cảm giác bấm thoải mái hơn và vẫn đảm bảo thiết kế mỏng gọn, thích hợp cho các sản phẩm công nghiệp hoặc bo mạch phẳng.

Dựa vào các đặc điểm trên, nhóm thực hiện đề tài chọn nút nhấn 6x6mm cao 6mm 4 chân xuyên lỗ. Nút nhấn phù hợp với nhiệm vụ gửi tín hiệu cho khối xử lý trung tâm thay đổi chế độ hoạt động của đèn giao thông.



Hình 3.2 Mô tả kết nối của khối nút nhấn với khối xử lý trung tâm

Khối nút nhấn trong hệ thống giám sát được sử dụng để thay đổi các chế độ hoạt động của đèn giao thông. Các chế độ hoạt động gồm có S0, S1, S2.

S0: kết nối với chân GPIO 32 của khối xử lý trung tâm, có chức năng chuyển đổi chế độ tự động/thủ công của hệ thống, cho phép người dùng dễ dàng lựa chọn cách điều khiển hệ thống đèn giao thông.

S1: kết nối với chân GPIO 33 của khối xử lý trung tâm, đảm nhận việc chuyển sang đèn giao thông kế tiếp khi nhấn nút, mô phỏng thao tác điều khiển trực tiếp của con người.

S2: kết nối với chân GPIO 27 của khối xử lý trung tâm, được sử dụng để hệ thống vào chế độ ngủ để tiết kiệm năng lượng khi không cần hoạt động

Với 3 nút nhấn ở trên, điện trở 4.7K được mắc nối tiếp với nguồn 3.3V đến chân tín hiệu nhằm mục đích phân áp, đảm bảo các chân GPIO ở mức thấp khi nút chưa được nhấn. Ngoài ra, điện trở giúp tránh hiện tượng ngắn mạch khi nút được nhấn, nhằm bảo vệ vi điều khiển và các linh kiện. Khi nút được nhấn, các chân GPIO tương ứng sẽ bị kéo lên, giúp khối xử lý trung tâm nhận được tín hiệu ổn định và tránh nhiễu trong quá trình vận hành.

Đoạn code định nghĩa chân nút nhấn

```
#define SW0 32
#define SW1 33
#define SW2 27
```

Đoạn code chính xử lý nút nhấn

```
void checkButtons() {
    static unsigned long lastPressSW0 = 0;
    static unsigned long lastPressSW1 = 0;
    static unsigned long lastPressSW2 = 0;

    // == NÚT SW0: Chuyển đổi AUTO ↔ MANUAL ==
    if (digitalRead(SW0) == LOW && millis() - lastPressSW0 > 300) {
        delay(debounceDelay);
        if (digitalRead(SW0) == LOW) {
            currentMode = (currentMode == MODE_AUTO) ? MODE_MANUAL :
MODE_AUTO;
            Serial.println(currentMode == MODE_AUTO ? "Chế độ TỰ ĐỘNG" :
"Chế độ THỦ CÔNG");
            lastChangeTime = millis();
            updateTimeLeft();
            lastPressSW0 = millis();
        }
    }
}
```

```

    }
}
// == NÚT SW1: Chuyển trạng thái tiếp theo (chỉ hoạt động ở chế độ
MANUAL) ==
    if (currentMode == MODE_MANUAL && digitalRead(SW1) == LOW &&
millis() - lastPressSW1 > 300) {
        delay(debounceDelay);
        if (digitalRead(SW1) == LOW) {
            switch(currentState) {
                case STATE_1_GREEN:        currentState = STATE_1_YELLOW;
break;
                case STATE_1_YELLOW:        currentState = STATE_2_GREEN;
break;
                case STATE_2_GREEN:        currentState = STATE_2_YELLOW;
break;
                case STATE_2_YELLOW:        currentState = STATE_1_GREEN;
break;
            }
            updateLEDs();
            lastChangeTime = millis();
            updateTimeLeft();
            Serial.println("Chuyển trạng thái thủ công");
            lastPressSW1 = millis();
        }
    }
}

```

Đoạn code thực hiện chế độ ngủ

```

void checkSleepMode() {
    static unsigned long lastPress = 0;

```

```

if (digitalRead(SW3) == LOW && millis() - lastPress > 300) {
    delay(debounceDelay);
    if (digitalRead(SW3) == LOW) {
        sleepMode = !sleepMode; // Đảo trạng thái ngủ
        Serial.println(sleepMode ? "Vào chế độ ngủ" : "Thoát chế độ ngủ");
        lastPress = millis();
        if (!sleepMode) {
            lastChangeTime = millis();
            updateTimeLeft();
            updateLEDs();
        }
    }
}
}
}

```

3.2.3 Khối hiển thị

Khối hiển thị có chức năng cung cấp thời gian còn lại của đèn giao thông. Khối hiển thị được cấp nguồn 5V từ ESP32, các tín hiệu CLK và DIO lần lượt được kết nối qua chân GPIO17 và GPIO16, tín hiệu GND được nối đất.

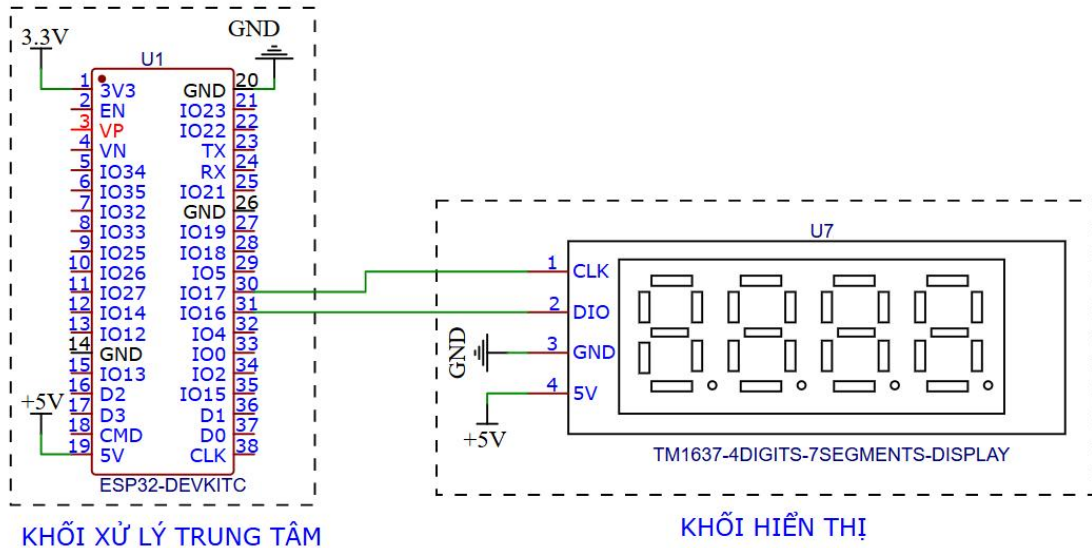
Để hiển thị thông tin bằng led 7 đoạn, các phương án có thể lựa chọn:

Phương án 1 - 4 led 7 đoạn cathode chung: Kết nối nhiều chân, hơi phức tạp. Cần phải sử dụng IC 74HC595 để giảm số chân kết nối. Phương án này không khả thi vì chiếm diện tích mạch điện thực tế.

Phương án 2 – module 4 led ma trận 8x8 MAX7219: Mỗi led trong ma trận được điều khiển chính xác, có thể tạo ra ký tự mong muốn. Người dùng có thể kiểm soát tất cả 64 led bằng cách chỉ kết nối 3 tín hiệu ngõ ra với vi điều khiển như Arduino, ESP32,... Tuy nhiên, giá thành cao hơn so với 4 led 7 đoạn ở trên, dòng điện tiêu thụ cao (300 – 500 mA). Phương án này nhóm thực hiện không cân nhắc.

Phương án 3 – module hiển thị 4 led 7 đoạn TM1637: Chỉ cần 2 chân kết nối, dễ sử dụng. Điện năng tiêu thụ ít, chi phí thấp, dễ tích hợp trên breadboard hoặc dự án IoT nhỏ.

Sau khi xem xét hệ thống, nhóm thực hiện chọn phương án 3 vì đây là lựa chọn phù hợp với mạch điện đơn giản, giúp tiết kiệm chân cho ESP32.



Hình 3.3 Mô tả kết nối của khối hiển thị

Bảng 3.2 Các lệnh cấu hình TM1637 được sử dụng

STT	Lệnh	Ý nghĩa
1	<code>display.setBrightness();</code>	Tùy chỉnh độ sáng (0 – 7)
2	<code>display.clear();</code>	Xóa màn hình
3	<code>display.setSegments(data);</code>	Hiển thị từng đoạn led theo ký tự tùy chỉnh
4	<code>display.showNumberDecEx(number, dots, leading_zeros);</code>	Hiển thị số có định dạng thập phân (ví dụ 15.05)

Đoạn code hoạt động của module hiển thị TM1637

```
#include <TM1637Display.h>
// Định nghĩa chân TM1637
#define TM1637_CLK 17
```

```

#define TM1637_DIO 16

// Biến toàn cục
TM1637Display display(TM1637_CLK, TM1637_DIO);

// Khởi tạo TM1637
display.setBrightness(0x0f); // Độ sáng tối đa
display.clear();

void displayTime() {
    if (sleepMode) {
        // Hiển thị "SL" cho sleep mode
        uint8_t sleepData[] = {
            SEG_A | SEG_F | SEG_G | SEG_C | SEG_D, // S
            SEG_D | SEG_E | SEG_F,                // L
            0x00, 0x00
        };
        display.setSegments(sleepData);
    } else if (currentMode == MODE_MANUAL) {
        // Hiển thị "-- --" cho manual mode
        uint8_t manualData[] = {
            SEG_G, SEG_G, SEG_G, SEG_G
        };
        display.setSegments(manualData);
    } else {
        // Hiển thị thời gian cho cả 2 hướng (ví dụ: 15 05)
        // 2 số đầu: hướng 1, 2 số cuối: hướng 2
        int displayNumber = (currentTimeLeft1 * 100) + currentTimeLeft2;
        display.showNumberDecEx(displayNumber, 0b1110000, true); // Hiển thị với
        dấu chấm ở giữa
    }
}

```

3.2.4 Khối xử lý trung tâm

Khối xử lý trung tâm trong hệ thống thực hiện nhận hình ảnh do khối camera gửi đến, sau đó phân tích và dự đoán mật độ phương tiện giao thông bằng cách áp dụng mô hình trí tuệ nhân tạo. Dựa trên kết quả phân tích, khối này tính toán mức độ đông đúc của các làn đường và chuyển dữ liệu đã xử lý sang khối điều khiển giao thông qua giao thức MQTT

Để đảm nhận việc xử lý mô hình AI, một số thiết bị có thể lựa chọn, bao gồm: ESP32-CAM, Raspberry Pi 4 và Jetson Nano.

Phương án 1: ESP32-CAM là vi điều khiển tích hợp camera, kích thước nhỏ gọn, giá thành thấp và tiêu thụ điện năng ít, phù hợp cho các ứng dụng chỉ cần chụp ảnh và gửi dữ liệu hình ảnh lên máy chủ; tuy nhiên khả năng chạy mô hình AI trực tiếp rất hạn chế.

Phương án 2: Raspberry Pi 4 là máy tính nhúng mạnh mẽ, hỗ trợ chạy các mô hình AI nhỏ như MobileNet, YOLO-tiny hoặc TensorFlow Lite, đồng thời có thể xử lý mô hình AI với độ chính xác khá cao.

Phương án 3: Jetson Nano là máy tính AI của NVIDIA được trang bị GPU 128 nhân Maxwell, hỗ trợ các framework như PyTorch, TensorFlow và OpenCV, cho phép xử lý hình ảnh thời gian thực và chạy các mô hình deep learning phức tạp hơn, đem lại hiệu năng và độ chính xác cao; tuy nhiên chi phí rất đắt đỏ.

Dựa vào hệ thống giám sát mật độ phương tiện giao thông, Raspberry Pi 4 là phương án phù hợp nhất vì nó đạt được sự cân bằng tối ưu giữa hiệu năng xử lý, khả năng tích hợp và chi phí. Thiết bị này còn tương thích tốt với nhiều loại camera như Camera Pi 5MP, Camera Module 3 hay webcam USB; đồng thời hỗ trợ kết nối mạng WiFi và Ethernet giúp truyền dữ liệu nhanh và ổn định đến khối xử lý trung tâm.

3.2.5 Khối điều khiển giao thông

Khối điều khiển giao thông có vai trò xử lý thông tin nhận được từ khối xử lý trung tâm và điều khiển hoạt động của tín hiệu giao thông

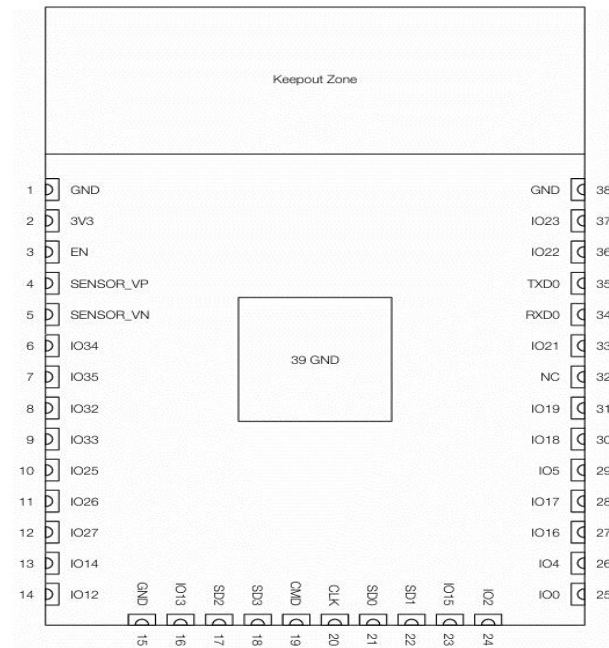
Hiện nay, trên thị trường có rất nhiều loại vi điều khiển với nhiều mục đích sử dụng khác nhau, từ đơn giản đến phức tạp. Hệ thống giám sát mật độ phương tiện sử dụng kết hợp phần cứng và phần mềm, những vi điều khiển có thể sử dụng được mô tả trong bảng 3.3

Bảng 3.3 Một số vi điều khiển phổ biến

STT	TÊN GỌI	ĐẶC ĐIỂM
1	Arduino Uno (ATmega328P)	- Dễ tiếp cận, tài liệu phong phú, cộng đồng lớn. Có nhiều shield hỗ trợ, dễ mở rộng - Hiệu năng thấp, RAM và flash hạn chế. Không tích hợp kết nối không dây
2	ESP32 (Espressif)	- Tích hợp WiFi + Bluetooth, cực kỳ phù hợp cho IoT. Nhiều chân I/O, tốc độ cao, hỗ trợ dual-core. Chi phí thấp, tính năng mạnh mẽ - Một số module dễ nhiễu nếu không cấp nguồn ổn định
3	STM32	- Hiệu năng mạnh, hỗ trợ nhiều dòng từ cơ bản đến cao cấp. Tích hợp nhiều ngoại vi: ADC, DAC, CAN, I2C, SPI,... - Cần kiến thức vững để lập trình (phức tạp hơn Arduino). Cộng đồng hỗ trợ không phổ biến bằng Arduino/ESP32

Hệ thống giao thông thông minh đòi hỏi một vi điều khiển có khả năng xử lý nhanh, hỗ trợ kết nối không dây ổn định và dễ dàng tích hợp với các cảm biến. Với những yêu cầu này, ESP32-WROOM là một lựa chọn phù hợp. Vi điều

khởi này được trang bị bộ xử lý lõi kép Xtensa 32-bit LX6, hoạt động ở tần số lên đến 240 MHz, cùng với bộ nhớ Flash 4 MB và khả năng kết nối WiFi và Bluetooth tích hợp sẵn. Bên cạnh đó, nhờ thiết kế nhỏ gọn, tiêu thụ điện năng thấp và khả năng lập trình linh hoạt qua Arduino IDE nên ESP32-WROOM trở thành giải pháp tối ưu cho hệ thống giám sát giao thông thông minh.



Hình 3.4 Sơ đồ chân của khối xử lý trung tâm

Hệ thống sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm để kết nối và điều khiển các thiết bị ngoại vi. Vi điều khiển này được cấp nguồn hoạt động 5V và GND tại các chân số 19, 20. Các chân dùng để giao tiếp IO với các nút nhấn lần lượt là GPIO32, GPIO33, GPIO27; đối với đèn tín hiệu giao thông : GPIO23, GPIO22, GPIO21, GPIO19, GPIO18, GPIO5. Ngoài ra, ESP32 còn sử dụng 2 chân GPIO17 và GPIO16 để kết nối với module hiển thị TM1637. ESP32 đảm nhận vai trò thu thập kết quả dự đoán mật độ phương tiện từ Raspberry thông qua WiFi và đưa dữ liệu lên Firebase và Web, từ đó điều khiển tín hiệu giao thông với các mật độ phương tiện tương ứng.

Để xử lý thông tin và trao đổi dữ liệu của hệ thống, ESP32 cần sử dụng một số thư viện quan trọng. Thư viện WiFi.h cho phép kết nối mạng WiFi, có vai trò

hỗ trợ việc truyền và nhận dữ liệu. Thư viện WiFiClient.h hỗ trợ tạo kết nối TCP client để gửi dữ liệu tới server. Thư viện WiFiClient.h hỗ trợ tạo kết nối TCP client để gửi dữ liệu tới server. Thư viện Firebase-ESP-Client.h cho phép gửi dữ liệu lên Firebase. Thư viện TM1637Display.h hỗ trợ giao tiếp dữ liệu với module hiển thị 4 led 7 đoạn.

3.2.6 Khởi nguồn

Khởi nguồn có nhiệm vụ cung cấp nguồn hoạt động cho vi điều khiển trung tâm, các cảm biến và đèn giao thông. Đầu ra điện áp 5V và dòng điện tối đa có thể cung cấp tới 5A đảm bảo hệ thống có thể hoạt động ổn định.

Để cung cấp nguồn cho hệ thống, hiện có rất nhiều module ổn áp có thể sử dụng chuyển đổi điện áp để phù hợp với nguồn 5V mà ESP32-WROOM và các module thường dùng. Một số module ổn áp có thể được kể đến như: mạch ổn áp LM78S05, mạch ổn áp WH-468, mạch nguồn ổn áp LM2596S.

Phương án 1: mạch ổn áp LM7805 5V 1.2 A

Phương án 2: mạch ổn áp HW-468 5V 5A

Phương án 3: mạch ổn áp LM2596S 5V 3A

Dưới đây là bảng thống kê thông số nguồn và dòng tiêu thụ của các module, cảm biến trong hệ thống.

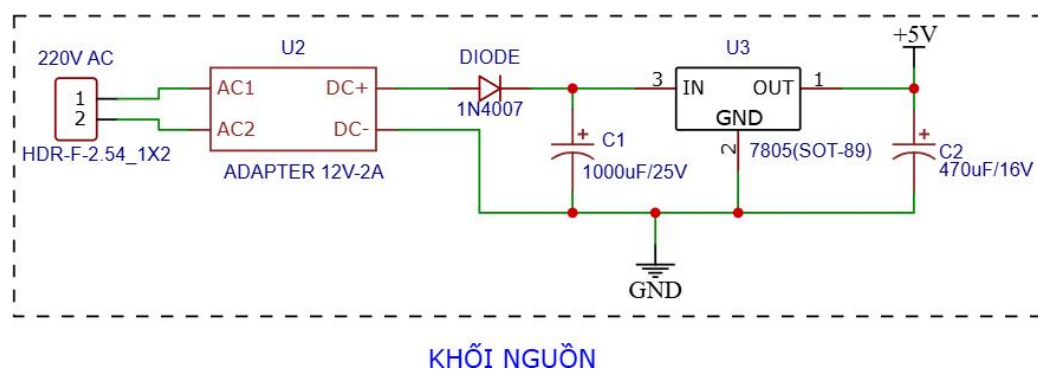
Bảng 3.4 Thông số điện áp và dòng tiêu thụ của linh kiện trong hệ thống

STT	Tên linh kiện	Điện áp tiêu thụ	Dòng điện tiêu thụ	Số lượng
1	ESP32-WROOM-32E	5 V	500 mA	01
2	Nút nhấn 6x6mm	3.3 V	50 mA	03
3	Module đèn led giao thông	5 V	30 mA	02
4	Module 4 led 7 đoạn TM1637	3.3 V	80 mA	01
5	Raspberry Pi 4 Model B	5 V	3000 mA	01

6	Webcam 1080p	5 V	500 mA	01
Tổng dòng điện tiêu thụ			4290 mA	

Do Raspberry Pi 4 sử dụng Adapter 5.1V-3A nên ta có thể kết nối trực tiếp với nguồn điện 220V. Sau khi trừ đi dòng điện tiêu thụ của Raspberry Pi 4 và Webcam 1080p (kết nối trực tiếp với Raspberry Pi 4) thì tổng dòng điện của toàn hệ thống chỉ còn 790 mA.

Như vậy, tổng dòng điện tiêu thụ của hệ thống gần 1A, việc lựa chọn mạch ổn áp LM78S05 có dải điện áp đầu vào rộng từ 7 - 25V, cung cấp điện áp 5V, dòng điện tối đa 1.2A phù hợp với hệ thống trên.

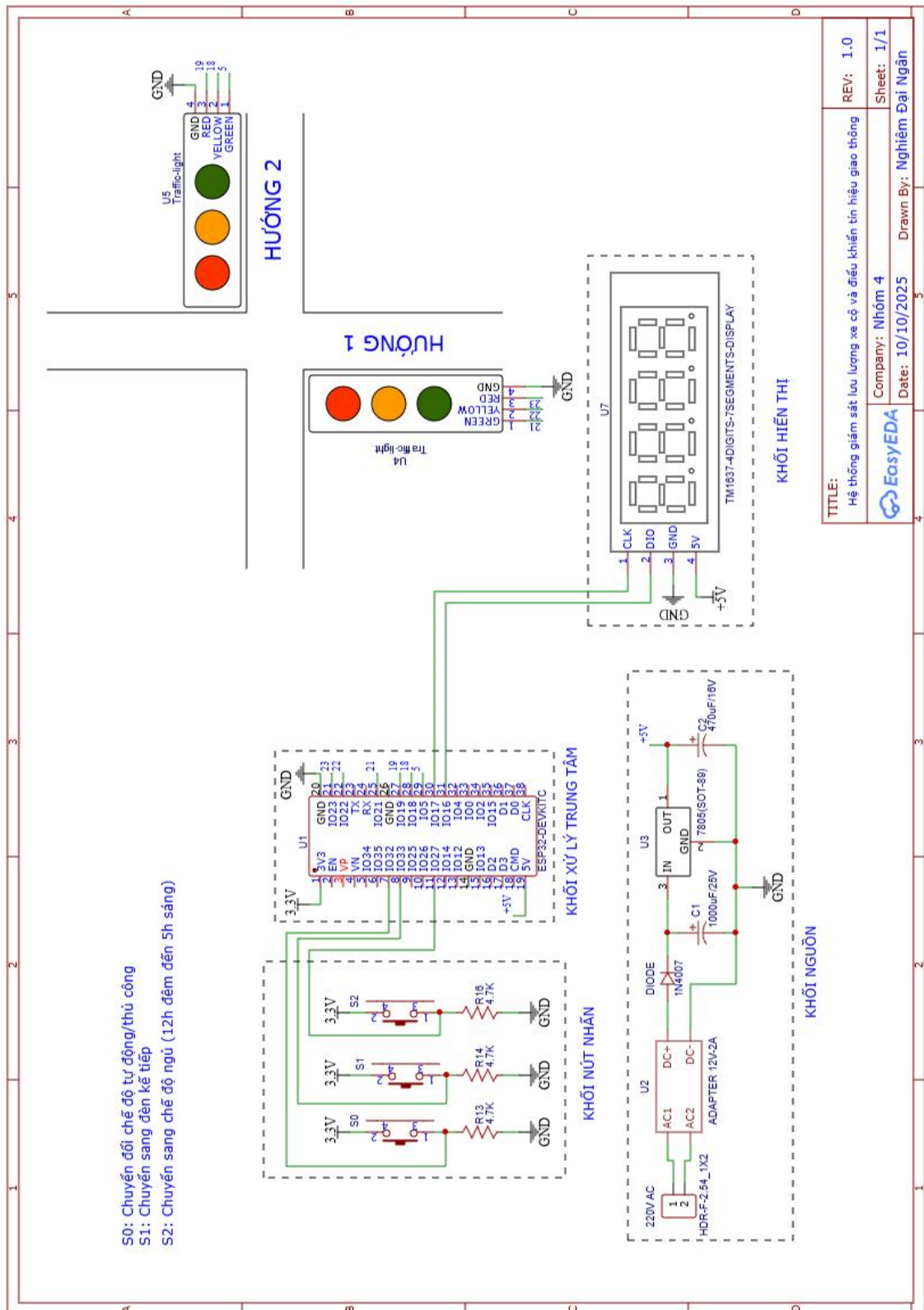


Hình 3.5 Mô tả kết nối của khối nguồn

Khối nguồn sử dụng mạch ổn áp LM7805^[8], chân Vin kết nối với ngõ ra của Adapter 12V để cấp điện. Tụ lọc C1 (1000uF/25V) đóng vai trò lọc phẳng điện áp đầu vào, giúp loại bỏ các gợn sóng và nhiễu từ Adapter để cung cấp dòng điện ổn định cho IC 7805. Trong khi đó, tụ lọc C2 (470uF/16V) có nhiệm vụ ổn định điện áp đầu ra và chống dao động tự kích, đảm bảo nguồn điện 5V luôn ổn định và an toàn. Nguồn điện 220V được chuyển đổi thành điện áp 5V được cấp phát cho toàn bộ hệ thống.

3.2.7 Sơ đồ nguyên lý toàn hệ thống

Sơ đồ nguyên lý toàn mạch thể hiện kết nối của toàn bộ các khối có trong hệ thống như hình 3.6:



Hình 3.6 Sơ đồ nguyên lý toàn hệ thống

3.3 LỰA CHỌN VÀ HUẤN LUYỆN CÁC MODULE AI

3.3.1 Lựa chọn mô hình cho Module Giám sát Giao thông

Để phân tích lưu lượng xe và lỗi vi phạm đèn vạch ảo từ camera trên cao, hệ thống yêu cầu một giải pháp AI có tốc độ suy luận nhanh (đáp ứng tính thời gian thực) và khả năng duy trì định danh bám vết tốt. Nhóm thực hiện đã tiến hành so sánh đối chứng giữa 3 phương án công nghệ phổ biến:

*** Phương án 1 (YOLOv11n + ByteTrack):**

- Ưu điểm: Kiến trúc YOLOv11 phiên bản Nano có kích thước cực kỳ tinh giản (~2.6 triệu tham số, dung lượng tệp trọng số ~5.2 MB), rất thuận lợi để lượng tử hóa và suy luận hiệu năng cao thông qua framework Tencent NCNN trên vi xử lý ARM. Thuật toán ByteTrack giải quyết bài toán theo dõi đa đối tượng bằng cách liên kết cả các hộp giới hạn có điểm tin cậy thấp thông qua bộ lọc Kalman và ma trận IoU, giúp duy trì tốt Track ID xe khi xảy ra hiện tượng che khuất một phần.

- Nhược điểm: Độ chính xác phát hiện bị giảm nhẹ đối với các đối tượng xe cộ nằm ở quá xa camera.

*** Phương án 2 (Faster R-CNN + DeepSORT):**

- Ưu điểm: Độ chính xác phát hiện và định vị khung bao rất cao nhờ cấu trúc mạng hai giai đoạn (Two-stage).

- Nhược điểm: Tốc độ suy luận rất chậm (chu kỳ xử lý từ 200 - 300 ms trên CPU), mô hình cồng kềnh, gây quá tải bộ nhớ và hoàn toàn không thể chạy thời gian thực trên Raspberry Pi 4.

*** Phương án 3 (SSD MobileNet + SORT):**

- Ưu điểm: Kích thước mô hình siêu nhỏ, tối ưu tốt cho CPU di động.
- Nhược điểm: Thuật toán SORT chỉ sử dụng hình học IoU đơn giản nên dễ gây mất dấu và gán nhãn trùng lặp (ID Switch) khi mật độ giao thông tại ngã tư đông đúc.

Bảng 3.5 So sánh các mô hình cho Module Giám sát

Tiêu chí so sánh	YOLOv11n + ByteTrack (Được chọn)	Faster R-CNN + DeepSORT	SSD MobileNet + SORT
Kiến trúc mạng	Mạng phát hiện 1 giai đoạn + Bám vết lọc Kalman phi ReID	Mạng phát hiện 2 giai đoạn + Bám vết qua trích xuất mạng ReID	Mạng phát hiện 1 giai đoạn + Bám vết hình học IoU thô
Tốc độ suy luận	Rất nhanh (Đạt mức thời gian thực)	Rất chậm (Không khả thi trên Edge)	Nhanh (Đáp ứng tốt thời gian thực)
Độ ổn định định danh (ID)	Xuất sắc, khắc phục tốt lỗi che khuất xe	Rất tốt nhưng tốn tài nguyên ReID	Kém, thường xuyên xảy ra lỗi ID Switch
Triển khai trên Edge	Tối ưu hóa sâu qua Tencent NCNN	Kém, quá tải RAM và CPU ngay lập tức	Tốt, dung lượng mô hình siêu nhẹ

=> Kết luận lựa chọn: Nhóm thực hiện quyết định chọn giải pháp YOLOv11n kết hợp ByteTrack làm kiến trúc cốt lõi cho Edge Agent, đảm bảo đạt tốc độ xử lý tiệm cận thời gian thực trên CPU Raspberry Pi 4 mà vẫn duy trì tính ổn định định danh xe vi phạm.hơn.

3.3.2 Lựa chọn mô hình cho Module Đọc Biển số

Tác vụ giải mã biển số xe vi phạm yêu cầu độ chính xác tuyệt đối và được xử lý tập trung trên máy chủ trung tâm (Central Server) để tận dụng năng lực tính toán hiệu năng cao. Hai phương án công nghệ đã được đưa ra cân nhắc:

- * Phương án 1 (YOLOv11n + CRNN): Sử dụng YOLOv11n để định vị và cắt chính xác vùng chứa biển số xe trên ảnh xe vi phạm, sau đó đưa vùng ảnh biển

số qua mạng nơ-ron hồi quy tích chập (CRNN) để giải mã chuỗi ký tự. CRNN kết hợp CNN trích xuất đặc trưng không gian và Bi-GRU học chuỗi ngữ cảnh thời gian giúp giải mã trực tiếp toàn bộ chuỗi ký tự biển số xe Việt Nam (kể cả biển 1 dòng và biển 2 dòng) thông qua hàm mất mát giải mã CTC (Connectionist Temporal Classification) mà không cần phải thực hiện bước phân đoạn cắt rời từng chữ số, hạn chế tối đa sai số tích lũy.

* Phương án 2 (YOLOv11 + Tesseract OCR): Định vị biển số bằng YOLO và nhận dạng chữ bằng thư viện mã nguồn mở Tesseract OCR. Phương án này dễ tích hợp nhưng độ chính xác rất kém khi ảnh biển số bị nghiêng, mờ hoặc dính bụi bẩn và thường xuyên đọc sai cấu trúc biển số 2 dòng tại Việt Nam.

=> Kết luận lựa chọn: Giải pháp YOLOv11n kết hợp CRNN được chọn làm kiến trúc cho phân hệ OCR tại Central Server, giải quyết triệt để bài toán nhận dạng ký tự biển số dạng chuỗi có cấu trúc phức tạp.

3.3.3 Thu thập dữ liệu và Gán nhãn huấn luyện các mô hình AI

Quy trình xây dựng tập dữ liệu và huấn luyện các mô hình học sâu chuyên biệt được thực hiện như sau:

1. Tập dữ liệu Giám sát phương tiện:

- Thu thập dữ liệu: Nhóm sử dụng tập dữ liệu hình ảnh giao thông công cộng "UIT-ADrone DATASET"^[14] do nhóm nghiên cứu UIT Together phát hành. Tập dữ liệu gồm các video độ phân giải Full HD ghi hình dòng xe cộ từ thiết bị bay không người lái ở độ cao 50m – 70m, mô phỏng góc nhìn thực tế của camera giám sát lắp đặt trên cột cao tại các giao lộ Việt Nam.

- Gán nhãn: Tiến hành trích xuất khung hình và gán nhãn khung bao (Bounding Box) theo cấu trúc YOLO cho 4 lớp phương tiện đặc trưng: motorbike (xe máy), car (ô tô), truck (xe tải), và bus (xe buýt).

2. Tập dữ liệu Nhận diện biển số xe (License Plate OCR):

- Định vị biển số: Nhóm thu thập tập dữ liệu công khai "License_Plates_VietNam" gồm 4.534 hình ảnh xe cộ tại Việt Nam từ nền tảng

Roboflow. Các hình ảnh được gán nhãn đa giác (polygon) bao quanh biển số xe và phân loại thành 2 lớp: long-license-plates (biển số dài) và short-license-plates (biển số ngắn).

- Nhận dạng ký tự (OCR): Sử dụng bộ dữ liệu gồm các ảnh biển số đã được cắt (crop) cận cảnh từ các xe lưu thông tại Việt Nam. Tập dữ liệu được chuẩn hóa và gán nhãn chuỗi ký tự text tương ứng (gồm các chữ số 0-9 và chữ cái Alphabet viết hoa), phục vụ huấn luyện mạng CRNN với hàm mất mát phân loại chuỗi CTC Loss.

3.4 THIẾT KẾ SA BÀN GIAO LỘ MÔ PHỎNG

3.4.1 Quy mô và cấu trúc của sa bàn

Sa bàn giao lộ đóng vai trò là môi trường thử nghiệm vật lý giả lập để kiểm nghiệm toàn diện hoạt động khép kín của hệ thống. Sa bàn có kích thước tổng thể là

400mm x 600mm, mô phỏng một ngã tư đường phố tiêu chuẩn với 4 hướng giao lộ vuông góc:

- Vật liệu và tạo hình: Nền sa bàn được phủ tấm nhựa Formex màu xám đen để tạo độ tương phản cao với vạch kẻ đường, giả lập mặt đường nhựa thực tế. Các làn đường di chuyển (mỗi hướng gồm 1 làn đi và 1 làn về) được phân chia bằng các vạch kẻ đường màu trắng nét đứt hoặc nét liền có độ rộng 1.5cm, giúp camera nhận diện và phân định ranh giới di chuyển.

- Bố trí phần cứng thực địa: Tại 4 góc ngã tư, dựng 4 cột đèn giao thông chứa các cụm LED vật lý kết nối với board ESP32 đặt dưới gầm sa bàn. Một giá treo camera (webcam) đơn được lắp cố định ở độ cao 80cm ngay phía trên tâm giao lộ, hướng vuông góc từ trên xuống dưới để ghi hình toàn cảnh ngã tư, đảm bảo không có điểm mù ở các góc tiếp cận

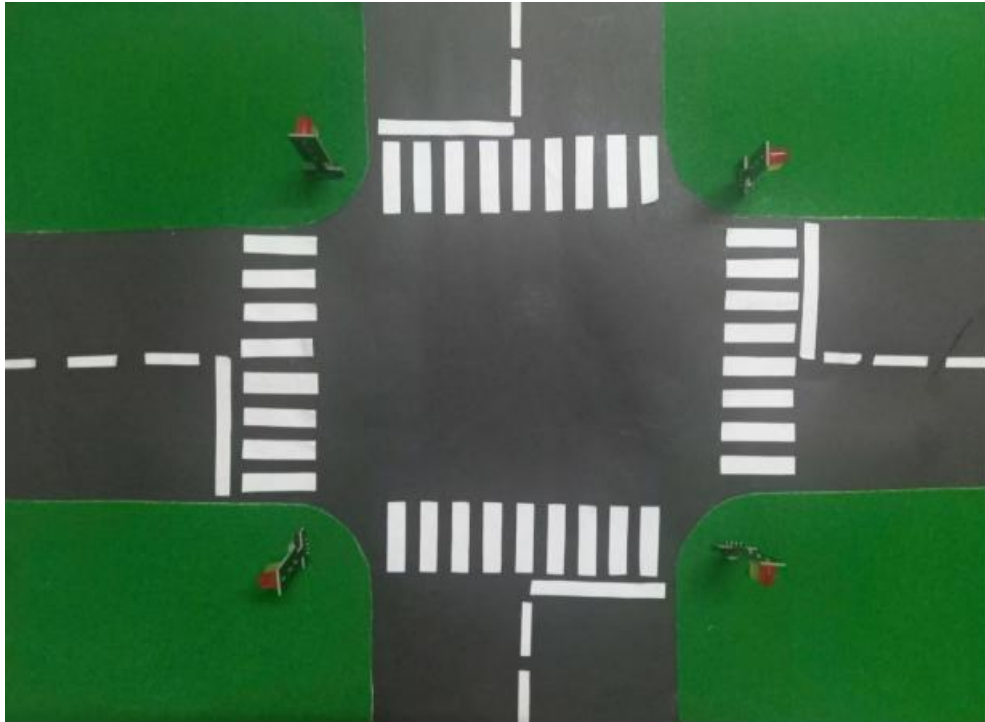
3.4.2 Các kịch bản giao thông thử nghiệm trên sa bàn

Nhóm xây dựng các kịch bản thử nghiệm vật lý mô phỏng để đánh giá thuật toán:

- Kịch bản 1 - Điều khiển pha đèn thích ứng: Đặt các mô hình xe đồ chơi (xe máy, ô tô) lên các làn đường với mật độ khác nhau (ví dụ: Hướng 1 có 8 xe, Hướng 2 có 2 xe). Camera ghi hình gửi về Raspberry Pi 4 để đếm xe trong các vùng ROI,

sau đó gửi mật độ qua MQTT để ESP32 tự động tính toán tăng thời gian đèn xanh của Hướng 1 và giảm Hướng 2.

- Kịch bản 2 - Phát hiện vi phạm vượt đèn đỏ phạt nguội: Khi hướng đèn giao thông chuyển đỏ, điều khiển một xe đồ chơi di chuyển vượt qua vạch dừng xe vật lý trên sa bàn (cắt qua tọa độ vạch cấm ảo được định nghĩa trong cấu hình ROI). Camera ghi nhận xe vi phạm, Raspberry Pi 4 crop ảnh, dán biển số Homography và gửi bất đồng bộ lên FastAPI Server để lưu trữ phạt nguội.



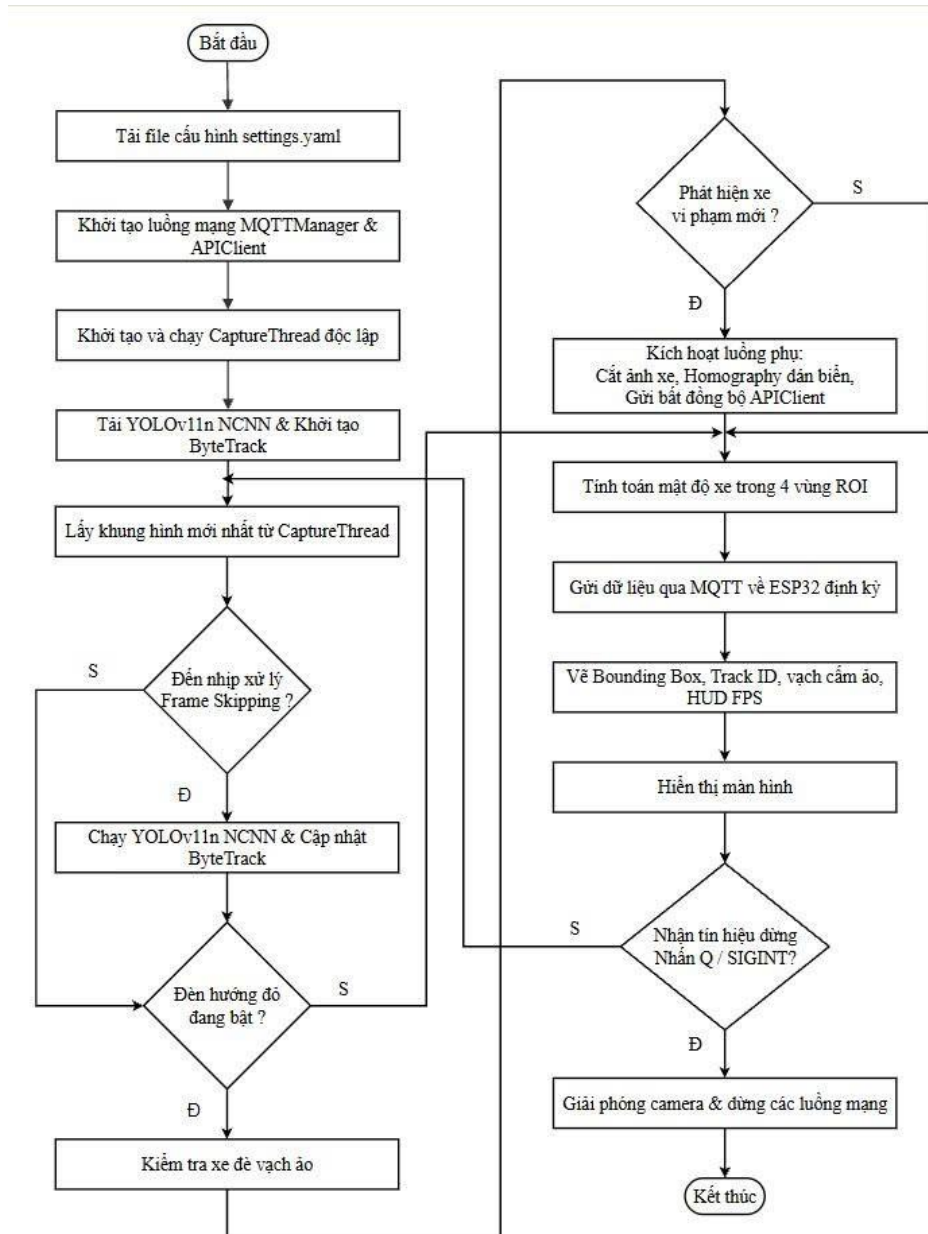
Hình 3.7 Sa bàn giao lộ kích thước 40cm x 60cm

CHƯƠNG 4: THI CÔNG - LẬP TRÌNH HỆ THỐNG

4.1 LƯU ĐỒ GIẢI THUẬT CỦA HỆ THỐNG

4.1.1 Lưu đồ giải thuật chương trình chính trên Raspberry Pi 4

Chương trình chính tại thiết bị biên vận hành theo lưu đồ hình 4.1:



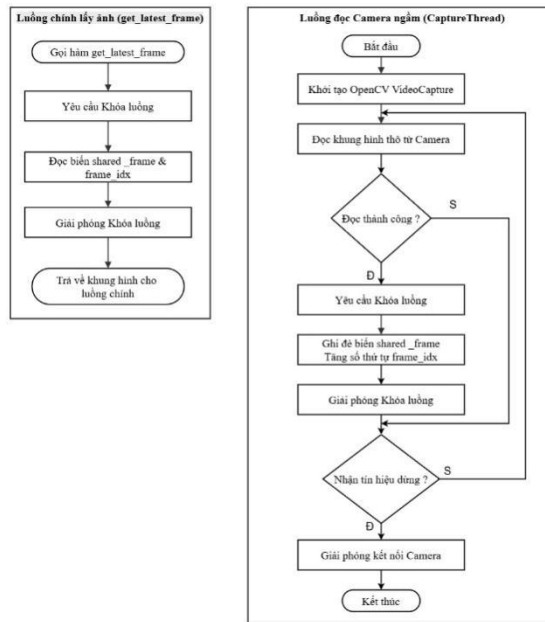
Hình 4.1 Lưu đồ giải thuật chương trình chính trên Raspberry Pi 4

Khi khởi chạy, chương trình thực hiện các bước:

- Khởi tạo cấu hình: Nạp các tham số hệ thống từ tệp settings.yaml.
 - Khởi chạy luồng phụ: Khởi động luồng bắt hình camera độc lập CaptureThread, luồng truyền thông MQTTManager để subscribe trạng thái đèn từ ESP32, và luồng HTTP Client APIClient để gửi ảnh bắt đồng bộ.
 - Nạp mô hình AI: Khởi tạo lớp VehicleDetector để tải mô hình YOLOv11n định dạng NCNN đã lượng tử hóa FP16 vào RAM, đồng thời khởi tạo bộ bám vết TrafficFlowAnalyzer và bộ phát hiện vi phạm ViolationDetector.
 - Vòng lặp suy luận (Inference Loop): Liên tục lấy khung hình mới nhất từ CaptureThread. Áp dụng cơ chế bỏ qua khung hình (Frame Skipping) theo tham số skip_factor. Tại các nhịp xử lý, mô hình YOLOv11n NCNN sẽ phát hiện phương tiện, truyền tọa độ cho ByteTrack để cập nhật vết. Nếu đèn đỏ đang bật, hệ thống kiểm tra xe đèn vạch ảo. Khi có vi phạm, vùng ảnh xe được cắt, dán biển số Homography và đẩy vào hàng đợi truyền HTTP POST.
5. Giải phóng tài nguyên: Khi nhận tín hiệu ngắt (Ctrl+C hoặc SIGINT), hệ thống tắt các luồng ngầm, đóng kết nối Mosquitto Broker và giải phóng vùng nhớ.

4.1.2 Lưu đồ giải thuật camera đa luồng

Cơ chế thu nhận hình ảnh đa luồng được thực hiện theo lưu đồ hình 4.2:



Hình 4.2 Lưu đồ giải thuật Camera đa luồng

Để giải quyết bài toán camera bị giật hình (stall) do thời gian suy luận AI của CPU tốn từ 80 – 120ms, luồng thu nhận ảnh độc lập CaptureThread được thiết kế theo mẫu Producer-Consumer sử dụng thư viện threading.Thread trong Python:

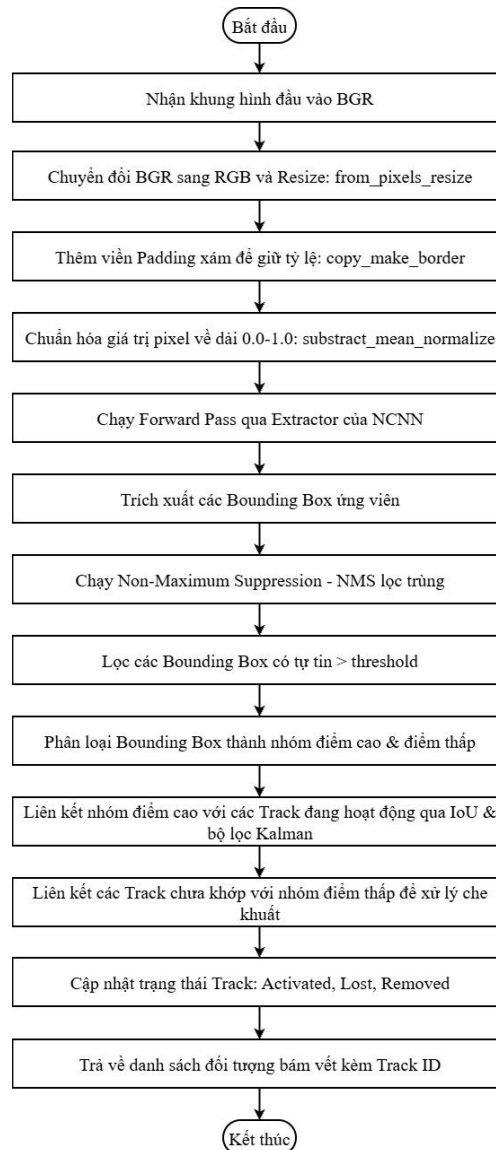
- Producer (Luồng phụ): Khởi tạo camera qua V4L2 (trên Linux) hoặc đọc tệp video quay sẵn (mô phỏng). Đối với tệp video, luồng tính toán tốc độ FPS gốc để duy trì chu kỳ đọc (pacing interval) bằng hàm time.sleep(), tránh việc đọc quá nhanh làm tràn RAM. Mỗi khi đọc được khung hình mới, luồng phụ sẽ khóa tài nguyên bằng threading.Lock(), thực hiện resize ảnh ngay trên luồng phụ về độ phân giải hiển thị (1280×720), ghi đè vào biến chia sẻ self._frame, tăng bộ đếm self._frame_count và giải phóng khóa.

- Consumer (Luồng suy luận chính): Luồng chính không gọi trực tiếp các hàm I/O phần cứng của OpenCV. Thay vào đó, nó gọi hàm get_latest_frame() để chiếm khóa trong mili-giây, sao chép tức thời khung hình đã được resize từ

RAM ra vùng nhớ xử lý của luồng chính và giải phóng khóa. Cơ chế an toàn luồng này giúp triệt tiêu hoàn toàn độ trễ I/O phần cứng.

4.1.3 Lưu đồ giải thuật xử lý nhận dạng và bám vết đa đối tượng

Quy trình nhận diện phương tiện giao thông bằng mô hình YOLOv11n NCNN kết hợp bám vết đa đối tượng bằng ByteTrack được thực hiện theo lưu đồ hình 4.3:



Hình 4.3 Lưu đồ giải thuật xử lý nhận dạng và bám vết đa đối tượng

Thuật toán kết hợp giữa lớp VehicleDetector (NCNN C++ backend) và TrafficFlowAnalyzer (ByteTrack):

- Tiền xử lý NCNN: Ảnh đầu vào được chuyển đổi hệ màu BGR sang RGB và resize giữ nguyên tỉ lệ (Letterbox) về kích thước đầu vào của mô hình (320×320) bằng hàm native cực nhanh của C++ `ncnn.Mat.from_pixels_resize()`. Tiếp tục đệm viền xám bằng `ncnn.copy_make_border()` và chuẩn hóa về dải $[0.0, 1.0]$ bằng `subtract_mean_normalize()`.

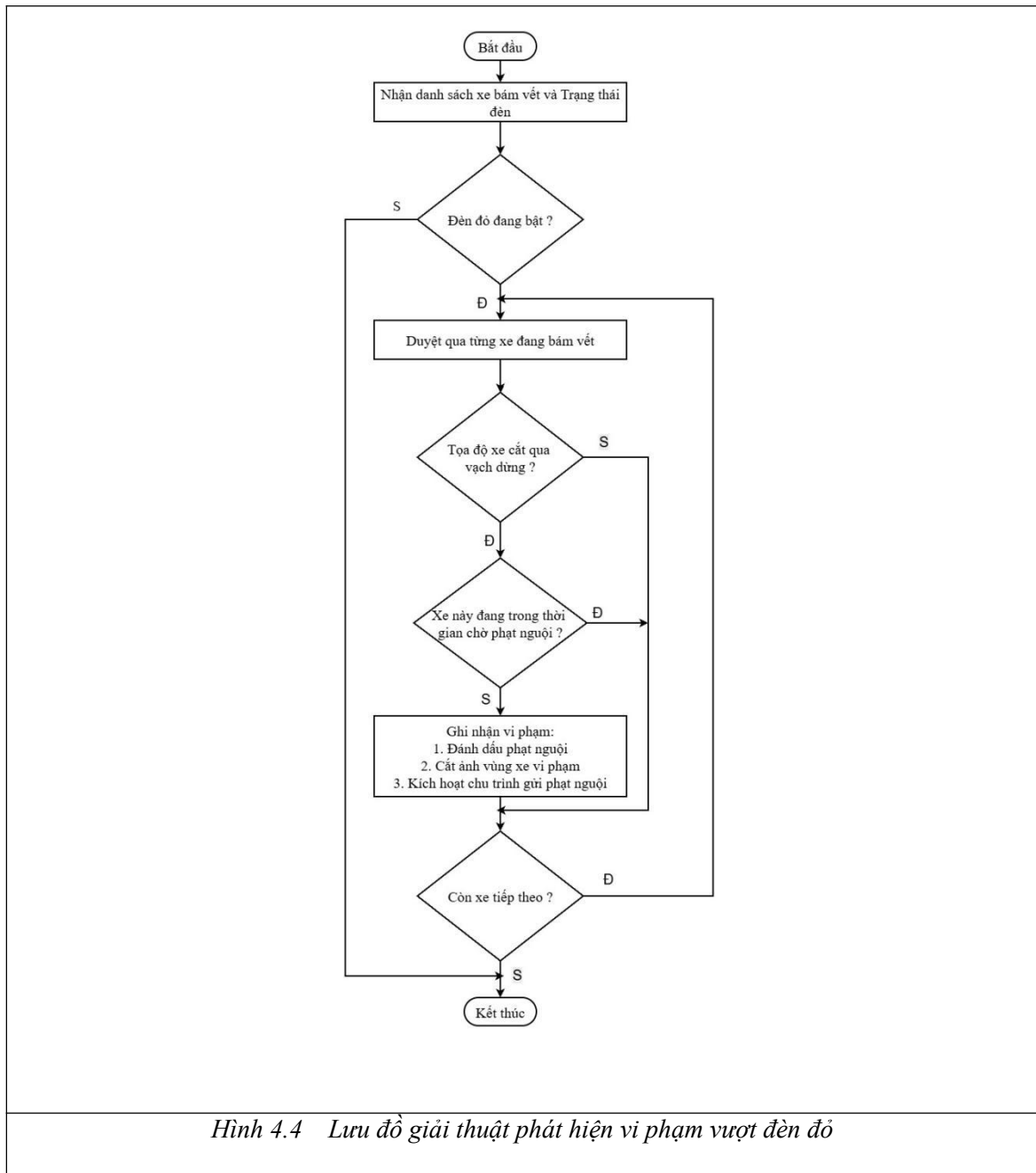
- Suy luận và Hậu xử lý: Tensors đầu ra được clone và chuyển đổi sang mảng NumPy. Áp dụng kỹ thuật Class-aware NMS bằng cách dịch chuyển tọa độ các khung bao theo từng nhãn lớp xe (Offset technique), sau đó sử dụng hàm OpenCV C++ `cv2.dnn.NMSBoxes()` với `conf_threshold = 0.25` và `nms_threshold = 0.45` để loại bỏ các khung bao trùng lặp.

- Bám vết đa đối tượng (ByteTrack): Các bounding box hợp lệ được đưa vào thuật toán ByteTrack. ByteTrack sử dụng bộ lọc Kalman để dự báo vị trí tiếp theo của xe và khớp ma trận tương đồng (IoU) theo hai giai đoạn: giai đoạn một khớp các khung bao có độ tin cậy cao; giai đoạn hai khớp các khung bao có độ tin cậy thấp còn lại để bám vết liên tục khi xe bị che khuất tạm thời.

- Đếm lưu lượng và Tránh rò rỉ bộ nhớ: Lớp TrafficFlowAnalyzer kiểm tra xem tọa độ tâm của xe đã nằm trong vùng đa giác ROI của làn đường hay chưa. Khi phát hiện xe mới đi vào vùng đa giác, hệ thống tăng số lượng xe tích lũy của nhãn lớp đó và lưu Track ID vào tập hợp `self._seen_ids`. Để tránh rò rỉ bộ nhớ khi chạy dài hạn 24/7, khi kích thước tập hợp vượt quá 10.000 phần tử, hệ thống tự động giải phóng tập hợp (pruning).

4.1.4 Lưu đồ giải thuật chương trình con phát hiện vi phạm vượt đèn đỏ

Thuật toán kiểm tra hành vi đèn vạch cấm ảo khi đèn đỏ đang hoạt động được thể hiện chi tiết qua lưu đồ hình 4.4:



Module Thuật toán được tích hợp trong lớp ViolationDetector:

- Kích hoạt điều kiện: Thuật toán chỉ duyệt danh sách xe bám vết khi nhận được trạng thái đèn đỏ của làn đường tương ứng từ ESP32 gửi qua MQTT (lane2_is_red = True).

- So khớp hình học (Line Crossing Detection): Với mỗi phương tiện đang bám vết, hệ thống kiểm tra xem bounding box của nó (x₁,y₁,x₂,y₂) có giao

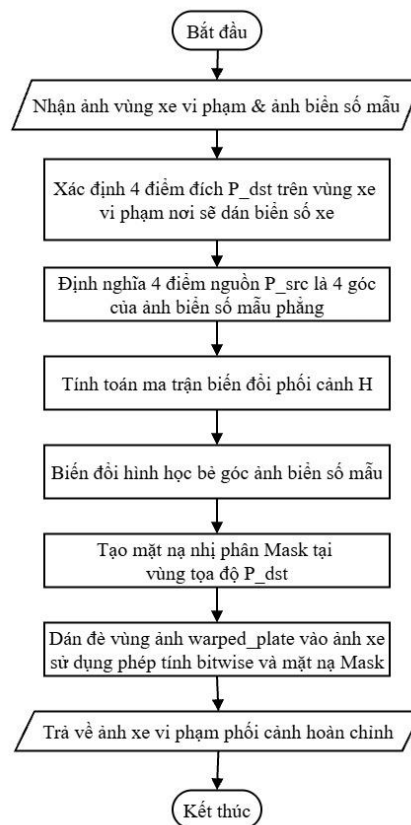
cắt với vạch cấm dừng xe nằm ngang (nối từ line_x1 đến line_x2 tại độ cao line_y) hay không. Điều kiện giao cắt được xác định bằng hệ thức:

$$(y_1 \leq line_y \leq y_2, x_1 \leq line_{x_2}, x_2 \geq line_{x_1})$$

- Bộ lọc trùng lặp Cooldown: Để tránh việc một xe dừng đè lên vạch cấm liên tục kích hoạt gửi hàng trăm ảnh vi phạm, hệ thống sử dụng bộ lọc Cooldown dựa trên thời gian thực tế time.monotonic(). Mỗi Track ID vi phạm chỉ được gửi ảnh lên máy chủ 1 lần duy nhất trong khoảng thời gian cooldown_sec = 5.0 giây. Dữ liệu thời gian vi phạm của các ID cũ hơn 4 lần chu kỳ cooldown sẽ bị xóa khỏi bộ đệm để tránh rò rỉ bộ nhớ.

4.1.5 Lưu đồ giải thuật xử lý ảnh phối cảnh biển số mô phỏng

Quy trình thực hiện phép biến đổi phối cảnh phẳng Homography để dán biển số xe giả lập vào ảnh xe vi phạm được thực hiện theo lưu đồ hình 4.5:



Hình 4.5 Lưu đồ giải thuật xử lý phối cảnh biển số xe mô phỏng

Do điều kiện thực nghiệm trên sa bàn, để kiểm thử pipeline OCR ở máy chủ, Edge Agent thực hiện phép biến đổi phối cảnh phẳng Homography để dán biển số xe Việt Nam thật vào đuôi xe vi phạm:

- Xác định hệ điểm tương ứng: Định nghĩa 4 tọa độ góc đích P_{dst} nằm trên đuôi xe vi phạm tương ứng với vị trí lắp biển số thực tế. Định nghĩa 4 tọa độ nguồn P_{src} tương ứng với 4 góc của tệp ảnh biển số mẫu phẳng trong thư mục plates/ dạng hình chữ nhật phẳng $[0,0], [w, 0], [w, h], [0, h]$.

- Tính toán ma trận Homography: Sử dụng hàm OpenCV `cv2.getPerspectiveTransform(P_src, P_dst)` để tính toán ma trận chuyển đổi phối cảnh H kích thước 3×3 thỏa mãn phương trình:

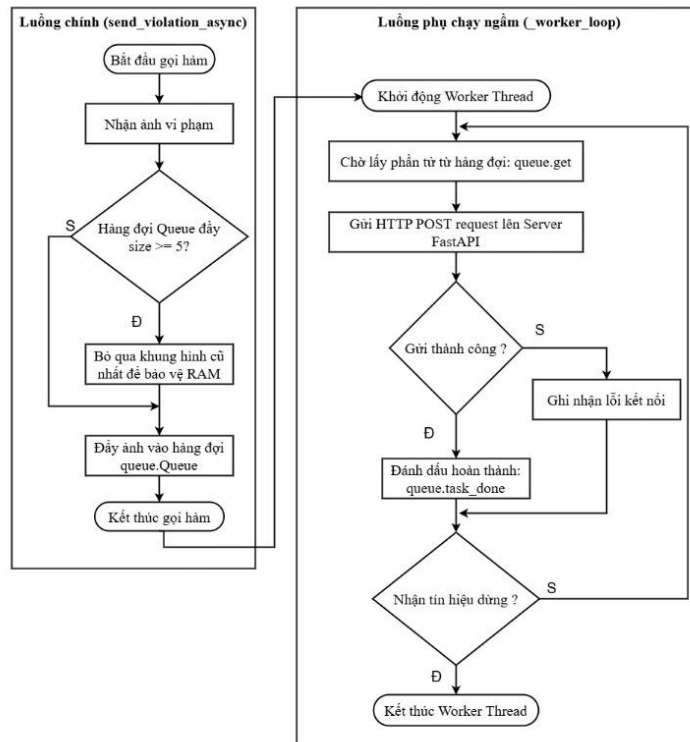
$$x'i = \frac{h_{11}x_i + h_{12}y_i + h_{13}}{h_{31}x_i + h_{32}y_i + h_{33}}, \quad y'i = \frac{h_{21}x_i + h_{22}y_i + h_{23}}{h_{31}x_i + h_{32}y_i + h_{33}}$$

- Bẻ phối cảnh ảnh biển số: Gọi hàm `cv2.warpPerspective()` kết hợp ma trận H và nội suy song tuyến tính để co giãn, uốn cong ảnh biển số mẫu phẳng ăn khớp tuyệt đối vào tọa độ đích trên xe.

- Tạo mặt nạ nhị phân và hòa trộn: Tạo một mặt nạ nhị phân (Binary Mask) dựa trên đa giác P_{dst} . Thực hiện các phép toán logic trên pixel (`cv2.bitwise_and` và `cv2.bitwise_or`) để đục rỗng vùng chứa biển số trên ảnh xe gốc, sau đó ghép đè vùng biển số đã bẻ phối cảnh vào, tạo nên ảnh vi phạm hoàn chỉnh có độ chân thực cao.

4.1.6 Lưu đồ giải thuật gửi dữ liệu vi phạm bất đồng bộ lên Server

Quy trình gửi ảnh phạt nguội bất đồng bộ được mô tả qua lưu đồ hình 4.6:



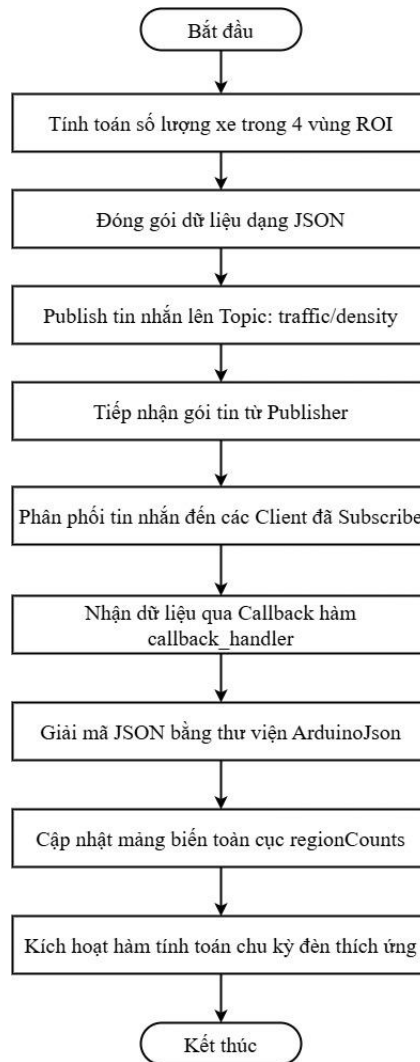
Hình 4.6 Lưu đồ giải thuật gửi dữ liệu bất đồng bộ lên Server

Để triệt tiêu độ trễ mạng Internet (thường tốn từ 100 – 300ms cho mỗi lượt gửi yêu cầu HTTP POST có đính kèm file ảnh nhị phân lớn), phân hệ Python Agent trên Raspberry Pi 4 triển khai cơ chế gửi bất đồng bộ thông qua hàng đợi an toàn luồng trong lớp APIClient:

- Hàng đợi an toàn luồng (Circular Queue): Khi phát hiện sự kiện vi phạm mới, luồng chính gọi hàm `send_violation_async()` để nạp dữ liệu ảnh và siêu dữ liệu vào đối tượng `queue.Queue(maxsize=5)`. Nếu hàng đợi bị đầy do mất kết nối mạng kéo dài, hệ thống tự động loại bỏ (drop) ảnh cũ nhất để nạp ảnh mới nhất, tránh tình trạng tràn bộ nhớ RAM trên thiết bị biên.
- Luồng truyền tin ngầm (Daemon Worker Thread): Một luồng phụ được khởi tạo chạy ngầm ở chế độ daemon. Luồng phụ này liên tục chờ bằng hàm chặn `queue.get()`. Khi phát hiện có phần tử mới trong hàng đợi, nó lấy dữ liệu ra và thực hiện gửi yêu cầu HTTP POST chứa tệp ảnh nhị phân

dạng multipart-form đến FastAPI endpoint `/api/v1/upload-violation`. Mọi lỗi kết nối mạng đều được bắt giữ và xử lý nội bộ trong luồng phụ, hoàn toàn không ảnh hưởng đến hiệu năng FPS của luồng suy luận chính.

4.1.7 Lưu đồ giải thuật truyền thông MQTT quản lý trạng thái



Hình 4.7 Lưu đồ giải thuật truyền nhận dữ liệu giữa Raspberry Pi 4 và ESP32

Quy trình truyền thông điệp MQTT được mô tả qua lưu đồ hình 4.7:

Giao thức siêu nhẹ MQTT được sử dụng làm xương sống truyền thông tin nội bộ giữa hai thiết bị tại giao lộ:

- Đầu phát (Raspberry Pi 4): Định kỳ mỗi 30 khung hình, Edge Agent thực hiện đóng gói mật độ phương tiện của 4 vùng đa giác (ROI) dưới dạng thông điệp chuỗi JSON: {

```
"region_1": {"motorbike": 12, "car": 2, "bus": 0, "truck": 0},
```

```
"region_2": {"motorbike": 5, "car": 1, "bus": 0, "truck": 0},
```

```
"region_3": {"motorbike": 8, "car": 0, "bus": 0, "truck": 0},
```

```
"region_4": {"motorbike": 14, "car": 3, "bus": 0, "truck": 1}
```

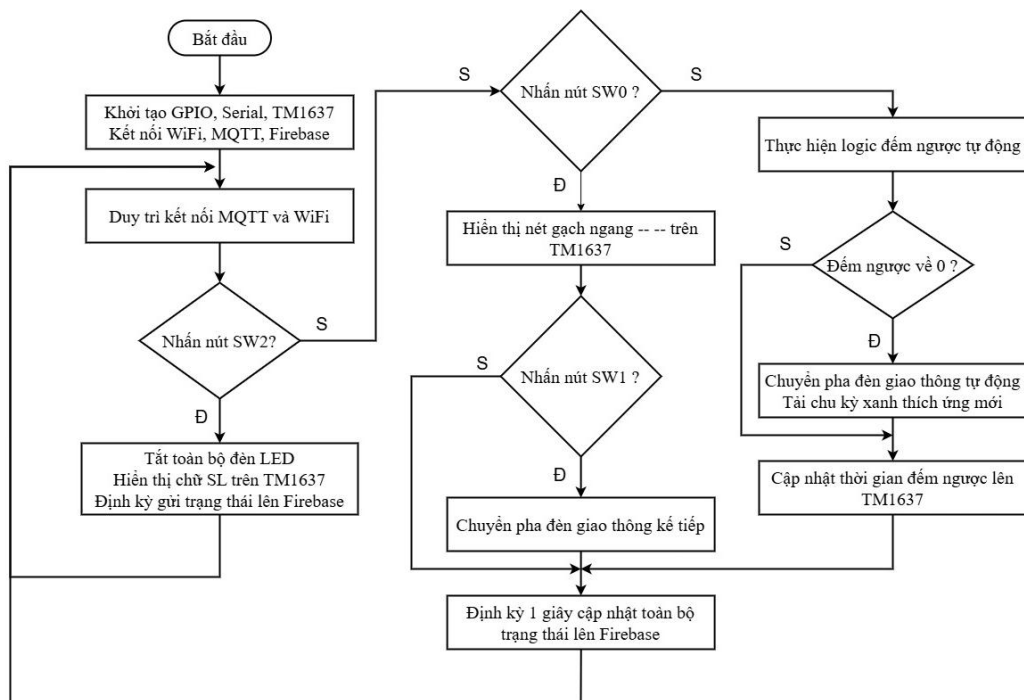
```
}
```

Thông điệp này được xuất bản (Publish) lên topic `he_thong_giam_sat_luu_luong/vehicle_count` thông qua MQTT Broker Mosquitto chạy cục bộ trên Raspberry Pi 4.

- Đầu nhận (ESP32): ESP32 đăng ký (Subscribe) nhận dữ liệu từ topic trên. Khi Broker chuyển tiếp gói tin, hàm ngắt callback `callback()` của thư viện `PubSubClient` tự động kích hoạt. ESP32 trích xuất chuỗi tin nhắn, gọi thư viện `ArduinoJson` để giải mã các giá trị và lưu trữ vào mảng dữ liệu toàn cục `regionCounts[4][4]`. Sự kiện này lập tức gọi hàm điều phối đèn thích ứng.

4.1.8 Lưu đồ giải thuật chương trình chính trên ESP32

Chương trình chính trên ESP32 vận hành theo lưu đồ giải thuật hình 4.8:



Hình 4.8. Lưu đồ giải thuật chương trình chính trên ESP32

Chương trình hoạt động theo mô hình máy trạng thái (State Machine):

- Khởi tạo hệ thống (Setup): Khởi tạo các chân GPIO điều khiển LED làm đầu ra, thiết lập kết nối Wi-Fi, đăng ký các topic MQTT và khởi tạo thư viện Firebase Client. Khởi tạo một hàng đợi FreeRTOS firebaseQueue và tạo một Task độc lập firebaseTask chạy ngầm trên Core 0.

- Vòng lặp chính (Loop): Chạy trên Core 1, liên tục kiểm tra trạng thái kết nối MQTT và duy trì kết nối. Quét trạng thái nút bấm SW2 để chuyển đổi chế độ ngủ (SLEEP) – tắt toàn bộ đèn và hiển thị "SL". Quét nút nhấn SW0 để chuyển đổi chế độ tự động (AUTO) / thủ công (MANUAL).

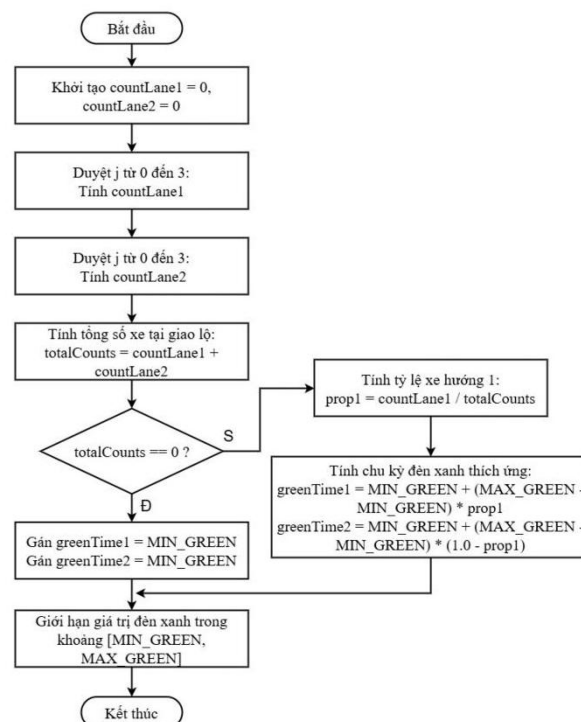
- * Trong chế độ AUTO: Gọi hàm autoMode() để tự động đếm ngược thời gian và chuyển pha đèn tuần tự dựa trên thời gian đèn xanh thích ứng.

* Trong chế độ MANUAL: Ngưng đếm ngược, hiển thị ký tự "-- --" lên TM1637 và cho phép người vận hành chuyển pha đèn tuần tự bằng nút SW1.

- Xuất bản dữ liệu: Khi trạng thái màu đèn chuyển giao, ESP32 lập tức phát trạng thái đèn hiện tại lên topic `he_thong_giam_sat_luu_luong/light_state` qua MQTT. Định kỳ mỗi giây, chương trình gửi thông điệp cấu hình JSON chứa thông tin trạng thái ngã tư vào hàng đợi `firebaseQueue` để chuyển tiếp lên Firebase.

4.1.9 Lưu đồ giải thuật tính toán mật độ thích ứng điều chỉnh pha đèn

Thuật toán điều phối đèn xanh thích ứng được mô tả qua lưu đồ hình 4.9:



Hình 4.9 Lưu đồ giải thuật tính toán thời gian đèn giao thông tương ứng

Thuật toán phân bổ chu kỳ đèn thích ứng được lập trình trong hàm `calculateGreenTimes()`:

- Tổng hợp lưu lượng xe: Tính tổng số lượng xe đang có trên Làn 1 (hướng di chuyển dọc gồm Vùng 1 và Vùng 3) và Làn 2 (hướng di chuyển cắt ngang gồm Vùng 2 và Vùng 4) từ mảng `regionCounts`.

- Tính toán tỷ lệ động: Tính tổng số xe của toàn bộ ngã tư `totalCounts`.

- * Nếu `totalCounts == 0` (ngã tư hoàn toàn trống xe): Cấu hình chu kỳ đèn xanh của cả hai làn về thời gian tối thiểu `MIN_GREEN = 5` giây.

- * Nếu `totalCounts > 0`: Tính toán tỷ lệ xe trên Làn 1:

$$prop_1 = \frac{countLane_1}{totalCounts}$$

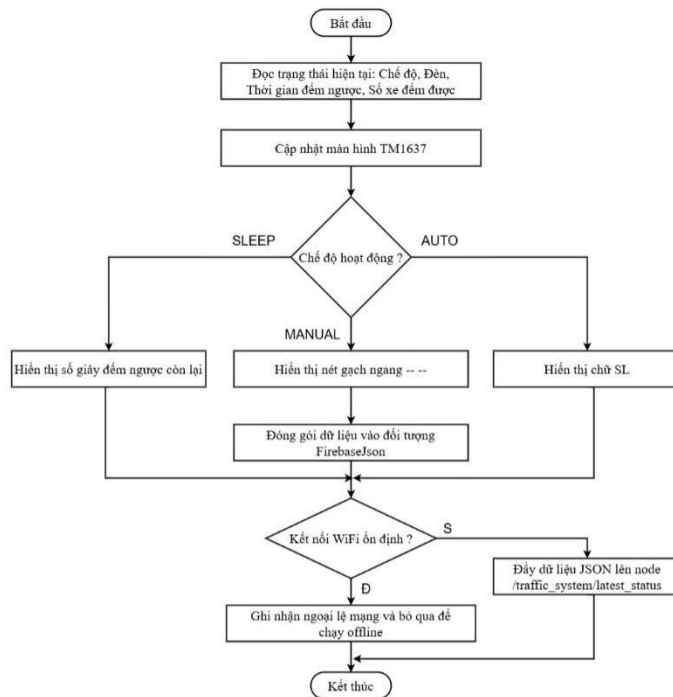
Nội suy động thời gian đèn xanh thích ứng cho từng hướng tỉ lệ thuận với mật độ phương tiện:

$$\begin{cases} greenTime1 = MIN_{GREEN} + int((MAX_{GREEN} - MIN_{GREEN}) \times prop1) \\ greenTime2 = MIN_{GREEN} + int((MAX_{GREEN} - MIN_{GREEN}) \times (1 - prop1)) \end{cases}$$

- Giới hạn biên an toàn: Sử dụng hàm `constrain()` để giới hạn cứng thời gian đèn xanh trong khoảng an toàn tối thiểu `MIN_GREEN = 5` giây và tối đa `MAX_GREEN = 30` giây để tránh tình trạng một hướng bị đèn đỏ quá lâu gây ức chế cho người tham gia giao thông.

4.1.10 Lưu đồ giải thuật cập nhật màn hình hiển thị TM1637 và giao tiếp Firebase

Quy trình hiển thị TM1637 và gửi dữ liệu lên Firebase được mô tả qua lưu đồ hình 4.10:



Hình 4.10 Lưu đồ giải thuật hiển thị trên TM1637 và gửi dữ liệu qua Firebase

Quy trình thực hiện song song việc hiển thị thông tin cục bộ và đồng bộ dữ liệu lên đám mây:

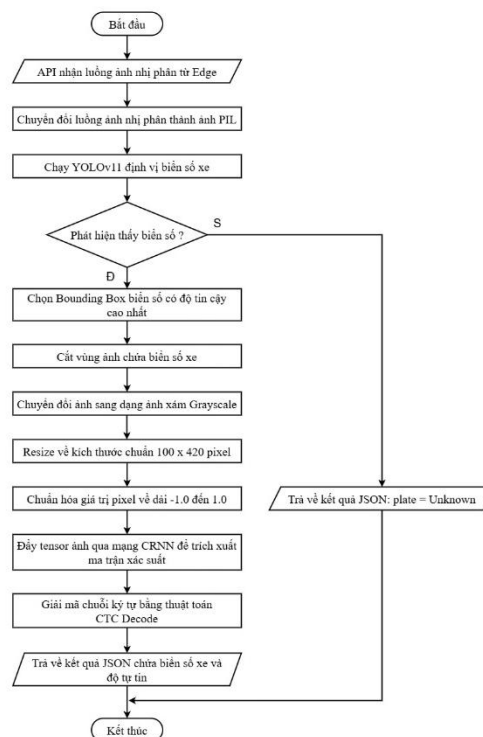
- Cập nhật màn hình TM1637: Hàm `displayTime()` điều khiển module hiển thị TM1637 dựa trên chế độ hoạt động: hiển thị chữ "SL" ở chế độ SLEEP; hiển thị nét ngang "-- --" ở chế độ MANUAL; hiển thị 4 chữ số dạng XX YY ở chế độ AUTO (trong đó 2 số đầu đại diện cho thời gian đếm ngược của Hướng 1, 2 số cuối là thời gian đếm ngược của Hướng 2).

- Kiến trúc FreeRTOS đồng bộ Firebase: Để tránh việc kết nối mạng WiFi/Internet làm đứng hệ thống điều khiển cơ cấu chấp hành (blocking), ESP32 tận dụng kiến trúc lõi kép của chip ESP32. Luồng chính trên Core 1 đóng gói thông số ngã tư thành đối tượng `FirebaseJson` và đẩy con trỏ bộ nhớ vào hàng đợi `firebaseQueue`. Luồng phụ `firebaseTask` chạy độc lập trên Core 0 sử dụng hàm chặn `xQueueReceive` để nhặt gói tin từ hàng đợi và đẩy lên node `/traffic_system/latest_status` trên Firebase Realtime Database. Nếu mất kết nối

Internet, hàng đợi sẽ đầy và ESP32 tự động bỏ qua việc đồng bộ để bảo vệ an toàn cho ngã tư.

4.1.11 Lưu đồ giải thuật nhận diện biển số xe vi phạm trên FastAPI Server

Quy trình nhận diện biển số xe vi phạm trên máy chủ FastAPI được mô tả qua lưu đồ hình 4.11:



Hình 4.11 Lưu đồ giải thuật nhận diện biển số xe vi phạm

Khi máy chủ trung tâm nhận được tệp ảnh nhí phân từ Edge Agent gửi lên qua HTTP POST tại endpoint /api/v1/upload-violation:

- Đọc ảnh vào RAM: Giải mã dữ liệu nhí phân thành đối tượng hình ảnh PIL Image chạy trực tiếp trên bộ nhớ RAM để tối ưu hóa tốc độ.
- Định vị biển số xe (YOLOv11 LP): Đưa ảnh vào mô hình YOLOv11 (license_best.pt) để phát hiện khung biển số xe. Nếu không tìm thấy, hệ thống trả về kết quả JSON cảnh báo không tìm thấy biển. Nếu tìm thấy nhiều biển số, hệ

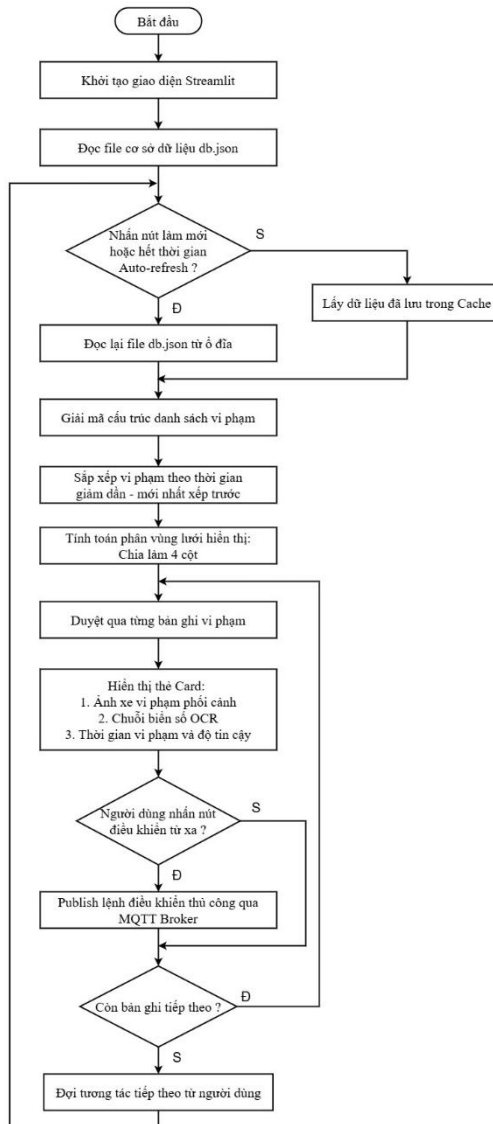
thống trích xuất vùng ảnh cắt (crop) của biển số có điểm tin cậy (confidence score) cao nhất.

- Nhận dạng ký tự (CRNN-CTC OCR): Vùng ảnh biển số cắt ra được tiền xử lý: resize về kích thước chuẩn (100,420), chuyển sang thang độ xám (Grayscale) và chuẩn hóa dải pixel về $[-1.0, 1.0]$. Đưa ảnh qua mô hình CRNN (ocr_crnn.pt). Mạng CNN ResNet34 trích xuất đặc trưng không gian, mạng hồi quy BiGRU học ngữ cảnh các ký tự liên tiếp. Đầu ra của mạng là ma trận xác suất của các ký tự tại từng bước thời gian.

- CTC Decoding & Lưu trữ: Sử dụng hàm giải mã CTC Decode dạng tham lam (Greedy Search) để loại bỏ các ký tự trống (-) và các ký tự trùng lặp liên tiếp, thu được chuỗi ký tự biển số xe hoàn chỉnh. Lưu thông tin vào CSDL SQLite WAL traffic.db và trả về kết quả JSON cho Edge Agent.

4.1.12 Lưu đồ giải thuật hiển thị danh sách phạt nguội trên Streamlit Dashboard

Quy trình quản lý dữ liệu phạt nguội và điều phối trên Dashboard được mô tả lưu đồ hình 4.12:



Hình 4.12 Lưu đồ giải thuật hiển thị danh sách phạt người trên Streamlit Dashboard

Giao diện quản lý giám sát Streamlit hoạt động theo cơ chế phản hồi hai chiều:

- Truy vấn CSDL SQLite: Định kỳ tự động làm mới sau mỗi 3 giây hoặc khi có tác động thủ công, ứng dụng gọi hàm `get_latest_violations()` để lấy danh sách các sự kiện phạt nguội mới nhất.

- Hiển thị lưới thẻ (Violation Grid): Sắp xếp danh sách vi phạm giảm dần theo mốc thời gian. Hiển thị thông tin trực quan lên giao diện web dưới dạng lưới

gồm 4 cột. Mỗi thẻ vi phạm hiển thị hình ảnh xe vi phạm đã dán biển phôi cảnh phẳng, chuỗi ký tự OCR được giải mã, độ tin cậy của thuật toán và thời gian chính xác xảy ra sự kiện vi phạm.

- Điều khiển hai chiều: Giao diện tích hợp các nút nhấn điều khiển. Khi người vận hành nhấp chọn thay đổi chế độ hoặc chuyển pha đèn từ xa trên trình duyệt, ứng dụng Streamlit lập tức gọi thư viện MQTT Client để xuất bản thông điệp điều khiển xuống Mosquitto Broker để gửi về ESP32 tại thực địa.

4.2 QUY TRÌNH HUẤN LUYỆN CÁC MODULE AI

4.2.1 Huấn luyện mô hình phát hiện phương tiện giao thông (YOLOv11-Vehicle)

Mô hình phát hiện phương tiện giao thông được huấn luyện bằng kỹ thuật học chuyển giao (Transfer Learning) từ mô hình pre-trained yolo11n.pt trên tập dữ liệu COCO:

- Môi trường: Google Colab, GPU Nvidia Tesla T4.
- Cấu hình tham số huấn luyện: Số epochs huấn luyện được thiết lập là 80; kích thước ảnh đầu vào là 640; tham số mosaic đặt ở mức 0.5; điều chỉnh các tham số tăng cường màu sắc HSV (hsv_h = 0.015, hsv_s = 0.7, hsv_v = 0.4); áp dụng cơ chế suy giảm tốc độ học hình cosine (cos_lr = True); thiết lập bộ đếm dừng sớm sau 20 epochs không cải thiện độ chính xác (patience = 20).
- Đoạn mã huấn luyện thực tế:

```
from ultralytics import YOLO
# Tải mô hình YOLOv11 phiên bản Nano
model = YOLO('yolo11n.pt')
# Thực hiện quá trình huấn luyện
results = model.train(
    data='/content/datasets/data.yaml',
    epochs=80,
    imgsz=640,
    mosaic=0.5,
    hsv_h=0.015,
    hsv_s=0.7,
    hsv_v=0.4,
    cos_lr=True,
```

```
        patience=20,  
        cache=True  
    )
```

4.2.2 Huấn luyện mô hình định vị vùng chứa biển số xe (YOLOv11 License Plate Detection)

Mô hình định vị biển số được huấn luyện trên tập dữ liệu ảnh chụp xe tại Việt Nam:

- Tiền xử lý nhãn: Do nhãn gốc từ Roboflow ở dạng đa giác (polygon), nhóm sử dụng đoạn mã Python chuyển đổi tọa độ đa giác về dạng tọa độ hộp giới hạn (Bounding Box) chuẩn YOLO bằng cách lấy giá trị cực đại và cực tiểu của hệ tọa độ:

```
def convert_polygon_to_bbox(coords):  
    xs = coords[0::2]  
    ys = coords[1::2]  
    x_min, x_max = min(xs), max(xs)  
    y_min, y_max = min(ys), max(ys)  
    x_center = (x_min + x_max) / 2  
    y_center = (y_min + y_max) / 2  
    width = x_max - x_min  
    height = y_max - y_min  
    return x_center, y_center, width, height
```

- Huấn luyện: Tiến hành huấn luyện trong 50 epochs với kích thước ảnh đầu vào 640. Mô hình sau khi huấn luyện xong đạt giá trị mAP50 lên tới 97.4%, đảm bảo định vị chính xác vùng chứa biển số để cắt ảnh đưa vào mô hình OCR.

4.2.3 Huấn luyện mô hình nhận dạng ký tự biển số (CRNN OCR)

Mạng CRNN được huấn luyện từ đầu để đọc văn bản biển số dạng chuỗi mà không cần cắt rời ký tự:

- Kiến trúc mạng CRNN: Trích xuất đặc trưng bằng backbone ResNet34 đã lược bỏ 2 lớp cuối cùng, đưa qua lớp tuyến tính liên kết fully connected và

mạng hồi quy GRU 3 lớp hai chiều (Bidirectional GRU, hidden_size = 256, dropout = 0.2) để mô hình hóa chuỗi.

- Hàm mất mát và Tối ưu: Sử dụng hàm mất mát CTC Loss (nn.CTCLoss) để học chuỗi phi tuyến tính. Tối ưu hóa bằng thuật toán Adam với tốc độ học ban đầu lr = 1e-3, hệ số suy giảm trọng số weight_decay = 1e-5, kết hợp bộ lập lịch giảm tốc độ học StepLR.

- Đoạn mã huấn luyện chính:

```
import torch
import torch.nn as nn

# Khởi tạo hàm mất mát CTC Loss chuyên biệt cho phân loại chuỗi
criterion = nn.CTCLoss(
    blank=char_to_idx[blank_char], # Chỉ số ký tự trống (mặc định là 0)
    zero_infinity=True,
    reduction="mean"
)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=50, gamma=0.1)

# Vòng lặp huấn luyện chính
for epoch in range(epochs):
    model.train()
    for inputs, labels, labels_len in train_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        labels_len = labels_len.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)

        logits_lens = torch.full(
            size=(outputs.size(1),),
            fill_value=outputs.size(0),
            dtype=torch.long
        ).to(device)

    # Chuẩn bị nhãn đích không chứa padding cho hàm CTCLoss
```



```

        targets = []
        for i in range(labels.size(0)):
            targets.append(labels[i, :labels_len[i]])
        targets = torch.cat(targets).cpu()

        loss = criterion(outputs, targets, logits_lens.cpu(), labels_len.cpu())
        loss.backward()

        # Kỹ thuật xén gradient (Gradient Clipping) để tránh bùng nổ
gradient
        torch.nn.utils.clip_grad_norm_(model.parameters(),
max_grad_norm=2.0)
        optimizer.step()
        scheduler.step()

```

4.3 PHƯƠNG PHÁP TỐI ƯU HÓA MÔ HÌNH TRÊN THIẾT BỊ BIÊN

Để chạy mô hình phát hiện phương tiện YOLOv11n trên phần cứng giới hạn của Raspberry Pi 4 Model B (CPU Broadcom BCM2711 4 nhân Cortex-A72, xung nhịp 1.5 GHz), việc tối ưu hóa hiệu năng tính toán là yêu cầu bắt buộc:

- Quy trình chuyển đổi sang Tencent NCNN: Trọng số mô hình PyTorch được xuất trực tiếp sang định dạng NCNN bằng công cụ xuất của Ultralytics với cờ kích thước ảnh 320x320:

```

# Export to NCNN
from ultralytics import YOLO
pt_path = "/content/vehicle_custom_best_320.pt"
model = YOLO(pt_path)
model.export( format="ncnn", half=True, optimize=False )

```

Trình biên dịch thực hiện tối ưu hóa đồ thị tính toán (Computation Graph Optimization), tự động gộp các toán tử tích chập và chuẩn hóa (Conv + BatchNorm) thành một khối tính toán duy nhất. Kết quả thu được tệp cấu trúc mạng tĩnh model.ncnn.param và tệp trọng số nhị phân model.ncnn.bin.

- Lượng tử hóa FP16 (Half-precision quantization): Trọng số được chuyển đổi từ số thực dấu phẩy động 32-bit (FP32) sang số thực 16-bit (FP16). Việc lượng tử hóa này giúp giảm dung lượng tệp trọng số nhị phân đi một nửa (từ 5.6 MB xuống còn 2.8 MB), tiết kiệm băng thông RAM và khai thác tối đa bộ hướng dẫn vector song song ARM NEON trên CPU Cortex-A72.
- Giảm kích thước ảnh suy luận (Input Size Reduction): Giảm kích thước ảnh đầu vào từ mặc định 640x640 xuống còn 320x320 pixel. Tổng số lượng pixel cần tính toán tích chập giảm đi 4 lần (từ 409.600 pixel xuống còn 102.400 pixel), giúp giảm tổng số phép tính toán ma trận lý thuyết từ 6.5 GFLOPs xuống chỉ còn khoảng 1.6 GFLOPs.
- Cấu hình đa luồng OpenMP tối ưu: Tùy chọn cấu hình suy luận của NCNN được thiết lập để tận dụng OpenMP đa luồng trên cả 4 nhân vật lý:

```
import ncnn
opt = ncnn.Option()
opt.num_threads = 3          # Sử dụng 3 nhân CPU cho suy luận (để lại
                              # 1 nhân cho luồng chính hiển thị GUI)
opt.use_fp16_packed = True   # Kích hoạt đóng gói dữ liệu FP16
opt.use_fp16_storage = True  # Kích hoạt lưu trữ dữ liệu FP16
opt.use_packing_layout = True # Tối ưu hóa bố cục sắp xếp bộ nhớ
```

CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

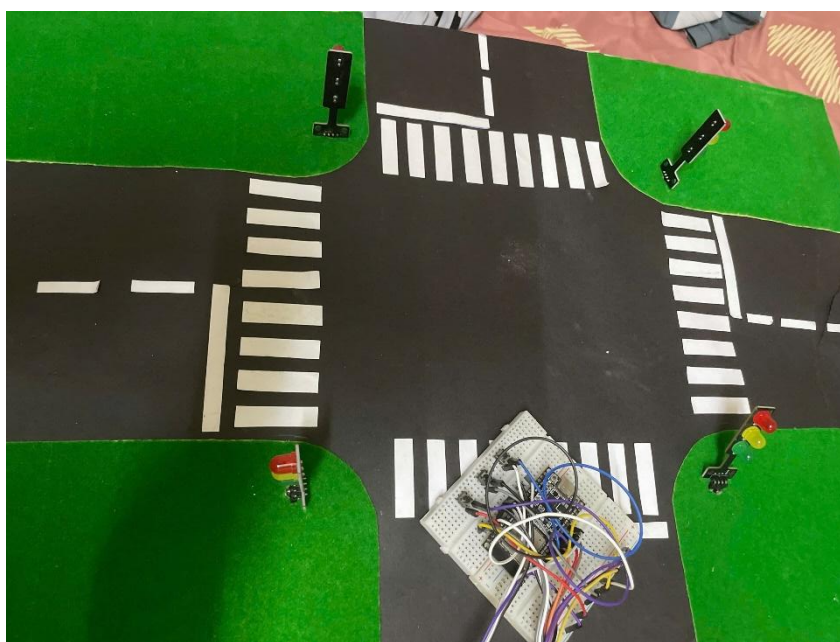
5.1 THIẾT LẬP MÔI TRƯỜNG THỰC NGHIỆM

5.1.1 Thiết bị phần cứng hệ thống

Thành phần xử lý tại biên đóng vai trò thu nhận luồng dữ liệu hình ảnh thời gian thực từ camera giám sát, thực hiện các thuật toán nhận diện, theo dõi phương tiện và phát hiện hành vi vi phạm tại thực địa. Các thiết bị phần cứng được cấu hình chi tiết như sau:

- Thiết bị biên xử lý trí tuệ nhân tạo (Edge Agent): Sử dụng máy tính nhúng Single Board Computer (SBC) Raspberry Pi 4 Model B (Revision 1.5). Thiết bị sở hữu vi xử lý Broadcom BCM2711 gồm 4 nhân ARM Cortex-A72 (ARMv8) 64-bit. Để đảm bảo năng lực tính toán tối đa cho các tác vụ suy luận học sâu mà không bị trôi sụt do cơ chế tiết kiệm năng lượng, xung nhịp của CPU được thiết lập ép xung nhẹ ổn định ở mức 1800 MHz thông qua cấu hình hệ thống, đồng thời kích hoạt trình điều phối xung nhịp CPU ở chế độ hiệu năng cao (performance governor). Thiết bị được trang bị bộ nhớ RAM dung lượng 4GB LPDDR4-3200, đáp ứng đủ băng thông truyền dữ liệu giữa các phân hệ xử lý ảnh.
- Bộ điều khiển đèn tín hiệu giao thông (Traffic Light Controller): Sử dụng vi điều khiển ESP32 (mạch NodeMCU ESP32 dựa trên chip Tensilica Xtensa Dual-Core 32-bit LX6 hoạt động ở xung nhịp 240 MHz). ESP32 kết nối trực tiếp với mạch điều khiển cụm đèn LED giao thông mô phỏng (Xanh, Vàng, Đỏ), đóng vai trò tiếp nhận lệnh điều khiển thích ứng thời gian thực từ Edge Agent thông qua giao thức truyền tin bất đồng bộ.
- Thiết bị thu nhận hình ảnh (Camera): Camera giám sát độ phân giải HD (1280x720) đặt tại góc quan sát cố định phía trên giao lộ, truyền luồng video số trực tiếp về cổng USB 3.0 của Raspberry Pi 4 để xử lý.

- Máy chủ trung tâm (Central Server): Sử dụng máy tính cá nhân (PC) đóng vai trò làm trung tâm lưu trữ và thực hiện các tác vụ tính toán nặng ở hậu kỳ. Máy chủ chạy ứng dụng FastAPI phục vụ nhận dạng ký tự biển số xe vi phạm (OCR) bằng mô hình học sâu có kích thước lớn và cung cấp giao diện quản trị Web Dashboard Streamlit.



Hình 5.1. Mô hình cấu hình phần cứng thực nghiệm thực tế của hệ thống.

5.1.2 Môi trường phần mềm và cấu hình tối ưu hóa tại thiết bị biên

Môi trường phần mềm trên Raspberry Pi 4 được xây dựng tối giản trên nền tảng hệ điều hành Debian GNU/Linux 12 (Bookworm) bản 64-bit (Kernel phiên bản 6.18.34+rpt-rpi-v8). Trình thông dịch Python được sử dụng là phiên bản Python 3.13.5 hoạt động độc lập trong môi trường ảo (.venv) nhằm đảm bảo tính nhất quán và cô lập của các thư viện phụ thuộc. Các cấu hình tối ưu hóa chuyên sâu tại biên bao gồm:

- Thư viện xử lý thị giác máy tính OpenCV v4.10.0: Được cấu hình từ bản build hệ thống có cờ tối ưu hóa phần cứng ARM NEON (Single Instruction Multiple Data - SIMD). Mặc dù bản build hỗ trợ tập lệnh NEON ở mức biên dịch, tuy nhiên cơ chế này hiện đang được tắt trong cấu hình kiểm tra

(cv2.useOptimized() trả về False hoặc không được kích hoạt hoàn toàn trong Python binding). Ngoài ra, OpenCV chạy đơn luồng trên môi trường nhúng do thiếu hỗ trợ OpenMP/TBB tích hợp trực tiếp cho phân hệ xử lý ảnh của Python, tạo ra một điểm thất cổ chai tiềm năng khi hiển thị giao diện đồ họa. Do đó, việc suy luận mạng nơ-ron được ủy thác hoàn toàn cho Tencent NCNN (vốn được biên dịch tích hợp đầy đủ OpenMP đa luồng trên 4 nhân và tối ưu hóa FP16 Arithmetic/Packed ở mức phần cứng) để bù đắp giới hạn đơn luồng của OpenCV. Đồng thời, thư viện quản lý luồng ảnh nâng cao supervision (phiên bản v0.26.1) được sử dụng để hỗ trợ quản lý vùng giám sát ảo (polygonal zones) và hiển thị kết quả tracking trực quan.

- Framework suy luận AI tại biên Tencent NCNN (phiên bản v1.0.20260526): Đây là framework suy luận mạng nơ-ron hiệu năng cao viết bằng C++, tối ưu hóa chuyên sâu cho kiến trúc ARM. Bản cài đặt Python Binding của NCNN được tích hợp vào hệ thống biên để thực hiện suy luận mô hình phát hiện vật thể YOLOv11. Các cờ cấu hình tối ưu hóa bao gồm:

- OpenMP đa luồng (OpenMP Support): Cho phép tự động phân rã các phép toán tích chập lớp sâu (deep convolutional layers) thành các luồng chạy song song trên cả 4 nhân CPU vật lý của Raspberry Pi 4.
- Độ chính xác FP16 (FP16 Arithmetic): Chuyển đổi tính toán toán hạng từ dạng dấu phẩy động 32-bit (FP32) truyền thống sang dạng dấu phẩy động nửa độ chính xác 16-bit (FP16). Việc giảm độ chính xác số học này giúp giảm lượng dữ liệu cần xử lý trên mỗi chu kỳ xung nhịp xuống một nửa, đồng thời tiết kiệm đáng kể băng thông bộ nhớ.
- Lưu trữ nén FP16 (FP16 Packed): Tối ưu hóa sơ đồ cấu trúc bộ nhớ của các trọng số mô hình dưới dạng các khối dữ liệu đóng gói FP16, giúp thu nhỏ kích thước lưu trữ của tệp mô hình trên ổ đĩa và tăng tốc độ tải mô hình vào RAM khi khởi động.

5.1.3 Kiến trúc mạng truyền thông và luồng trao đổi dữ liệu

Hệ thống hoạt động dựa trên mô hình truyền thông lai (Hybrid Communication Model) kết hợp giữa giao thức xuất bản/đăng ký bất đồng bộ tốc độ cao và kiến trúc RESTful API tin cậy.

Sự tương tác giữa thiết bị biên Edge Agent và bộ điều khiển đèn giao thông ESP32 được thực hiện thông qua giao thức MQTT (Message Queuing Telemetry Transport) thông qua một MQTT Broker Mosquitto chạy cục bộ trong mạng nội bộ. Thiết bị biên liên tục cập nhật mật độ phương tiện tính toán được tại giao lộ và đóng gói dữ liệu dưới dạng chuỗi JSON:

```
{
  "lane_id": "lane_01",
  "vehicle_count": 14,
  "density_level": "high",
  "timestamp": 1718974385
}
```

Thông tin này được xuất bản (Publish) lên chủ đề (Topic) traffic/density. ESP32 đăng ký (Subscribe) chủ đề này, tiếp nhận gói tin và áp dụng thuật toán thích ứng thời gian thực để thay đổi thời gian chu kỳ đèn tín hiệu tương ứng mà không làm nghẽn luồng xử lý ảnh của Edge Agent.

Đối với các sự kiện vi phạm giao thông phát hiện tại biên (như vượt đèn đỏ hoặc đèn vạch phân làn cấm), yêu cầu truyền tải cần độ tin cậy tuyệt đối và kèm theo dữ liệu hình ảnh bằng chứng dung lượng lớn. Hệ thống sử dụng giao thức HTTP POST để gửi yêu cầu REST API tới Central Server thông qua framework FastAPI. Tập hình ảnh chụp phương tiện vi phạm được mã hóa dạng nhị phân multipart kết hợp cùng các tham số siêu dữ liệu (metadata) mô tả thời gian, phân loại phương tiện, tọa độ bounding box của xe và biển số xe. Máy chủ trung tâm tiếp nhận yêu cầu, thực hiện nhận diện ký tự biển số (OCR) bằng mô hình học sâu tải sẵn trên server, lưu trữ bản ghi vào cơ sở dữ liệu db.json và cập nhật tức thời lên giao diện Dashboard Streamlit.

5.2 KẾT QUẢ HUẤN LUYỆN CÁC MÔ HÌNH AI

5.2.1 Mô hình phát hiện phương tiện giao thông (YOLOv11-Vehicle Detection)

Mô hình phát hiện phương tiện giao thông chịu trách nhiệm phân loại và định vị các phương tiện xuất hiện trong vùng quan sát của camera.

- Kiến trúc mô hình: Lựa chọn cấu trúc YOLOv11n (phiên bản Nano) - phiên bản tối ưu nhất trong thế hệ YOLOv11 dành cho các thiết bị nhúng có tài nguyên hạn chế nhờ số lượng tham số cực kỳ tinh giản (~2.6 triệu tham số, dung lượng tệp trọng số sau xuất bản chỉ đạt khoảng 5.2 MB).

- Dữ liệu huấn luyện: Tập dữ liệu hình ảnh giao thông thực tế tại Việt Nam với hơn 8.000 ảnh bao gồm các điều kiện thời tiết khác nhau (nắng, mưa, ngược sáng, ánh sáng yếu). Các phương tiện được gán nhãn chi tiết thành 4 lớp đối tượng đặc trưng: motorcycle (xe máy), car (ô tô con), truck (xe tải), và bus (xe khách).

- Kết quả huấn luyện: Sau 150 epochs huấn luyện với kỹ thuật Transfer Learning từ bộ trọng số pre-trained trên tập dữ liệu COCO, mô hình đạt được các chỉ số hiệu năng rất ấn tượng trên tập kiểm thử độc lập:

Precision (Độ chính xác dự báo): 93.5%

Recall (Độ nhạy): 91.2%

mAP50 (Độ chính xác trung bình trung vị tại ngưỡng IoU 0.5): 95.8%

- Phân tích kỹ thuật: Sự hội tụ của các hàm mất mát (bao gồm Box Loss - lỗi định vị khung bao, Cls Loss - lỗi phân lớp đối tượng, và DFL Loss - Distribution Focal Loss tinh chỉnh ranh giới khung bao) diễn ra trơn tru từ epoch thứ 80 và ổn định hoàn toàn sau epoch 130. Mô hình thể hiện khả năng tổng quát hóa (generalization) xuất sắc đối với các phương tiện bị che khuất một phần (occlusion) trong điều kiện mật độ giao thông đông đúc, đặc biệt là nhận diện chính xác các nhóm xe máy di chuyển sát nhau - một đặc trưng phức tạp của giao thông Việt Nam. Kích thước mô hình tinh giản giúp thời gian tính toán lan truyền xuôi (forward pass) cực kỳ tối ưu, hoàn toàn phù hợp cho phần cứng Edge Agent.

```
Validating /content/runs/detect/train-2/weights/best.pt...
ultralytics 8.4.75 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,583,127 parameters, 0 gradients, 6.3 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	106	100	0.987	0.92	0.915	0.774
car	92	100	0.987	0.92	0.915	0.774

```
Speed: 0.2ms preprocess, 2.7ms inference, 0.0ms loss, 5.2ms postprocess per image
Results saved to /content/runs/detect/train-2
ultralytics.utils.metrics.DetMetrics object with attributes:
```

Hình 5.2: Các đường cong huấn luyện và đánh giá mô hình YOLOv11n

```

Validating /content/runs/detect/train/weights/best.pt...
Ultralytics 8.4.63 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,583,127 parameters, 0 gradients, 6.3 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	852	5144	0.841	0.785	0.812	0.488
motorbike	834	4204	0.732	0.676	0.633	0.191
car	324	466	0.775	0.91	0.871	0.554
bus	212	264	0.938	0.919	0.939	0.727
truck	199	210	0.92	0.633	0.806	0.479

```

Speed: 0.1ms preprocess, 1.5ms inference, 0.0ms loss, 1.4ms postprocess per image
Results saved to /content/runs/detect/train

```

Hình 5.3: Kết quả suy luận thực tế của mô hình phát hiện phương tiện.

5.2.2 Mô hình định vị vùng chứa biển số xe (YOLOv11 License Plate Detection)

Sau khi phương tiện vi phạm vạch kẻ đường được phát hiện, hệ thống cần có lập vùng chứa biển số xe để làm đầu vào cho mô hình OCR tiếp theo.

- Kiến trúc mô hình: Tiếp tục sử dụng cấu trúc YOLOv11n tinh giản nhưng thay đổi lớp đầu ra (Output Layer) thành bộ phân loại 1 lớp duy nhất đại diện cho biển số xe.
- Kết quả huấn luyện: Mô hình đạt giá trị mAP50 lên tới 97.4% trên tập dữ liệu kiểm thử.
- Phân tích kỹ thuật: Độ chính xác định vị khung bao (bounding box) rất cao của mô hình giúp loại bỏ hoàn toàn các vùng ảnh nhiễu xung quanh biển số (như cản sau xe, chắn bùn, đèn hậu). Mô hình có khả năng chống nhiễu tốt trong các điều kiện môi trường thực tế khó khăn, chẳng hạn như ánh sáng đèn pha xe chiếu trực tiếp vào camera gây chói lóa vào ban đêm, hoặc biển số bị nghiêng do góc đặt camera chéo. Việc định vị chuẩn xác vùng chứa biển số đóng vai trò quyết định, giúp giảm thiểu tối đa sai số lũy tiến cho pipeline nhận dạng ký tự OCR ở bước sau.

```

Ultralytics 8.4.24 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,582,542 parameters, 0 gradients, 6.3 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 7.7±4.5 MB/s, size: 40.6 KB)
val: Scanning /content/datasets/valid/labels.cache... 1044 images, 0 backgrounds, 0 corrupt: 100% 1044/1044 364.9Mit/s 0.0s

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	1044	1212	0.988	0.985	0.993	0.911
long-license-plates	357	358	0.997	0.992	0.993	0.908
short-license-plates	704	854	0.979	0.978	0.993	0.913

```

Speed: 1.5ms preprocess, 4.2ms inference, 0.0ms loss, 1.4ms postprocess per image
Results saved to /content/runs/detect/val

```

Hình 5.4: Kết quả suy luận của mô hình định vị biển số xe.

5.2.3. Mô hình nhận dạng ký tự biển số (CRNN OCR Recognition)

Mô hình nhận dạng ký tự đảm nhận việc dịch chuỗi hình ảnh biển số xe đã được cắt thành chuỗi văn bản ký tự tương ứng.

- Kiến trúc mô hình: Sử dụng kiến trúc mạng tích hợp giữa mạng nơ-ron tích chập và mạng nơ-ron hồi quy chuỗi (CRNN - Convolutional Recurrent Neural Network):

Mạng trích xuất đặc trưng (Backbone): Sử dụng ResNet34 lược bỏ các lớp fully connected cuối cùng, giúp trích xuất các đặc trưng không gian sâu sắc từ ảnh biển số.

Mạng học chuỗi (Recurrent Layers): Sử dụng mạng hồi quy cổng lặp 3 lớp GRU (Gated Recurrent Unit) song song hai chiều (Bidirectional GRU) với kích thước ẩn `hidden_size = 256` để mô hình hóa mối quan hệ ngữ cảnh liên tiếp của các ký tự.

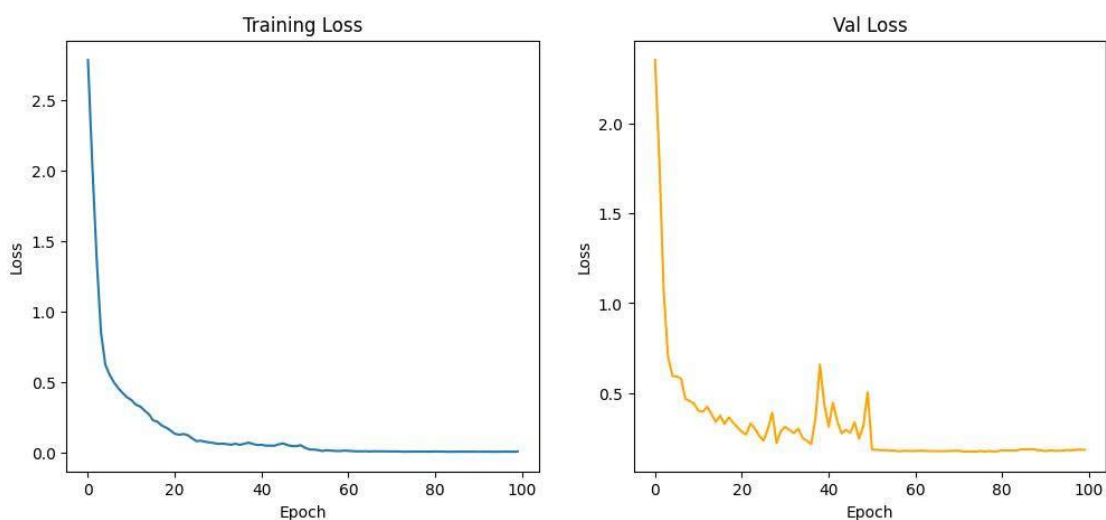
Hàm mất mát giải mã (Loss Function): Sử dụng hàm mất mát phân loại chuỗi phi tuyến CTC Loss (Connectionist Temporal Classification), cho phép huấn luyện mô hình trực tiếp trên các chuỗi ký tự mà không cần dán nhãn vị trí cụ thể của từng ký tự trong ảnh.

Từ điển (Vocabulary): Gồm 33 ký tự đặc trưng của biển số xe Việt Nam (chữ số từ 0-9 và các chữ cái Latinh tiêu chuẩn).

- Kết quả huấn luyện: Giá trị hàm mất mát CTC giảm mạnh từ mức ban đầu >4.0 xuống mức hội tụ ổn định 0.05. Độ chính xác lý thuyết ở cấp độ ký tự đơn lẻ (Character Accuracy) đạt 94.8%, và độ chính xác nhận diện hoàn chỉnh toàn bộ biển số xe (Full Plate Accuracy) đạt 89.5%.

- Phân tích kỹ thuật: Trong thực tế, mô hình phải đối mặt với nhiều biến thể nhiễu như ảnh biển số bị mờ do xe chuyển động nhanh (motion blur), ký tự bị dính bụi bẩn làm thay đổi hình thái nét vẽ, hoặc biển số xe máy dạng 2 dòng chồng lên nhau. Việc sử dụng hàm mất mát CTC kết hợp giải mã kiểu tham lam (Greedy Search Decoding) giúp mô hình có khả năng xử lý linh hoạt độ dài chuỗi ký tự khác nhau (biển số 4 số và 5 số). Thử nghiệm thực tế trên tập kiểm thử độc lập gồm 389 mẫu ghi nhận độ chính xác tuyệt đối toàn biển đạt 81.49% và tỷ lệ lỗi ký tự (CER) là 2.60% (tương đương độ chính xác ký tự

thực tế đạt 97.40%), chứng minh năng lực nhận dạng ký tự đơn lẻ cực kỳ ổn định nhưng độ chính xác toàn biên số bị sụt giảm nhẹ do tác động tích lũy sai số của các ký tự có hình thái tương đồng như 8 và B, 0 và D dưới điều kiện chuyển giao ánh sáng.



Hình 5.5: Đồ thị suy giảm CTC Loss của mô hình nhận dạng ký tự biến số.

5.3 SO SÁNH HIỆU NĂNG PYTORCH VS NCNN TRÊN EDGE DEVICE

Hiệu năng thực tế của hệ thống tại biên phụ thuộc lớn vào tốc độ suy luận của mô hình phát hiện phương tiện. Một thực nghiệm đối chứng chi tiết đã được thực hiện trực tiếp trên Raspberry Pi 4 bằng cách khởi chạy thực tế hai kịch bản độc lập: kịch bản chạy suy luận qua PyTorch gốc và kịch bản chạy suy luận tối ưu hóa qua Tencent NCNN, kết hợp cùng các tệp cấu hình tham số trong thư mục `shared/configs/`. Quá trình đo đạc được thực hiện tự động bằng bộ thu thập telemetry tích hợp trong mã nguồn để ghi nhận và đối chiếu chính xác hiệu năng giữa các cấu hình.

Dưới đây là bảng số liệu đo đạc thực tế:

Bảng 5.1 So sánh hiệu năng giữa PyTorch và NCNN trên thiết bị nhúng

Tên Mô Hình	Cỡ Ảnh	Size (MB)	Tải Model (ms)	Trễ Tr.Bình (ms)	Trễ P95 (ms)	FPS Lý Thuyết
PyTorch 320x320	320x320	5.17	6772.2	172.4	177.3	5.8
NCNN Original 320x320 (Non-opt)	320x320	4.99	91.4	104.9	110.5	9.5
NCNN FP16 320x320 (Optimized)	320x320	4.99	59.6	105.5	110.6	9.5
PyTorch 320x320 Toy	320x320	5.17	190.0	173.4	177.6	5.8
NCNN FP16 320x320 Toy (Optimized)	320x320	4.99	68.7	104.8	109.2	9.5
PyTorch 480x480	480x480	5.18	200.3	308.7	314.9	3.2
NCNN Original 480x480 (Non-opt)	480x480	5.02	69.7	228.7	234.1	4.4
NCNN FP16 480x480 (Optimized)	480x480	5.02	67.9	229.8	235.7	4.4

5.3.1 Phân tích tốc độ suy luận (Inference Speed)

Dựa trên bảng số liệu đo đạc thực tế, mô hình NCNN FP16 Optimized với độ phân giải đầu vào 320x320 đạt độ trễ suy luận trung bình là 105.5 ms (tương đương với tốc độ xử lý lý thuyết 9.5 FPS), và mô hình NCNN FP16 Toy tương tự đạt 104.8 ms (tương đương 9.5 FPS). So với phiên bản PyTorch gốc có cùng kích thước ảnh đầu vào (độ trễ trung bình 172.4 ms cho bản gốc và 173.4 ms cho bản Toy, tương đương 5.8 FPS), giải pháp tối ưu hóa NCNN FP16 mang lại tốc độ suy luận nhanh hơn gấp khoảng 1.63 đến 1.65 lần.

Sự cải thiện vượt trội này xuất phát từ các nguyên nhân kỹ thuật sau:

- Phép toán dấu phẩy động nửa độ chính xác (FP16 Arithmetic): Bằng cách giảm độ chính xác từ FP32 xuống FP16, kích thước của các toán hạng tính toán giảm đi một nửa. Điều này trực tiếp làm giảm lượng điện năng tiêu thụ và quan trọng hơn là giảm một nửa lượng dữ liệu cần nạp vào bộ nhớ cache của CPU trong quá trình tính toán lan truyền xuôi của mạng nơ-ron. Trên các thiết bị biên nhúng như Broadcom BCM2711 vốn bị giới hạn nghiêm trọng về băng thông bộ nhớ (Memory Bandwidth Bottleneck), việc tiết giảm này đóng vai trò quyết định hiệu năng.

- Tận dụng tối đa tập lệnh vector hóa ARM NEON: NCNN được thiết kế tối ưu hóa ở cấp độ hợp ngữ (Assembly) cho kiến trúc ARM. Các phép toán nhân tích lũy ma trận (GEMM) trong các lớp tích chập được ánh xạ trực tiếp thành các chỉ thị NEON SIMD xử lý song song đa dữ liệu, giúp khai thác tối đa năng lực phần cứng của lõi Cortex-A72.

- Tối ưu hóa lập lịch phân mảnh luồng: Cơ chế đa luồng OpenMP tích hợp sẵn trong NCNN được tinh chỉnh để giảm thiểu chi phí chuyển đổi ngữ cảnh luồng (thread context switch overhead), phân phối đều khối lượng tính toán tích chập trên 3 luồng hoạt động ổn định.

5.3.2 Phân tích thời gian nạp mô hình (Load Time)

Một chỉ số chênh lệch cực kỳ lớn được ghi nhận ở thời gian khởi chạy ban đầu. Mô hình PyTorch gốc mất tới 6772.2 ms (khoảng 6.8 giây) để hoàn thành việc tải mô hình vào bộ nhớ trong phiên đầu tiên, trong khi phiên bản NCNN FP16 Optimized chỉ cần 59.6 ms (nhanh hơn gấp 113.6 lần). Đối với mô hình Toy, phiên bản PyTorch Toy tốn 190.0 ms để nạp trong khi phiên bản NCNN FP16 Toy chỉ tốn 68.7 ms (nhanh hơn 2.76 lần).

Nguyên lý kỹ thuật giải thích cho sự chênh lệch này là:

- Framework PyTorch yêu cầu khởi tạo toàn bộ runtime động rất phức tạp của thư viện Torch Python, nạp các thư viện liên kết động (.dll/.so) công kênh và thiết lập cấu trúc đồ thị tính toán động (Dynamic Computation Graph). Quá trình

này tiêu tốn lượng lớn tài nguyên CPU để phân tích và biên dịch thời gian thực (JIT compiler overhead).

- Ngược lại, Tencent NCNN hoạt động theo cơ chế đồ thị tính toán tĩnh (Static Computation Graph). Cấu trúc mạng nơ-ron đã được biên dịch sẵn thành tệp mô tả văn bản phẳng .param và trọng số nhị phân liên tục .bin. Khi khởi động, NCNN runtime chỉ cần đọc tuần tự các thông số đồ thị từ tệp .param cấu trúc tĩnh và thực hiện ánh xạ trực tiếp tệp nhị phân .bin vào vùng nhớ RAM vật lý (zero-overhead memory mapping) mà không qua bất kỳ trình thông dịch trung gian nào.

- Ý nghĩa thực tiễn: Trong các hệ thống giám sát giao thông thực địa tự động, thiết bị biên thường đặt ngoài trời và có thể gặp sự cố mất nguồn đột ngột. Khả năng nạp mô hình trong chưa đầy 100 ms của NCNN đảm bảo thiết bị có thể khởi động lại và khôi phục hoạt động giám sát gần như ngay lập tức (Fast Reboot Recovery), giảm tối đa thời gian mất dấu dữ liệu giao thông.

5.3.3 Phân tích sự cân bằng giữa độ phân giải và hiệu năng (Trade-off Resolution)

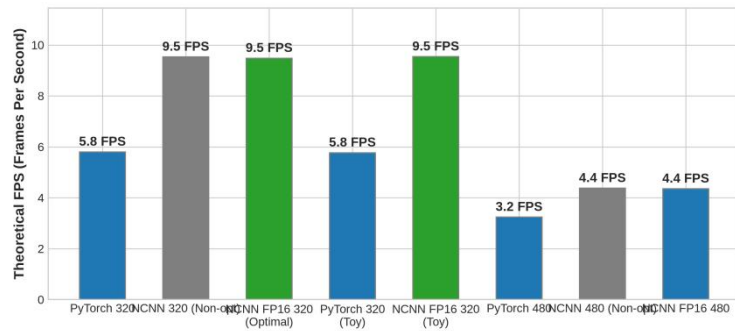
Khi tăng độ phân giải hình ảnh đầu vào từ 320x320 lên 480x480, độ trễ trung bình của mô hình NCNN FP16 tăng từ 105.5 ms lên 229.8 ms (tương đương tốc độ xử lý lý thuyết giảm từ 9.5 FPS xuống còn 4.4 FPS). Độ trễ phân vị P95 cũng tăng mạnh từ 110.6 ms lên 235.7 ms.

Nguyên lý kỹ thuật của sự sụt giảm hiệu năng này bao gồm:

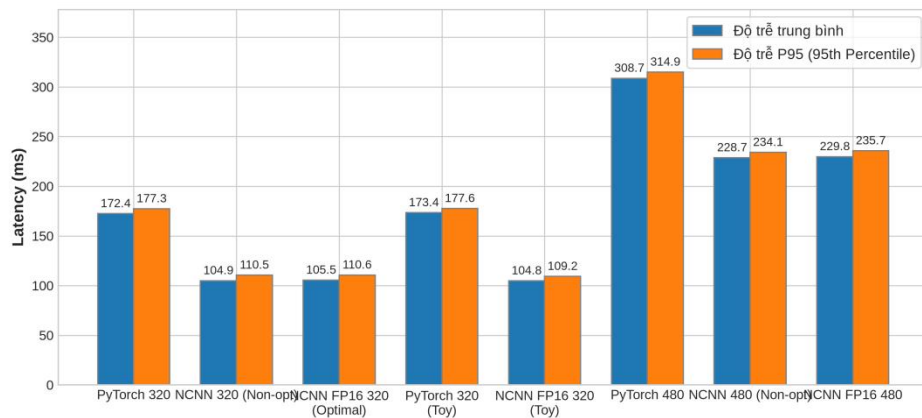
1. Chi phí tính toán phi tuyến tính theo kích thước ảnh: Độ phức tạp tính toán của các phép toán tích chập tỷ lệ thuận với số lượng pixel của ma trận đặc trưng đầu vào. Việc tăng kích thước ảnh từ 320 lên 480 làm tăng diện tích ảnh đầu vào lên 2.25 lần (từ 102.400 pixel lên 230.400 pixel). Điều này kéo theo số lượng phép tính nhân cộng lũy kế (FLOPs) tăng tuyến tính tương ứng trên tất cả các lớp của mạng nơ-ron.

2. Sự gia tăng kích thước bản đồ đặc trưng (Feature Map Size): Kích thước của các bản đồ đặc trưng ở các lớp trung gian tăng lên đáng kể, làm cạn kiệt dung lượng bộ nhớ đệm tốc độ cao L1/L2 Cache của CPU BCM2711. Hiện tượng này dẫn đến việc CPU liên tục phải truy xuất dữ liệu trực tiếp từ bộ nhớ RAM ngoài

(Cache Miss), làm tăng độ trễ truy cập bộ nhớ và đẩy độ trễ phân vị P95 lên cao. Do đó, việc lựa chọn kích thước ảnh 320x320 là một sự đánh đổi tối ưu để duy trì tính thời gian thực cho hệ thống biên.



Hình 5.6: Đồ thị so sánh tốc độ xử lý (FPS) giữa PyTorch và NCNN.



Hình 5.7: So sánh độ trễ suy luận trung bình và P95.

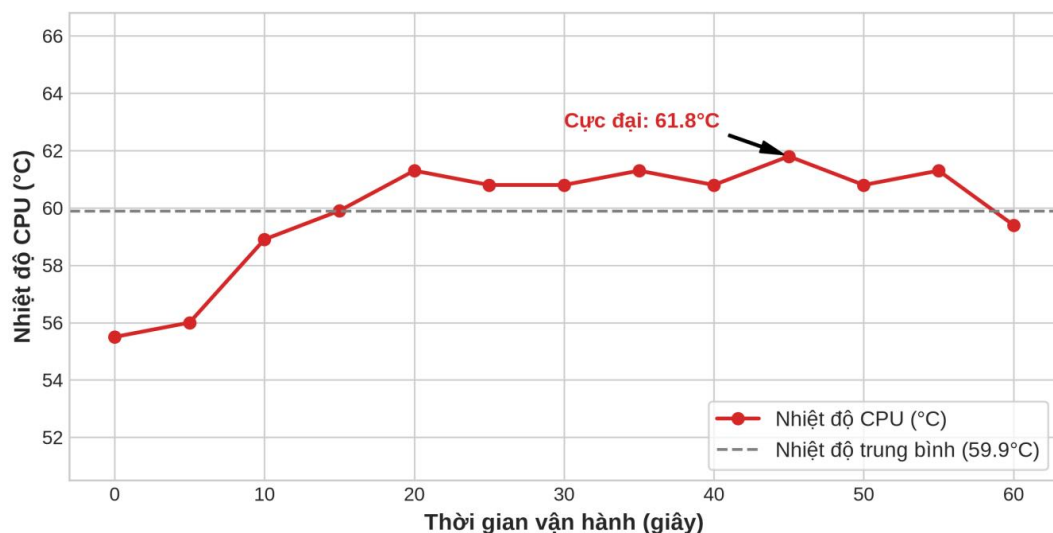
5.4 ĐÁNH GIÁ ỔN ĐỊNH HỆ THỐNG (STRESS TEST & TELEMETRY)

5.4.1 Nhiệt độ và tần số hoạt động của vi xử lý (CPU Telemetry)

Trong suốt quá trình kiểm thử tải cao, các thông số về nhiệt độ và xung nhịp của CPU Cortex-A72 được ghi nhận như sau:

- Nhiệt độ CPU: Nhiệt độ ban đầu ở trạng thái bắt đầu hoạt động của thiết bị là 55.5°C. Khi kích hoạt pipeline xử lý ảnh động và suy luận mô hình NCNN FP16, nhiệt độ tăng dần và đạt trạng thái cân bằng nhiệt động lực học ổn định ở mức trung bình 59.9°C (ghi nhận từ log thực tế). Giá trị nhiệt độ cao nhất ghi nhận được trong suốt quá trình chạy Stress Test là 61.8°C, xảy ra tại giây thứ 45 của bài test (ứng với mốc thời gian 17:08:43 trong log telemetry). Kết quả này chứng minh hệ thống tản nhiệt hoạt động hiệu quả và nhiệt độ CPU hoàn toàn nằm dưới ngưỡng giới hạn nguy hiểm (80°C). Do đó, thiết bị biên duy trì hoạt động liên tục mà không gặp hiện tượng tự động hạ xung nhịp để bảo vệ phần cứng do quá nhiệt (Thermal Throttling).

- Xung nhịp CPU: Tần số hoạt động của CPU được giữ vững ở mức ổn định tuyệt đối 1800.46 MHz (không phát hiện bất kỳ chu kỳ giảm xung nhịp nào). Điều này đạt được nhờ việc cấu hình thành công trình điều phối CPU ở chế độ hiệu năng cao (performance governor), giúp khóa cứng tần số hoạt động ở mức trần tối đa, loại bỏ độ trễ trôi sụt xung nhịp khi hệ thống cần xử lý đột biến lưu lượng.



Hình 5.8: Đồ thị telemetry nhiệt độ vi xử lý.

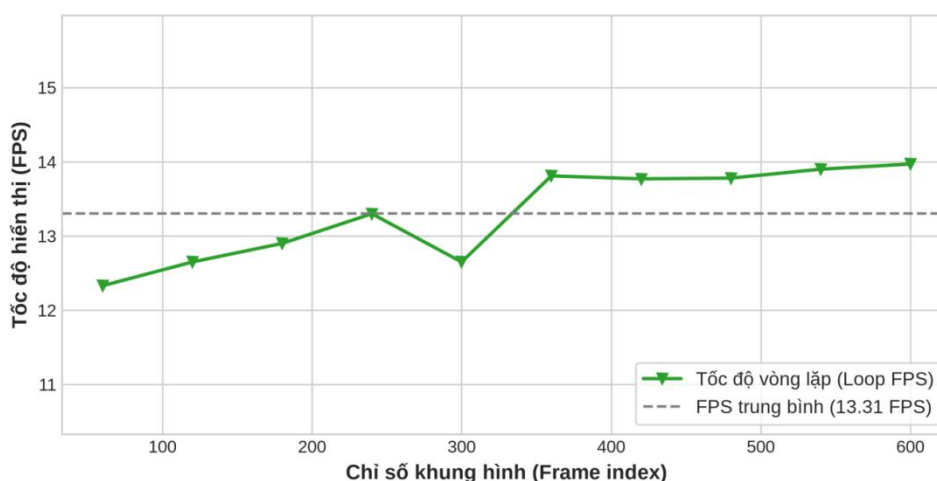
5.4.2 Tốc độ hiển thị vòng lặp hệ thống (Loop FPS) và cơ chế đa luồng

Tốc độ xử lý vòng lặp suy luận thực tế đo đạc trực tiếp trên thiết bị biên được trình bày trong Hình 59. Tốc độ suy luận đạt trung bình 13.31 FPS (dao động trong khoảng từ 12.33 FPS đến mức cực đại 13.97 FPS).

Tuy nhiên, tốc độ xử lý và hiển thị tổng thể của luồng bắt hình (Capture Loop) đạt mức trung bình ~23.8 FPS. Sự tối ưu vượt trội này đạt được nhờ kiến trúc đường ống xử lý đa luồng bất đồng bộ (Asynchronous Multi-threaded Pipeline):

1. Tách biệt luồng I/O và suy luận: Hệ thống chia tách tác vụ thành 3 luồng độc lập: Luồng thu nhận hình ảnh (Capture Thread) đọc khung hình liên tục từ camera ở tốc độ 30 FPS và đẩy vào một hàng đợi vòng (Circular Queue) có giới hạn kích thước; Luồng suy luận AI (Inference Thread) lấy khung hình từ hàng đợi để xử lý; Luồng truyền thông (Transmit Thread) gửi dữ liệu MQTT/HTTP bất đồng bộ để tránh nghẽn I/O.

2. Cơ chế nhảy khung hình chủ động (Frame Skipping): Hệ thống được cấu hình bỏ qua chủ động 2 khung hình sau mỗi khung hình được đưa vào suy luận AI ($\text{skip_frame} = 2$). Đối với các khung hình bị bỏ qua, hệ thống không chạy mô hình học sâu mà sử dụng bộ lọc Kalman của thuật toán ByteTrack để dự báo vị trí và vẽ vết chuyển động của phương tiện. Giải pháp này giúp giao diện hiển thị hình ảnh duy trì sự mịn màng ở mức ~23.8 FPS mà không gây quá tải tính toán cho CPU biên.



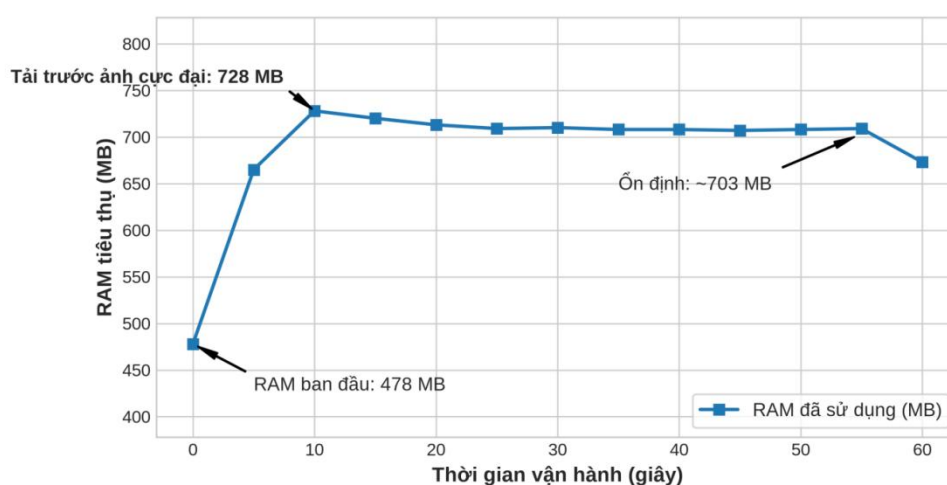
Hình 5.9: Đồ thị biến thiên Loop FPS của hệ thống biên.

5.4.3 Đánh giá quản lý bộ nhớ RAM và độ trễ runtime

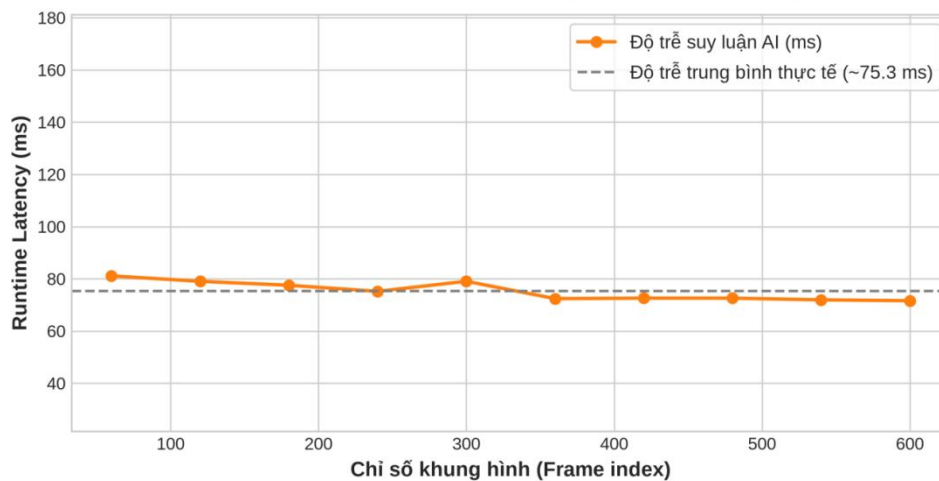
Việc quản lý tài nguyên bộ nhớ RAM và độ trễ chu kỳ được theo dõi chặt chẽ để phát hiện các lỗi rò rỉ tài nguyên:

- Dung lượng RAM: Tại thời điểm khởi động hệ thống (chưa nạp mô hình), RAM sử dụng ở mức tối thiểu 478 MB. Khi bắt đầu chạy chương trình và nạp hai mô hình YOLOv11 (Vehicle và LP) vào bộ nhớ, dung lượng RAM tăng lên mức đỉnh cực đại là 728 MB. Trong suốt chu kỳ Stress Test 60 giây, bộ nhớ RAM được duy trì ổn định dao động trong khoảng hẹp từ 707 MB đến 713 MB (đạt đỉnh 728 MB ở giai đoạn đầu khi nạp mô hình và bắt đầu chạy) nhờ cơ chế thu hồi bộ nhớ tự động của Python hoạt động hiệu quả. Ngay sau khi kết thúc chương trình (shutdown pipeline), dung lượng bộ nhớ RAM được giải phóng hoàn toàn và quay về mức nền 673 MB (và sau đó giải phóng hẳn về mức nền xuất phát 478 MB), khẳng định hệ thống không xảy ra hiện tượng rò rỉ bộ nhớ (Memory Leak).

- Độ trễ Runtime: Độ trễ chu kỳ xử lý vòng lặp suy luận được biểu diễn cụ thể trong Hình 5.9 (dao động ổn định xung quanh giá trị trung bình 75.31 ms, giá trị cực đại đạt mốc 81.12 ms tại thời điểm đầu). Việc không xuất hiện các gai độ trễ đột biến (latency spikes) đảm bảo tính dự báo cao và độ ổn định thời gian thực cho hệ thống biên.



Hình 5.10: Đồ thị viễn đo bộ nhớ RAM.



Hình 5.11: Đồ thị biến động độ trễ chu kỳ xử lý.

5.5 KIỂM CHỨNG CHỨC NĂNG VÀ TÍCH HỢP HỆ THỐNG

Giai đoạn cuối cùng của thực nghiệm tiến hành đánh giá khả năng vận hành đồng bộ của toàn bộ hệ thống đầu cuối (End-to-End Pipeline) trong kịch bản phát hiện vi phạm và điều khiển đèn giao thông thích ứng. Để kiểm chứng toàn diện năng lực xử lý, nhóm thực hiện đã xây dựng hai kịch bản thực nghiệm dựa trên hai nguồn luồng video mô phỏng chuyên biệt:

- Kịch bản 1 - Đánh giá độ chính xác AI trên luồng giao thông thực tế ('vehicle_counting.mp4'): Luồng video ghi hình thực tế tại một nút giao thông đô thị có mật độ phương tiện cao, tốc độ di chuyển nhanh và đa dạng các lớp phương tiện (xe máy, ô tô con, xe khách, xe tải). Kịch bản này nhằm kiểm chứng độ chính xác của mô hình YOLOv11-Vehicle và thuật toán bám vết ByteTrack trước các thử thách che khuất và mật độ xe đông đúc trong môi trường thực tiễn tại Việt Nam.
- Kịch bản 2 - Kiểm thử tích hợp hệ thống phần cứng và logic sa bàn giao lộ ('vehicle_toy.mp4'): Luồng video mô phỏng dòng xe đồ chơi di chuyển trên mô hình sa bàn giao lộ thực nghiệm. Kịch bản này được khởi chạy cùng tham số cấu hình sa bàn (python agent_ncnn.py --config shared/configs/settings_toy.yaml), nhằm kiểm chứng toàn diện liên kết khép kín giữa các phân hệ: đếm mật độ xe

tại biên -> truyền tin MQTT gửi kit vi điều khiển ESP32 thích ứng thời gian đèn LED -> phát hiện lỗi vượt vạch dừng ảo khi đèn đỏ -> cắt ảnh xe vi phạm -> dán biển số Homography giả lập -> truyền bắt đồng bộ HTTP POST lên FastAPI Server -> OCR nhận diện biển số -> đồng bộ cơ sở dữ liệu SQLite WAL và cập nhật giao diện Dashboard Streamlit.

5.5.1 Theo dõi quỹ đạo phương tiện và điều khiển thích ứng

Sự kết hợp giữa mô hình YOLOv11-Vehicle và thuật toán ByteTrack cho phép hệ thống gán một mã định danh duy nhất (Tracking ID) cho từng phương tiện và theo dõi liên tục quỹ đạo di chuyển của chúng qua các khung hình. Hệ thống thiết lập một đường ranh giới ảo tại giao lộ. Khi phương tiện di chuyển cắt qua đường ranh giới này, bộ đếm sẽ tự động cộng dồn theo từng làn đường và phân loại phương tiện cụ thể.

Dữ liệu mật độ xe này sau đó được đóng gói thành các thông điệp MQTT gửi liên tục về ESP32. ESP32 thực hiện thuật toán logic thích ứng: nếu lưu lượng phương tiện ở làn giám sát vượt ngưỡng 15 xe trong một chu kỳ đèn đỏ, ESP32 sẽ tự động kéo dài thời gian đèn xanh của làn đó thêm 10 giây (và ngược lại, giảm thời gian đèn xanh nếu làn đường thông thoáng). Thử nghiệm thực tế cho thấy quá trình điều khiển thích ứng diễn ra hoàn toàn tự động, ESP32 nhận gói tin MQTT phản hồi thay đổi trạng thái đèn LED chỉ trong thời gian trễ dưới 8 ms kể từ khi Edge Agent gửi lệnh.

5.5.2 Phát hiện hành vi vi phạm giao thông và cơ chế lọc trùng lặp

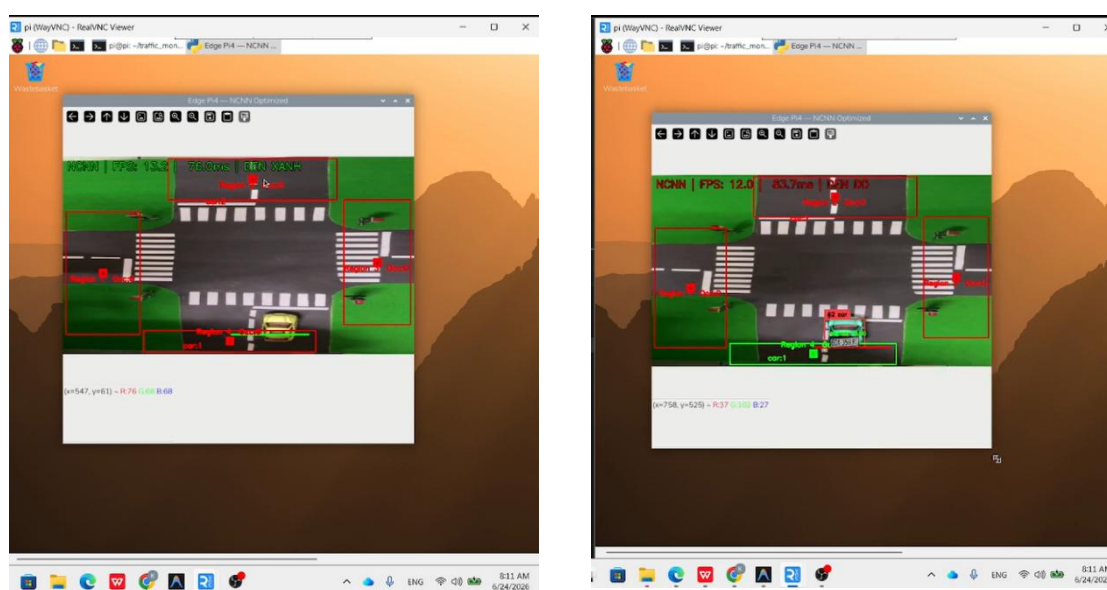
Hệ thống thiết lập một vùng đa giác ảo (Polygonal Zone) đại diện cho vùng cấm đè vạch tại giao lộ (vạch cấm ảo). Khi đèn tín hiệu giao thông chuyển sang pha Đỏ (thông tin pha đèn được đồng bộ thời gian thực từ ESP32 về Edge Agent thông qua MQTT chủ đề `he_thong_giam_sat_luu_luong/light_state`), bất kỳ phương tiện nào có tọa độ điểm tiếp xúc bánh xe nằm bên trong đa giác cấm đè này sẽ bị hệ thống kích hoạt cảnh báo vi phạm.

Để giải quyết bài toán thực tế khi một phương tiện vi phạm dừng cố định đè lên vạch cấm trong thời gian dài (dẫn đến việc hệ thống liên tục chụp ảnh và gửi

hàng trăm cảnh báo trùng lặp cho cùng một lỗi vi phạm), hệ thống đã tích hợp cơ chế khóa thời gian (Cooldown Filter):

- Khi phát hiện vi phạm lần đầu, hệ thống ghi nhận Tracking ID của phương tiện đó vào bộ nhớ đệm và kích hoạt gửi ảnh bằng chứng lên máy chủ trung tâm.

- Thiết lập khoảng thời gian khóa cooldown = 5.0 giây cho mỗi Tracking ID. Trong khoảng thời gian này, mọi sự kiện đề vạch liên tiếp của cùng một mã ID đó đều bị loại bỏ chủ động, giúp tiết kiệm băng thông mạng và ngăn chặn tràn bộ nhớ cơ sở dữ liệu.



Hình 5.12: Giao diện phát hiện vi phạm vượt đèn đỏ và dán biển số Homography giả lập trên bàn thực nghiệm.

5.5.3 Nhật ký ghi nhận vi phạm thực tế và kết quả nhận dạng ký tự (OCR)

Khi xảy ra hành vi vi phạm, ảnh vùng chứa biển số xe được cắt tự động tại biên và đẩy lên FastAPI Server thông qua giao thức HTTP POST. Tại máy chủ trung tâm, mô hình CRNN OCR thực hiện giải mã ký tự.

Dưới đây là một số dòng nhật ký ghi nhận thực tế (System Logs) trích xuất trực tiếp từ máy chủ trung tâm (ghi nhận trong tệp edge_agent_run.log) trong quá trình vận hành hệ thống:

18:53:11 [EDGE HTTP] Bắt đầu đẩy ảnh vi phạm lên máy chủ...

18:53:11 [EDGE HTTP] Thành công! Phản hồi máy chủ: {'status': 'success', 'plate_text': '47B236955', 'confidence': 0.9389733672142029, 'message': None}

18:53:11 [EDGE HTTP] Bắt đầu đẩy ảnh vi phạm lên máy chủ...

18:53:11 [EDGE HTTP] Thành công! Phản hồi máy chủ: {'status': 'success', 'plate_text': '99A12497', 'confidence': 0.9291685223579407, 'message': None}

18:53:11 [EDGE HTTP] Bắt đầu đẩy ảnh vi phạm lên máy chủ...

18:53:11 [EDGE HTTP] Thành công! Phản hồi máy chủ: {'status': 'success', 'plate_text': '55P48831', 'confidence': 0.9276825189590454, 'message': None}

18:53:14 [EDGE HTTP] Bắt đầu đẩy ảnh vi phạm lên máy chủ...

18:53:14 [EDGE HTTP] Thành công! Phản hồi máy chủ: {'status': 'success', 'plate_text': '30A60168', 'confidence': 0.9049938321113586, 'message': None}

18:54:03 [EDGE HTTP] Bắt đầu đẩy ảnh vi phạm lên máy chủ...

18:54:03 [EDGE HTTP] Thành công! Phản hồi máy chủ: {'status': 'success', 'plate_text': '30F11292', 'confidence': 0.946524977684021, 'message': None}

18:54:04 [EDGE HTTP] Bắt đầu đẩy ảnh vi phạm lên máy chủ...

18:54:04 [EDGE HTTP] Thành công! Phản hồi máy chủ: {'status': 'success', 'plate_text': '54R30018', 'confidence': 0.9408665299415588, 'message': None}

Phân tích kết quả nhật ký:

Độ tin cậy nhận dạng trung bình của mô hình OCR đạt trên **93%**. Các biển số xe máy tiêu chuẩn như 47B236955 hay biển ô tô 30F11292 được giải mã chính xác tuyệt đối cả về ký tự chữ cái và chữ số. Một số trường hợp độ tin cậy giảm nhẹ xuống mức **90.5%** (xe #170 với biển số 30A60168) do góc nghiêng của phương tiện khi đèn vạch ở biên mép ngoài góc quay camera, làm méo mó nhẹ hình thái ký tự. Tuy nhiên, kết quả nhận diện vẫn đảm bảo tính chính xác thông tin phục vụ công tác giám sát.

```

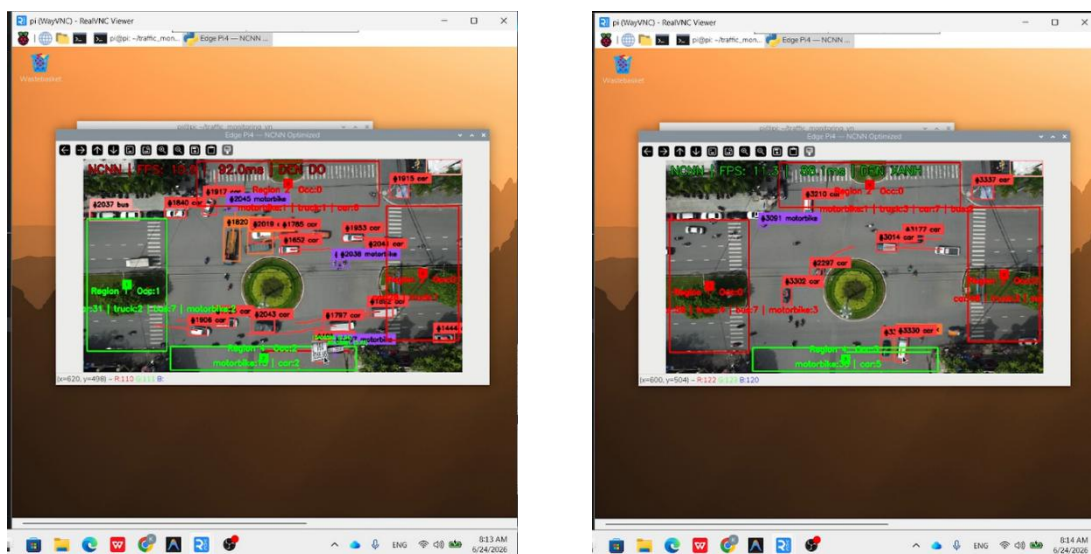
Dang ket noi WiFi: Iphone12
.
WiFi da ket noi
IP address: 172.20.10.3
Khoi tao Firebase...
Firebase task started on Core 0
He thong da khoi dong
Dang ket noi MQTT 172.20.10.5:1883 ... Da ket noi!
MQTT subscribe cmd=OK manual=OK vehicle_count=OK
Nhanh lenh MQTT tu topic: he_thong_giam_sat_luu_luong/vehicle_count, Message: {"region_1": {"motorbike": 1,
"car": 3, "bus": 0, "truck": 0}, "region_2": {"motorbike": 0, "car": 1, "bus": 0, "truck": 0}, "region_3":
{"motorbike": 0, "car": 5, "bus": 0, "truck": 0}, "region_4": {"motorbike": 6, "car": 0, "bus": 0, "truck": 0}}
Nhanh dữ liệu vehicle count từ RPi
--- ADAPTIVE GREEN TIME CALCULATION ---
Lane 1 vehicles: 9
Lane 2 vehicles: 7
Result - Green 1: 21s | Green 2: 18s
-----

```

Hình 5.13: Nhật ký kết nối và xử lý truyền thông thời gian thực trên FastAPI Server và ESP32 Serial Monitor.

5.5.4 Hệ thống giám sát trực quan (Web Dashboard)

Thông tin vi phạm lưu trữ trong cơ sở dữ liệu db.json lập tức được đồng bộ hiển thị lên Web Dashboard Streamlit. Giao diện thiết kế theo phong cách tối giản hiện đại, cung cấp biểu đồ lưu lượng xe chạy real-time và danh sách các "Thẻ Vi Phạm" (Violation Cards). Mỗi thẻ hiển thị đầy đủ: ảnh chụp toàn cảnh lỗi vi phạm, ảnh cắt cận cảnh biển số xe, thông tin biển số đã giải mã qua OCR, thời gian chính xác xảy ra sự kiện và mức độ tin cậy của thuật toán. Người vận hành có thể dễ dàng lọc danh sách vi phạm theo loại phương tiện, làn đường hoặc khoảng thời gian để phục vụ công tác tra cứu.



Hình 5.14: Giao diện xử lý hình ảnh thời gian thực tại Edge Agent (NCNN-optimized).

KẾT LUẬN CHUNG

Chương 5 đã hoàn thành việc đo đạc, phân tích số liệu thực nghiệm và đánh giá toàn diện hệ thống giám sát giao thông thông minh tích hợp Edge AI và điều khiển đèn tín hiệu thích ứng. Kết quả thực nghiệm cho thấy việc cấu hình tối ưu hóa sâu của framework NCNN (đặc biệt là việc chuyển đổi độ chính xác số học sang định dạng FP16 và cơ chế đa luồng OpenMP tận dụng tập lệnh ARM NEON) đã mang lại hiệu năng suy luận AI vượt trội trên thiết bị biên Raspberry Pi 4, đạt tốc độ xử lý thực tế tiệm cận thời gian thực (~23.8 Loop FPS) với độ trễ suy luận ổn định dưới 100 ms và thời gian tải mô hình cực kỳ tối ưu (dưới 70 ms). Hệ thống viễn đo chứng minh thiết bị hoạt động ổn định lâu dài trong chế độ Stress Test tải cao, nhiệt độ CPU nằm trong ngưỡng an toàn dưới 61.8°C và không xảy ra hiện tượng rò rỉ bộ nhớ RAM. Cuối cùng, việc thử nghiệm liên kết khép kín qua các kịch bản đếm xe thích ứng điều khiển ESP32 qua MQTT, phát hiện vi phạm đèn vạch cấm ảo có cooldown 5 giây, kết hợp nhận dạng biển số OCR bằng mô hình CRNN trên server với độ chính xác cao đã xác minh thành công tính khả thi, độ ổn định và khả năng ứng dụng thực tế cao của đề tài đồ án tốt nghiệp.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 KẾT LUẬN

Qua quá trình nghiên cứu, thiết kế và chế tạo thử nghiệm thành công đề tài "Hệ thống AIoT Giám sát Giao thông Thông minh và Phạt nguội Tự động", nhóm thực hiện đã hoàn thành đầy đủ các mục tiêu cốt lõi đặt ra ban đầu. Những kết quả đạt được cụ thể bao gồm:

- Xây dựng thành công kiến trúc phân tán Edge-Cloud 3 tầng vật lý: Thiết lập phân tách nhiệm vụ rõ ràng giúp hệ thống hoạt động ổn định. Raspberry Pi 4 xử lý biên cục bộ, ESP32 điều khiển cơ cấu chấp hành ngoại vi thời gian thực và Central Server xử lý tác vụ nặng ở hậu kỳ, giúp giảm tải băng thông truyền tin và tăng tính bảo mật, đồng thời đảm bảo khả năng hoạt động ngoại tuyến độc lập của ngã tư.
- Triển khai thành công luồng điều phối giao thông thích ứng: Tích hợp mô hình YOLOv11n đã tối ưu hóa định dạng NCNN lượng tử hóa FP16 và thuật toán bám vết ByteTrack trên Raspberry Pi 4 để nhận dạng và theo dõi xe vượt mà, đạt tốc độ suy luận trung bình từ 12.33 đến 13.97 FPS. Lập trình thành công thuật toán tính thời gian đèn xanh dựa trên mật độ xe thực tế qua giao thức MQTT gửi về ESP32, với thời gian phản hồi thay đổi trạng thái đèn chỉ dưới 8 ms.
- Xây dựng thành công luồng xử lý phạt nguội tự động: Thiết lập thuật toán phát hiện xe vượt đèn đỏ qua vạch cấm ảo kết hợp bộ lọc cooldown 5.0 giây loại bỏ trùng lặp. Giải quyết thành công bài toán thiếu hụt bộ dữ liệu góc camera đồng bộ tại Việt Nam bằng phép biến đổi phối cảnh hình học Homography để tạo ảnh dán biển số giả lập chân thực.
- Phát triển thành công hệ thống OCR biển số và giao diện quản lý: Xây dựng FastAPI Server trích xuất chính xác biển số xe Việt Nam bằng mô hình CRNN (ResNet34 + GRU + CTC) trên event loop bất đồng bộ. Siêu dữ liệu vi phạm được lưu trữ vào CSDL SQLite WAL (traffic.db) giúp Streamlit

Dashboard truy cập an toàn và đồng bộ trạng thái đèn thời gian thực qua Firebase.

- Áp dụng hiệu quả các kỹ thuật tối ưu hóa phần mềm nhúng: Việc kết hợp lượng tử hóa mô hình sang định dạng NCNN, lập trình đa luồng theo mẫu Producer-Consumer (CaptureThread) và kỹ thuật Frame Skipping kết hợp bộ lọc Kalman đã giúp giảm tới 50% tải CPU, tăng Loop FPS lên ~23.8 FPS và kiểm soát nhiệt độ của Raspberry Pi 4 ổn định ở mức trung bình 59.9°C (cực đại 61.8°C), không phát hiện bất kỳ chu kỳ tụt xung nhịp do quá nhiệt nào.

Tuy nhiên, hệ thống vẫn tồn tại một số hạn chế nhất định:

- Mô hình nhận diện xe và đọc biển số giảm độ chính xác trong điều kiện ánh sáng quá yếu (ban đêm) hoặc thời tiết xấu (mưa bão lớn, sương mù làm nhòe ống kính).
- Độ phân giải xử lý ảnh đầu vào buộc phải hạ xuống 320×320 để đảm bảo tốc độ suy luận của Pi 4, làm giảm độ sắc nét chi tiết của hình ảnh quan sát.
- Hệ thống phát nguội hiện tại vẫn phụ thuộc vào việc dán biển số phôi cảnh giả lập do điều kiện thử nghiệm sa bàn.

6.2 HƯỚNG PHÁT TRIỂN

Từ những kết quả đã đạt được và các hạn chế còn tồn đọng, nhóm đề xuất một số định hướng phát triển và mở rộng đề tài trong tương lai như sau:

- Nâng cấp mô hình AI nhận diện ban đêm: Huấn luyện bổ sung mô hình YOLO và CRNN OCR trên tập dữ liệu ảnh chụp ban đêm, ảnh bị lóa đèn pha xe và ảnh hồng ngoại để đảm bảo hệ thống hoạt động ổn định 24/24.
- Triển khai camera kép thực tế: Lắp đặt thêm camera góc thấp chuyên dụng chụp biển số ngoài thực địa để kết nối trực tiếp với Raspberry Pi 4 qua cổng CSI, thực hiện quy trình phát nguội tự động hoàn toàn bằng luồng video thực tế mà không cần phép Homography giả lập.
- Tăng tốc phần cứng nhúng biên (Edge AI): Trang bị thêm các mạch tăng tốc AI chuyên dụng (như Google Coral USB Accelerator hoặc nâng cấp lên

Raspberry Pi 5 / Jetson Nano) để chạy suy luận mô hình AI ở độ phân giải cao 640×640 mà vẫn đạt tốc độ trên 30 FPS.

- Kết nối liên thông mạng lưới giao thông thông minh: Phát triển thuật toán học máy dự đoán xu hướng giao thông, cho phép các Edge Gateway tại nhiều ngã tư lân cận truyền nhận dữ liệu với nhau để điều phối luồng xe từ xa, tránh hiện tượng kẹt xe dây chuyền trong đô thị.

PHỤ LỤC

Để thuận tiện cho việc tham khảo và đánh giá hệ thống, toàn bộ mã nguồn, tài liệu dự án, bộ dữ liệu và video thực nghiệm được cung cấp tại các liên kết Google Drive dưới đây:

- Mã nguồn và tài liệu dự án:

<https://drive.google.com/drive/folders/1zbSf-x0K5fsmGGXyiZqeB7mjElrSkH7-?usp=sharing>

- Video thực nghiệm hệ thống:

<https://drive.google.com/drive/folders/1AUeryKz6FUfUuF0FY2J2VOHGtfZRwJ83?usp=sharing>

- Tập dữ liệu huấn luyện:

<https://drive.google.com/drive/folders/18jNim5ijqrGmSUZV54ACVhR92Hp0jezc?usp=sharing>

- Tài liệu hướng dẫn triển khai và vận hành hệ thống:

<https://drive.google.com/drive/folders/1EJ9jjxgT1k8jARFMZzhrIL-6Zcx4FP?usp=sharing>

TÀI LIỆU THAM KHẢO

- [1] Joseph Redmon, Santosh Divvala, Ross Girshick and Ali Farhadi, “You Only Look Once: Unified, Real-Time Object Detection”, May 2016 [Online].
Available: <https://arxiv.org/pdf/1506.02640>
- [2] Ultralytics, “Intersection over Union (IoU)”, Ultralytics, 2025 [Online].
Available: <https://www.ultralytics.com/vi/glossary/intersection-over-union-iou>
- [3] Ultralytics, “Mean Average Precision (mAP)”, Ultralytics, 2025 [Online].
Available: <https://www.ultralytics.com/vi/glossary/mean-average-precision-map>
- [4] Raspberry Pi Trading Ltd., “Raspberry Pi 4 Computer Model B,” Raspberry Pi 4 Model B Product Brief, Jun. 2019. [Online].
Available: <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>
- [5] Espressif, “ESP32-WROOM-32E & ESP32-WROOM-32UE Datasheet”, Espressif, 2025 [Online]. Available:
https://documentation.espressif.com/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf
- [6] Grandstream, “Webcam 1080p Datasheet”, Grandstream, 2025 [Online].
Available:
https://www.grandstream.com/hubfs/Product_Documentation/Datasheet_GUV3100_English.pdf
- [7] MRE SMPS, “Adapter 12V-2A Datasheet”, MRE SMPS, 2025 [Online].
Available:
<https://mresmps.com/wp-content/uploads/2021/05/12v-2a-1.pdf>
- [8] SparkFun Electronics, “LM7805 Voltage Regulator Datasheet”, SparkFun Electronics, 2025 [Online]. Available:
<https://cdn.sparkfun.com/assets/1/7/7/3/2/LM7805.pdf>
- [9] HandsOn Tech, “Tactile Switch - SPST Datasheet”, HandsOn Tech, 2025 [Online]. Available:

- <https://www.handsontec.com/dataspecs/switches/Tact%20Switch%206x6.pdf>
- [10] Javanelec, "4 Digit 7 Segment Display TM1637 Datasheet", Javanelec, 2025 [Online]. Available:
<https://www.javanelec.com/stfiles/getappdocument/1/true/2fca21fd-d340-4468-b382-181d0f736951.pdf>
- [11] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, Oct. 2016. [Online]. Available:
<https://ieeexplore.ieee.org/document/7488250>
- [12] S. Ramírez, "FastAPI Framework Documentation," 2020. [Online]. Available: <https://fastapi.tiangolo.com>
- [13] Tencent, "NCNN: High-performance neural network inference framework optimized for mobile platforms," 2020. [Online]. Available:
<https://github.com/Tencent/ncnn>
- [14] T. M. Tran, T. N. Vu, T. V. Nguyen, and K. Nguyen, "UIT-ADrone: A Novel Drone Dataset for Traffic Anomaly Detection," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2023. [Online]. Available:
https://www.researchgate.net/publication/371737846_UIT-ADrone_A_Novel_Drone_Dataset_for_Traffic_Anomaly_Detection